

用神经物理构建世界模型

Building World Models with Neural Physics

Pingchuan Ma

博士论文 · Massachusetts Institute of Technology · 2025

Model: gpt-6.1-terra（本次续修；早期待核）

审核：曹溪能

Updated: 2026-11-05

译本说明

本工作稿依据 Pingchuan Ma 于 2025 年提交的麻省理工学院博士论文翻译. 全书译稿已形成，本次修订及译本仍待最终验收. 原文页码与译文页码不一定相同. 本次续修使用 gpt-6.1-terra；早期初译模型尚待核实. 封面审核字段按项目标准预置，不表示已完成审核.

论文信息

用神经物理构建世界模型

Pingchuan Ma
南开大学工学学士（2019）
麻省理工学院理学硕士（2023）

本论文提交麻省理工学院电气工程与计算机科学系，作为取得哲学博士学位的部分要求. 2025 年 2 月.

© 2025 Pingchuan Ma. 保留所有权利. 作者在此授予麻省理工学院非排他性的、全球范围内的、不可撤销的、免版税的许可，使其能够行使版权项下的任何及全部权利，包括复制、保存、分发和公开展示本论文的副本，或依据开放获取许可发布本论文.

作者：Pingchuan Ma，电气工程与计算机科学系，2025 年 1 月 20 日.
认证：Wojciech Matusik，电气工程与计算机科学教授，论文导师.
接受：Leslie A. Kolodziejski，电气工程与计算机科学教授，研究生委员会主席.

摘要

世界模型以数据驱动的方式学习环境的动力学；由于能够以较低成本进行仿真且具有可微性，它们可以提高控制、设计、识别和生成等下游任务的性能与效率. 理想情况下，预训练的世界模型应当能够准确模拟真实动力学、轻松适应新的配置，并推广至多种物理效应. 此前在这一领域的尝试，要么使用基于模型的可微物理方法，但只开放少量参数；要么针对特定场景训练，并且只融入极少的物理先验. 这些世界模型未能达到上述目标，因而限制了它们在现实世界中的 accurate-critic 部署上的适用性，也限制了向更大型预训练世界模型扩展的能力.[1]

本论文旨在利用神经物理构建世界模型. 神经物理是一种神经网络与物理相结合的框架：它用可微物理对基本动力学建模，同时通过神经网络学习所有其他模块. 通过引入神经物理，世界模型能够紧密遵循物理原理，同时高效地学习多种效应. 神经物理的模块化结构使世界模型只需装入不同的预训练神经模块，便可推广到新的配置. 本论文将在重建、机器人控制和科学发现等应用中展示这一新框架的有效性.

本论文于 2025 年 1 月 20 日提交电气工程与计算机科学系，作为取得哲学博士学位的部分要求.

论文导师：Wojciech Matusik

职称：电气工程与计算机科学教授

[1]译注：原文使用非标准表达 `accurate-critic deployments`，其确切含义未获作者证据确认；此处保留英文，不将其确定解释为特定的评估器部署.

先前发表的材料

第 2 章修订自先前发表的论文 [113]：Pingchuan Ma, Tao Du, Joshua B. Tenenbaum, Wojciech Matusik, and Chuang Gan. "RISP: Rendering-Invariant State Predictor with Differentiable Simulation and Rendering for Cross-Domain Parameter Estimation." In *International Conference on Learning Representations*. 2021.

第 3 章修订自先前发表的论文 [115]：Pingchuan Ma, Peter Yichen Chen, Bolei Deng, Joshua B. Tenenbaum, Tao Du, Chuang Gan, and Wojciech Matusik. "Learning Neural Constitutive Laws from Motion Observations for Generalizable PDE Dynamics." In *International Conference on Machine Learning*. 2023.

第 4 章修订自先前的稿件 [116]：Pingchuan Ma, Tsun-Hsuan Wang, Minghao Guo, Zhiqing Sun, Joshua B. Tenenbaum, Daniela Rus, Chuang Gan, and Wojciech Matusik. "LLM and Simulation as Bilevel Optimizers: A New Paradigm to Advance Physical Scientific Discovery." In *International Conference on Machine Learning*. 2024.

致谢

攻读博士学位是一段充满挑战而又收获丰厚的旅程，幸运的是，一路上我都有支持我的人相伴.

首先，我要向我的论文导师和研究指导教师 Wojciech Matusik 教授致以诚挚的谢意. Wojciech 一直鼓励我把研究看作开启新机遇的途径，而不仅仅是一个项目；这一视角深刻影响了我的研究品味. 我们几乎每天都会花大量时间讨论、甚至争论研究的未来. 在调整研究方向和做出重大的职业决定时，我从他的智慧中获益良多. 他还训练我自信地面对批评、捍卫自己的研究想法，使我在报告和交锋时更有勇气. 博士生涯在人生中相对短暂，我非常幸运能够有机会从 Wojciech 身上汲取知识.

我也感谢 Daniela Rus 教授和 Kaiming He 教授担任我的论文委员会成员. 我与 Daniela 的合作可以追溯到我在麻省理工学院发表的第一篇论文. 她渊博的知识和对机器人的热情帮助我在整个博士生涯中 “保持梯度”. Kaiming 是一位洞察力敏锐的导师，指导我将这篇论文提升到更高的层次. 他向我传授了对学习与物理的深刻理解和远见，使我的思考超越了本论文的范围. 我由衷希望将来有更多合作机会.

我还感谢在博士生涯中与我并肩工作的导师们. Tao Du 教授是实验室的师兄，在我最初的几个项目中给予了指导. 他非常注重实践，教会了我许多成为优秀研究者的宝贵经验，从发现并阐明一个引人入胜的问题，到寻找优雅的解决方案. 在学术之外，他也是一位好朋友，分享见解并帮助我在人生中做出明智的决定. 另一位师兄 Jie Xu 博士也是我在 NVIDIA 西雅图机器人实验室的导师. 他与我紧密合作完成实习项目，提供指导，并让我对机器人研究的整个流程有了全面认识. Chuang Gan 教授是我加入麻省理工学院后第一次实习时在 MIT-IBM Watson AI Lab 的导师. 我们从那里开始合作，并一直合作至今，这对我在该领域的成长和经验积累贡献巨大. 我还要感谢本科时期的导师：Bo Ren 教授、Zherong Pan 博士、Dinesh Manocha 教授、Yao Zhou 和 Ping Luo 教授. 他们对当时尚为初学者的我的耐心和包容，对我后来进入麻省理工学院至关重要.

我要感谢其他所有合作者：Stan Birchfield, Peter Yichen Chen, Bolei Deng, Yilun Du, Dieter Fox, Yuchao Gu, Minghao Guo, Ankur Handa, Changyu Hu, Josie Hughes, Joshua Jacob, Robert Katzschmann, Byungchul Kim, Yichen Li, Yifei Li, Chunru Lin, Ziming Liu, Mike Michelis, Yashraj Narang, Elvis Nava, Crystal Owens, Juan Salazar, Eftychios Sifakis, Andrew Spielberg, Shinjiro Sueda, Yuchen Sun, Zhiqing Sun, Max Tegmark, Joshua B. Tenenbaum, Yunsheng Tian, Sebastien Wah, Bohan Wang, Tsun-Hsuan Wang, Yixuan Wang, Bowen Wen, Kui Wu, Zhou Xian, Xiaoyu Xiong, Wei Yang, John Zhang, Tianyuan Zhang, Wei Zhang, Yu Zhang, Allan Zhao, Juntian Zheng 和 Bo Zhu. 没有他们极大的支持和贡献，我的研究工作都不可能完成.

除了研究，我还从朋友们那里获得了无比珍贵的关爱和支持. 我怀念和实验室同伴一起烧烤、共进年末节日晚餐的时光，也珍惜参加 CGGAR 静修活动和麻省理工学院计算机图形学组 ABCDEFG 滑雪活动的经历. 与 NVIDIA 西雅图机器人实验室实习生一起度过的夏日活动也令人难忘. 我与 Ji Lin、Siyu Yao、Lijie Fan、Tianhong Li 和 Wendi Gao 一起享受无菜单料理、单板滑雪、网球、德州扑克和麻将；与 Yonglong Tian、Lei Xu、Hanrui Wang 和 Yue Wang 一起玩《阿瓦隆》也给我带来很多欢乐. 我与 Lijie Chen、Qicheng Zhao、Toru Lin、Baian Chen、Brabeeba Wang、Zihao Xu 和

Aoxiao Zhong 多次把酒畅谈，还与 Jiayuan Mao、Xinyi Gu、Minghao Guo、Tianyu Li、Xiaolin Fang、Ching-Yun Ko、Yifei Li、Tsun-Hsuan Wang、Shuang Li 和 Heng Yang 探访了许多餐馆. 与 Tiancheng Yu、Haotian Tang 和 Yian Wang 同住的经历非常美好. 此外，在异国他乡，我与 Yunsheng Tian、Ruihan Yang 和 Yujun Shi 保持着从本科时期延续至今的长久友谊. 博士期间，我对健身产生了兴趣，非常感谢 Tiancheng Yu、Guangxuan Xiao、Bohan Wang 和 Bolei Deng 在训练时的保护和陪伴. 我也开始探索人像摄影，幸运地遇到了许多耐心的模特，他们支持我并帮助我成长. 我尤其感谢我的第一位人像模特 Gege Qian 的鼓励与耐心. 人们常说，成年后难得找到真正的朋友——他们真心为你的成功欢庆，也对你的悲伤感同身受——而我非常幸运，生命中有这样美好的人.

最后，我要向家人表达最诚挚的感谢. 我知道，在国外攻读博士不仅对我，对父母也是一个巨大的挑战；然而，他们毫不犹豫地支持了我的每一个决定. 一直以来，我抓住每一个机会培养独立能力，但我始终知道，这一切之所以可能，是因为他们无条件的爱与支持. 最后，我要特别感谢已故的祖父 Rujiao Ma. 他在我读博第三年去世，由于种种复杂原因，我未能参加他的葬礼. 他在我的成长中付出了许多，也一直鼓励我接受更高等的教育. 我希望他会为我的成就感到骄傲，愿他永远安息.

目录

论文信息 ii

摘要 iii

先前发表的材料 iv

致谢 v

第 1 章 引言 1
- 1.1 研究动机 . . . 1
- 1.2 提出的解决方案及应用 . . . 1
  - 1.2.1 渲染不变状态预测器（第 2 章） . . . 2
  - 1.2.2 神经本构律（第 3 章） . . . 2
  - 1.2.3 科学生成智能体（第 4 章） . . . 3
- 1.3 论文贡献 . . . 3
- 1.4 论文影响 . . . 3

第 2 章 RISP：利用可微仿真与渲染进行跨域参数估计的渲染不变状态预测器 5
- 2.1 引言 . . . 5
- 2.2 相关工作 . . . 6
- 2.3 方法 . . . 7
  - 2.3.1 可微仿真与渲染引擎 . . . 8
  - 2.3.2 RISP 网络 . . . 8
- 2.4 实验 . . . 10
  - 2.4.1 实验设置 . . . 10
  - 2.4.2 状态估计结果 . . . 11
  - 2.4.3 系统辨识结果 . . . 12
  - 2.4.4 模仿学习结果 . . . 12
  - 2.4.5 真实世界应用 . . . 12
- 2.5 结论、局限性和未来工作 . . . 14

第 3 章 从运动观测中学习神经本构律以实现可泛化的偏微分方程动力学 15
- 3.1 引言 . . . 15
- 3.2 相关工作 . . . 16
- 3.3 方法 . . . 18
  - 3.3.1 背景 . . . 18
  - 3.3.2 神经本构律 . . . 19
  - 3.3.3 物理感知网络架构 . . . 20
- 3.4 实验 . . . 20
  - 3.4.1 实验设置 . . . 21
  - 3.4.2 重建 . . . 21
  - 3.4.3 泛化 . . . 21

3.4.4 极端泛化 . . . 23
3.4.5 多物理泛化 . . . 23
3.4.6 真实世界实验 . . . 24
3.4.7 消融研究 . . . 26
3.5 结论、局限性和未来工作 . . . 26

第 4 章 将大语言模型与仿真作为双层优化器：推进物理科学发现的新范式 28
4.1 引言 . . . 28
4.2 科学生成智能体 . . . 29
4.2.1 双层优化流程 . . . 29
4.2.2 LLM 驱动的外层搜索 . . . 31
4.2.3 可微内层优化 . . . 32
4.3 实验 . . . 32
4.3.1 问题定义 . . . 32
4.3.2 实验设置 . . . 33
4.3.3 物理科学发现 . . . 33
4.3.4 消融研究 . . . 35
4.3.5 案例研究 . . . 39
4.4 相关工作 . . . 39
4.4.1 自动化科学发现 . . . 39
4.4.2 大语言模型与智能体 . . . 41
4.4.3 双层优化 . . . 41
4.5 结论 . . . 41

第 5 章 结论 43
5.1 主要认识 . . . 43
5.2 局限性和未来方向 . . . 44

附录 A RISP 附录：利用可微仿真与渲染进行跨域参数估计的渲染不变状态预测器 45
A.1 定理的证明 . . . 45
A.2 实现细节 . . . 46
A.2.1 环境 . . . 46
A.2.2 RISP 网络 . . . 47
A.2.3 训练细节 . . . 47
A.3 更多实验 . . . 47
A.3.1 视觉运动控制 . . . 47
A.3.2 真实世界实验 . . . 47
A.3.3 消融研究 . . . 48

附录 B 从运动观测中学习神经本构律以实现可泛化 PDE 动力学：附录 51
B.1 物质点法 . . . 51
B.1.1 弱形式 . . . 51

B.1.2 空间离散化 . . . 51
B.1.3 时间离散化 . . . 52
B.1.4 MPM 伪代码 . . . 52
B.2 用于训练的材料模型 . . . 53
B.3 神经本构律算法 . . . 54
B.3.1 算法 . . . 54
B.3.2 旋转不变性的证明 . . . 55
B.3.3 旋转等变性的证明 . . . 55
B.4 实现细节 . . . 56
B.4.1 仿真 . . . 56
B.4.2 网络与训练 . . . 56
B.4.3 真实世界实验 . . . 56
B.5 额外讨论 . . . 56
B.5.1 向基于网格的仿真泛化 . . . 56
B.6 额外实验 . . . 57
B.6.1 运行时间比较 . . . 57
B.6.2 与较近期方法比较 . . . 57
B.6.3 参数辨识 . . . 58
B.6.4 数据效率 . . . 58
B.6.5 辅助损失 . . . 59
B.6.6 更多可视化 . . . 59

附录 C LLM 与仿真作为双层优化器：物理科学发现新范式的附录 63
C.1 完整提示词 . . . 63
C.2 更多说明 . . . 67
C.2.1 数据流程 . . . 67
C.2.2 与符号回归任务的区别 . . . 68
C.3 更多实验 . . . 68
C.3.1 符号回归 . . . 68
C.3.2 增加迭代次数 . . . 68
C.4 更多结果 . . . 70
C.4.1 本构律发现 (a) . . . 70
C.4.2 本构律发现 (b) . . . 71
C.4.3 本构律发现 (c) . . . 72
C.4.4 本构律发现 (d) . . . 73
C.4.5 虚构本构律 . . . 74

参考文献 75

术语索引 95

插图

2-1 四种环境（从左至右）在三个渲染域（从上至下）中的展示. 对每种环境，我们使用由可微渲染器在不同光照、背景和材料下生成的图像（上排）训练一个 RISP. 随后，每种环境的目标是找到合适的系统和控制参数，以仿真并渲染物理系统（中排），使其与渲染配置未知的参考视频（下排）中的动态运动相匹配. 我们刻意让三排使用渲染配置差别极大的渲染器. 图内 Quadrotor、Cube、Hand、Rod 分别表示四旋翼、立方体、手和杆. 6

2-2 我们的方法概览（第 2.3 节）. 首先使用以随机状态和渲染参数渲染的图像训练 RISP（上）. 然后将 RISP 接在可微渲染器的输出端，形成一条从系统和控制参数到由图像预测出的状态的完全可微流程（中）. 给定目标域中由未知参数（灰色虚线框）生成的参考图像（下），我们将其输入 RISP，并最小化预测状态之间的差异（下方绿灰色框），以重建底层系统参数、状态或动作. 各行分别标出源域训练、源域预测和目标域预测；浅绿及灰色节点分别对应源域和目标域，灰色虚线框依原图标记未知的参数及中间计算. 两路预测分别记为 s_{src} 和 s_{tgt}，共同输入下方损失框. . . 7

2-3 手环境中的模仿学习（第 2.4.4 节）. 给定参考视频（下排，以五个中间帧显示），目标是重建一个动作序列，使其运动与之相似. 图中展示了以随机选择的动作初始猜测值（上排）和使用我们带渲染梯度方法优化后的动作（中排）生成的运动. 图内 Random、Ours、Reference 分别表示随机初始值、我们的方法和参考视频. 13

2-4 真实世界实验中的模仿学习（第 2.4.5 节）. 给定参考视频（上排），目标是重建发送给虚拟四旋翼飞行器的动作序列，使其运动与参考视频相似. 下排展示了我们的方法重建的运动. 图内 Input Video 和 Ours 分别表示输入视频和我们的方法. 14

3-1 NN–PDE 混合时间步进. (a) 我们的方法通过连续 “时间步进” 求解动力系统. (b) 在时间步进算法内部，我们先使用神经弹性本构律得到应力，再求解控制 PDE（式 (3.1)）更新状态，最后通过神经塑性本构律得到新的弹性变形梯度. 相应伪代码见算法 2. 上方原图中 Time-Step 表示时间步进；下方图按原图重排，保留原图的速度符号 $\boldsymbol{v}$. 17

3-2 泛化. 我们首先在 training 指定的环境中训练所有方法，初始速度由黑色箭头表示. 然后，在 testing 指定的四项任务中评估所有方法的泛化能力：(a) 延长时间，(b) 未见过的初始速度，(c) 有挑战性的几何形状，(d) 倾斜边界. 虚线左侧显示初始化，右侧显示经过一段时间后的仿真结果. 我们将方法（ours）与基线（neural 和 gnn）、我们方法的初始化（ours-init）及真实仿真（ground-truth）比较，并以红框标出 ours 和 ground-truth. 图中 training、testing 分别为训练和测试；time、velocity、geometry、boundary 分别为时间、速度、几何形状和边界；Jell-O、Sand、Plasticine、Water 分别为果冻、沙、橡皮泥和水. 22

3-3 极端泛化. 我们在两项极端泛化实验中比较初始状态、我们的方法以及传统仿真的真实结果：(a) 包含超过 100 万个粒子的复杂水龙；(b) 四个橡胶玩具相互及与地面剧烈碰撞. 图中 init、ours、ground-truth 分别表示初始状态、我们的方法和真实结果；one-million、collision 分别表示百万粒子和碰撞. 24
3-4 进阶实验. 左：在两个多物理环境中比较我们的方法与传统仿真的真实结果：(a) 橡皮鸭落入水池；(b) 融化的冰淇淋逐渐从一种材料转变为另一种材料. 右：我们用真实面团下落数据训练方法，学习面团的本构律：(c) 方法成功重建训练序列；(d) 方法能够很好地泛化到未见过的条件，例如复杂几何形状. 图中 coupled-physics、phase-transition、real-reconstruction、real-generalization 分别为耦合物理、相变、真实重建和真实泛化；ground-truth、real、ours、time 分别为真实仿真、真实数据、我们的方法和时间. 25
4-1 科学生成智能体（SGA）的整体流程. 以本构律搜索为例，输入是一个初始猜测（纯弹性材料），输出是朝真实目标（弱可压缩流体）优化得到的另一个本构律. 初始猜测首先初始化一个存储解的 top-K 堆. 在外层优化中，LLM 以先前提出的 top-K 解为输入，在此基础上修改连续参数化 Θ 和离散表达式 $\mathcal{E}$，生成更好的解. 在内层优化中，基于梯度的优化通过仿真求解最优 Θ，并将优化后的解加入堆中. 经过若干轮双层优化后，堆返回排名第一的解作为最终解. 上方原图的 Purely Elastic Material 和 Weakly Compressible Fluid 分别为纯弹性材料和弱可压缩流体；下方流程按原图重排；Loss 和 Iteration 分别表示损失和迭代. 30
4-2 损失趋势比较. 图示 LLM 驱动优化的不同迭代中，最佳解的损失在不同随机种子上的平均值；阴影表示最小 / 最大值. 图内 Loss、Iteration、Zoom In、Ours 分别表示损失、迭代、放大和我们的方法. 34
4-3 双层优化的消融实验. 有和无双层优化的优化轨迹分别以红色圆点和橙色三角形表示；本方法在内层优化之前的中间步骤以橙色叉号表示. 外层 LLM 优化与内层仿真优化分别以橙色和红色箭头突出显示. 图内 Loss、Iteration 分别为损失和迭代；w/o bilevel、w/ bilevel (pre-inner-optim)、w/ bilevel (ours) 分别表示无双层优化、双层优化的内层优化前步骤以及采用双层优化的本方法. 37
4-4 LLM 骨干模型消融实验. 我们比较四个选定骨干 LLM 的表现并报告其排名. 越靠外的曲线表示性能越好；(a)–(h) 对应实验中的八项任务. . . . 38
4-5 探索与利用的消融实验. (a) 不同迭代中可用于仿真的有效解的直方图. (b) 不同迭代中最佳解的损失在不同随机种子上的平均值，阴影表示最小 / 最大值. 图内 Success、Training Error、Syntax Error、Success Rate Curve 分别为成功、训练错误、语法错误和成功率曲线；Ours、No Exploration、No Exploitation 分别为本方法、无探索和无利用；Loss、Iteration、#Solutions 分别为损失、迭代和解的数量. 39

4-6 案例研究. (a) 搜索所得本构律的具体示例；(b) 两个针对不同目标优化的新分子及其 SMILES 字符串. 图中 Molecule A、Molecule B 分别为分子 A、分子 B；代码原样保留，注释依次表示杨氏模量的对数、泊松比的 sigmoid、剪切模量、变形梯度行列式 J、体积部分、偏量部分及计算 Kirchhoff 应力张量. 40
A-1 真实实验中的模仿学习. 给定参考视频（上排），目标是重建一个与其运动相似的动作序列. 图示本方法（上中排）、逐像素损失（下中排）及其增强变体（下排）重建的运动. Input Video 为输入视频，RISP 为本方法，GradSim 和 GradSim-Enhanced 分别为逐像素损失方法及其增强变体. . 49
A-2 (a) epoch 数与损失的曲线. 实线和虚线分别为 ours 和 ours-no-grad 的训练曲线. 颜色表示训练集中的渲染配置数量：绿色、橙色分别为 1 和 10，红色曲线对每个训练数据采样不同的渲染配置. (b) 损失与杨氏模量的曲线. 橙、蓝、绿、灰曲线表示不同时间采样率下关于杨氏模量的损失曲面，红色竖线为真实杨氏模量. 图内 loss、#epoch、Young's Modulus 分别为损失、训练轮数和杨氏模量；First and last frames、Every frame、Every 5th frame、First and middle frames 分别表示仅首末帧、每帧、每五帧及仅首帧和中间帧. 50
B-1 真实实验. 实验装置的正视图和侧视图；虚线轮廓表示面团的下落位置，虚线箭头表示方向. Front view、Side view、Dropping test、Sample、Light source、Camera 分别为正视图、侧视图、下落测试、样品、光源、相机. 57
B-2 训练数据. 展示 (a) 果冻、(b) 沙、(c) 橡皮泥、(d) 水的颗粒与真实感渲染. 60
B-3 果冻和沙的几何泛化. 果冻采用犰狳几何形状，沙采用金鱼几何形状；比较 ours、ground-truth 与基线 gnn 和 neural. 其中 ground-truth 为真实结果，ours 为本方法. 61
B-4 橡皮泥和水的几何泛化. 橡皮泥采用兔子几何形状，水采用奶牛几何形状；比较 ours、ground-truth 与基线 gnn 和 neural. 其中 ground-truth 为真实结果，ours 为本方法. 62

表格

2.1 状态估计结果（第 2.4.2 节）. 表中每项报告由四种渲染配置下的 800 张图像计算得到的状态估计误差的均值和标准差. 11
2.2 系统辨识结果（第 2.4.3 节）. 每一项报告由四组随机初始猜测值和渲染条件计算出的参数估计误差的均值和标准差. 12
2.3 模仿学习结果（第 2.4.4 节）. 每一项报告由四组随机生成的初始猜测值和渲染条件计算出的状态差异的均值和标准差. N/A 表示优化后未能收敛. 13
3.1 重建. 不同方法的重建损失. 对每个环境，基线及我们的方法中排名第一者以深绿色表示，第二者以浅绿色表示；最后一列给出每种方法的统计量（均值 ± 标准差）. 图中 Method、Overall 分别表示方法和总体；材料名称译法见正文. 22
3.2 泛化. 不同方法的泛化损失. 定量结果对应图 3-2 的三项主要任务：(a) 延长时间，(b) 未见过的初始速度，(c) 有挑战性的几何形状. 对任务 (b)，我们使用三个随机初始速度评估并报告平均损失，以进行更充分的验证. 对各环境中的每项任务，所有基线和我们的方法中排名第一者以深绿色表示，第二者以浅绿色表示；最后一列给出各方法的均值 ± 标准差. Method 为方法，Overall 为总体；材料名称译法见正文. 23
4.1 基准测试. 我们将方法与四个基线及本方法的两个变体比较，并列出架构或超参数差异. #Iter. 为迭代次数，#Hist. 为历史优化步骤中 top-k 检索的 K 值，#Exploit/#Explore 为利用与探索的后代数，Bilevel 表示是否启用双层优化. 八个任务分为本构律搜索 (a)–(d) 和分子设计 (e)–(h). 所有任务均以较低损失为佳，最低损失的最佳方法用粗体标出. 图内 Method、Constitutive Law Search 和 Molecule Design 分别为方法、本构律搜索和分子设计；Ours (no bilevel)、Ours (no exploit) 为无双层优化和无利用的变体. 34
4.2 与符号回归比较. 我们将方法与 SRBench 中表现最好的五种方法以及三种预训练符号回归方法比较. 最后一列表示结果是否为符号表达式. . . . 36
4.3 与基于种群的分子设计方法比较. 我们将方法与传统的基于种群的分子设计方法 GhemGE 比较，报告分子设计任务 (e)–(h) 的结果. 36
4.4 虚构本构律实验. 我们构造一种虚构本构律，以防止 LLM 通过记忆取巧，并报告本方法和基线的结果. 36
A.1 视觉运动控制结果. 每项报告由四组随机生成的初始猜测和渲染条件计算的状态差异均值及标准差. 47
B.1 运行时间比较. 57
B.2 与较近期方法比较. 58
B.3 参数辨识. 58
B.4 数据效率. 59
B.5 辅助损失. 59
C.1 符号回归. 末列表示结果是否为符号表达式. 69

C.2 增加迭代次数. #Iterations 为迭代次数, Improvement 为改进率; (a)–(h) 对应八项任务. 69

Chapter 1

引言

1.1 研究动机

长期以来，科学家一直在探索构建高效世界表征的有效方法，这种表征称为世界模型（world model）[51, 95]. 世界模型使智能体能够与环境交互，以预测未来事件 [54, 57, 58, 80, 209, 211]，或计算关于当前状态或动作的梯度 [35, 37, 66, 67, 140, 142, 141, 191]. 通常，世界模型能够感知来自环境的多模态信号，并以多模态反馈响应输入的控制信号 [51, 80, 113]. 以较低成本获得未来轨迹及导数，能够显著提高机器人控制与设计 [57, 58, 113, 114]、推理 [51] 和生成 [192] 等下游任务的数据效率、鲁棒性和性能.

近期工作使用可微物理仿真器或残差物理构建基于物理的世界模型，以对已有充分研究的物理效应建模，例如关节刚体 [141, 199]、可变形体 [140, 142, 37, 191]、流体 [35, 196] 和多物理系统 [101]. 通过优化预先定义的系统参数，这些世界模型通常与现实世界的物理过程较为吻合，从而能够有效推广到未见过的配置. 但是，依赖人工设计的物理引擎和系统参数，限制了它们向未建模物理效应的泛化能力. 例如，使用基于专用有限元方法的可微可变形体仿真来模拟流体环境，需要付出大量工作，甚至可能无法实现. 这一限制凸显了将这些模型扩展到多样而复杂的物理现象所面临的挑战.

另一方面，另一类研究尝试使用图神经网络 [154, 137] 或潜在动力学模型 [53, 54]，以极少的物理先验对世界建模. 这些方法旨在构建底层世界表征，借助结构和学习得到的表征理解并预测动力学，而不严重依赖预先定义的物理参数. 这可能产生更灵活的模型，避免人工设计物理规律所带来的限制. 然而，由于缺乏基本物理原理，也可能导致样本效率降低、计算资源浪费和物理一致性受损.

近期研究还探索了利用物理信息神经网络（PINN）[146, 144, 145, 81] 将物理先验融入普通神经网络. 在这种方法中，物理约束被表述为损失函数，并以自监督方式反向传播至神经网络. PINN 将数据与物理定律相结合，为物理世界建模提供了新的视角. 然而，这种结合往往过弱，难以可靠地支持长时间跨度的下游任务. 此外，PINN 通常针对特定时间范围内的特定场景配置进行优化，限制了其在更广泛场景中的适用性.

1.2 提出的解决方案及应用

本论文计划重新思考如何将神经网络和物理融入世界模型，并开发一种增强世界模型的*神经物理*（neural physics）混合方法. 借助神经物理，我们通过完善基础物理定律并从数据中学习额外的复杂性，连接世界模型与*可微物理*（differentiable physics）. 神经物理具有以下优点：

1. **准确性.** 神经物理将基础物理融入其架构，并使用神经网络对变化最大、最关键的组成部分（例如材料模型或感知模型）进行参数化，从数据中学习这些参数. 因

此，神经物理既受益于物理原理所提供的准确性，也受益于数据驱动学习所提供的准确性.

2. **适应性.** 我们采用模块化层次结构设计神经物理，使用户能够从一系列预训练的神经物理模块中自由选择，并以即插即用的方式部署，以适应新的配置.

3. **泛化能力.** 神经物理将各种物理效应与基础物理定律解耦，使其能够推广到多种物理现象.

本论文将探索神经物理的多种实现和应用，具体如下.

1.2.1 渲染不变状态预测器（第 2 章）

首先，我们提出渲染不变状态预测器（RISP）[113]，将神经物理嵌入世界模型的感知模型中. 这项工作考虑直接从渲染配置不可获取的视频中，校准表征物理系统动力学运动的参数. 现有方法要么需要海量训练数据，要么缺乏对未知渲染配置的泛化能力. 为解决这一问题，我们提出一种结合神经感知模型和可微物理的新方法. 核心思想是训练一个渲染不变状态预测（RISP）网络，将图像差异转换为不依赖渲染配置（例如光照、阴影或材料反射率）的状态差异. 为训练该预测器，我们利用可微渲染的梯度，针对渲染变化构造了一个新的损失函数. 此外，我们提出一种高效的二阶方法计算该损失的梯度，使其能够无缝集成到现代深度学习框架中. 我们在包含刚体和可变形体的多物理世界模型中，通过状态估计、系统辨识、模仿学习和视觉运动控制四项任务评估该方法. 我们还在一个真实世界例子中展示了方法的有效性：从四旋翼飞行器的运动视频推断其状态与动作序列. 与现有方法相比，我们的方法重建误差显著更低，并且对未知渲染配置具有更好的泛化能力.

1.2.2 神经本构律（第 3 章）

我们提出一种神经物理混合方法，从运动观测中学习具有泛化能力的世界模型. 近期世界模型通常学习一个端到端模型，以隐式地同时对控制偏微分方程（PDE）和本构模型（或材料模型）建模. 缺少显式 PDE 知识时，这些模型无法保证物理正确性，且泛化能力有限. 我们认为，控制 PDE 往往是已知的，应当显式施加，而不是通过学习得到. 相反，本构模型由于具有数据拟合的特性，特别适合通过学习获得. 为此，我们引入一种称为神经本构律（NCLaw）的新神经物理框架 [115]，其网络架构严格保证标准本构先验，包括旋转等变性和未变形状态下的平衡. 我们将 NCLaw 与可微仿真连接，通过最小化基于仿真与运动观测之间差异的损失函数来训练模型. 我们在从固体到流体的多种大变形动力系统上验证 NCLaw. 仅使用一条运动轨迹训练后，我们的方法便能推广到新的几何形状、初始 / 边界条件、时间范围，乃至多物理系统. 在这些极端分布外泛化任务中，NCLaw 比以往世界模型的准确度高出数个数量级. 真实世界实验表明，我们的方法能够从视频中学习本构律.

1.2.3 科学生成智能体（第 4 章）

大型语言模型（LLM）因其广泛的知识和先进的推理能力，近期在科学发现领域受到广泛关注. 然而，它们难以有效地模拟观测反馈并将反馈与语言联系起来，以推动物理科学发现. 相比之下，人类科学家通过提出假说、进行实验并通过观测分析修正理论来开展科学发现. 受此启发，我们提出用世界模型的计算能力增强 LLM 由知识驱动的抽象推理能力. 我们引入科学生成智能体（SGA）[116]，这是一种双层优化框架：LLM 充当知识丰富、多才多艺的思考者，提出科学假说，对物理方程或分子结构等离散部分进行推理，并将它们嵌入神经物理模块. 与此同时，世界模型充当实验平台，提供观测反馈，并利用可微性对物理参数等连续部分所对应的神经物理模块进行优化. 我们通过大量实验展示框架在本构律发现和分子设计中的有效性，揭示出不同于人类常规预期、但经分析仍自洽的新解.

1.3 论文贡献

本论文及其核心模块的主要贡献如下：

- **渲染不变状态预测器（第 2 章）**引入一个与可微物理协作的神经模块，以缩小仿真到现实的差距. 该神经模块通过最小化不同渲染参数下的输出方差进行训练. 借助高效的可微工作流程，我们使用二阶优化训练渲染不变状态预测器，以弥合仿真到现实的差距. 我们在从复杂关节机器人手到弹性绳索的四种不同环境中展示了渲染不变状态预测器的有效性.

- **神经本构律（第 3 章）**引入一组神经网络，对与物理定律对应的映射建模，从而在统一表征中参数化多种物理现象. 神经本构律通过将旋转等变性和静态平衡内嵌于神经网络架构，协调二者，以提高性能并稳定训练. 神经本构律提供了首个统一的神经参数化方案，为更准确的仿真到现实校准、更快速的物理仿真以及更好地理解物理世界打开了大门.

- **科学生成智能体（第 4 章）**引入一种科学发现的迭代流程，通过结合 LLM 与物理仿真的双层优化来实现. 科学生成智能体在基于自然语言的通用系统与基于代码的领域专家之间建立接口，并将二者级联应用于多个科学发现问题. 我们通过搜索新的本构律，证明了科学生成智能体比传统符号回归更加准确、具有更强的泛化能力和可扩展性. 此外，科学生成智能体在分子设计任务上的表现显示出有竞争力的多学科能力.

1.4 论文影响

我们认为，将神经物理引入世界模型可能显著提高其性能，尤其是在速度、效率和物理真实性方面. 传统世界模型的建模方式往往要么严格限定不同模块之间的连接，缺乏灵活性；要么选择完全端到端的训练，完全忽略模块之间的联系. 我们预期神经物理将为世界模型构建流程的模块化带来新的可能.

其次，我们预期神经物理将对世界模型的可控性产生积极影响. 现实世界包含广泛的条件，包括接触等物理交互，以及对话等智能交互. 通过这些多样而异质的交互，世

界模型为智能体的学习与演化创造动态环境. 将神经模块融入物理仿真后，模型能够接受更多样的控制信号；将模块化引入世界模型后，它能够更严格地施加物理约束. 我们期望神经物理所提供的互操作性和交互性将为世界模型创造新的应用与部署场景.

最后，本论文强调世界模型在机器人、科学发现和通用人工智能等领域潜在的广泛应用. 作为物理动力学的静态知识库，世界模型包含智能发展与交互所需的全部资源. 在神经物理的增强下，世界模型能够实现更高的物理真实性和效率，从而支持超越单纯重建的更多应用. 论文中展示的机器人技术就是一种直接应用：智能体与模型交互并据此规划动作. 准确模拟物理动力学能够显著降低科学发现的成本，加快迭代过程. 通用人工智能寻求对人类水平的智能建模，并探索通往超人能力的路径. 世界模型为研究智能体的演化与交互提供了理想环境.

总之，本论文提出了用于构建整体世界模型的神经物理方法，并通过实例具体展示了潜在的新应用. 我们希望本论文能为下一代物理智能的发展奠定基础.

Chapter 2

RISP：利用可微仿真与渲染进行跨域参数估计的渲染不变状态预测器

2.1 引言

直接从视频重建物理系统的动力学信息，已受到机器人、机器学习、计算机视觉和图形学领域的广泛关注. 由于系统的物理、几何和感知深度耦合，这一问题从根本上具有挑战性. 运动捕捉系统等传统方案 [180, 131, 143] 可以提供高质量结果，但需要价格高得难以承受的外部硬件平台. 近年来，可微仿真与渲染的发展为运动捕捉系统提供了一种成本低廉且有吸引力的替代方案，并已展现出有前景的概念验证结果 [126]. 然而，这一方向的现有方法通常假设视频来自已知的渲染器. 这一假设限制了它们从未知渲染域中推断动力学信息的能力；由于渲染视频与真实世界视频之间存在差异，未知渲染域在现实应用中很常见. 对齐不同渲染域的现有技术，例如 CycleGAN [214]，或许有助于缓解这一问题，但它们通常需要访问目标域中的海量数据，而这些数据并非总能获得. 据我们所知，在未知渲染条件下直接从视频推断物理系统的动力学参数，仍远未得到解决；我们的工作旨在填补这一空白.

为应对这一挑战，我们提出一种结合三个思想的新方法：域随机化、状态估计和渲染梯度. 域随机化是一种在域之间迁移知识的经典技术，它生成大量样本，使其变化范围能够覆盖域间差异. 我们通过两项关键创新对其加以改进. 首先，我们注意到图像差异对渲染配置的变化很敏感，这掩盖了我们真正希望推断的、与动力学有关且不随渲染变化的参数. 这一观察促使我们提出渲染不变状态预测器（RISP），从视频中提取物理系统的状态信息. 第二项创新是利用可微渲染器提供的渲染梯度. 从本质上说，要求 RISP 的输出与渲染配置无关，等价于要求其关于渲染参数的梯度为零. 我们利用渲染梯度提出一个新的损失函数，并展示了将其集成到深度学习框架中的高效方法.

结合上述思想，我们开发出一个有效的流程，能够直接从随机渲染配置下的视频输入中推断物理系统的参数. 我们在四种环境中的多项具有挑战性的任务上，以及在一个真实世界应用中，展示了该方法的有效性（见第 2.4 节、图 2-1 和图 2-4）. 实验结果表明，由于训练过程中引入了渲染梯度，我们的方法在大多数任务上大幅优于当时最先进的技术.

概括而言，本工作的贡献如下：

- 我们研究并确定了在不同渲染配置下，从视频推断物理系统的状态、系统及控制参数时的瓶颈（第 2.3.1 节）；

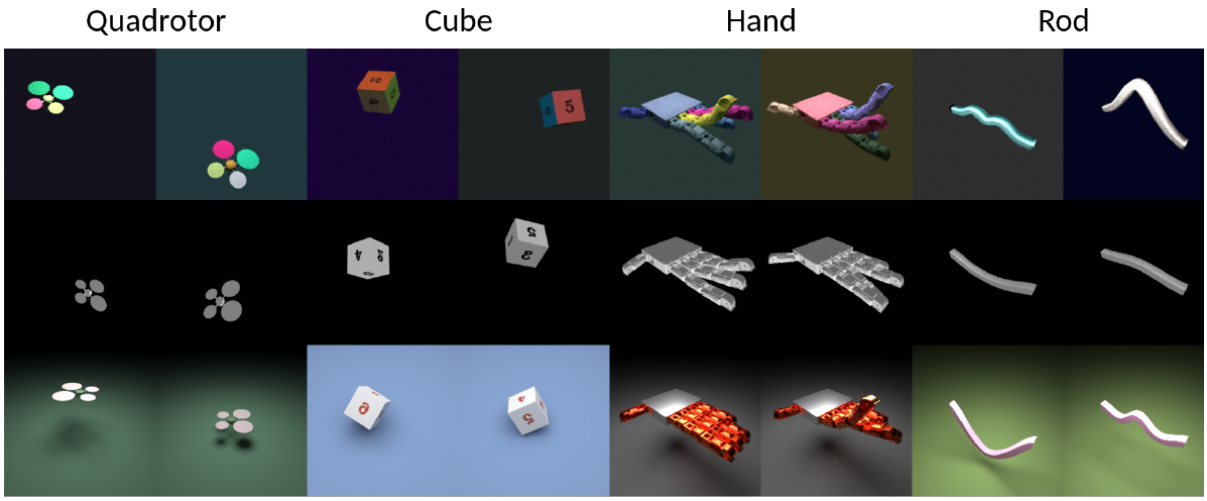


图 2-1: 四种环境（从左至右）在三个渲染域（从上至下）中的展示. 对每种环境，我们使用由可微渲染器在不同光照、背景和材料下生成的图像（上排）训练一个 RISP. 随后，每种环境的目标是找到合适的系统和控制参数，以仿真并渲染物理系统（中排），使其与渲染配置未知的参考视频（下排）中的动态运动相匹配. 我们刻意让三排使用渲染配置差别极大的渲染器. 图内 Quadrotor、Cube、Hand、Rod 分别表示四旋翼、立方体、手和杆.

- 我们提出一种结合域随机化、状态估计和渲染梯度的新方案，以实现跨渲染域的泛化（第 2.3.2 节）;
- 我们在仿真和真实世界环境中的若干具有挑战性的任务上展示了方法的有效性（第 2.4 节）.

2.2 相关工作

可微仿真. 可微仿真为传统仿真提供梯度信息. 这一额外的梯度信息将仿真任务与数值优化技术联系起来. 以往工作已经展示了可微仿真器的梯度在刚体动力学 [47, 32, 30, 199, 64, 141]、可变形体动力学 [37, 70, 36, 66, 45, 55, 114, 142]、流体 [35, 121, 67] 和余维物体 [140, 104] 中的作用. 本工作大量使用可微仿真器，但我们的贡献与它们正交：我们将可微仿真视为黑箱，提出的方法不依赖于仿真器的具体选择.

可微渲染. 可微渲染提供关于渲染输入(例如光照、材料或形状)的梯度 [149, 97, 73]. 当时最先进的可微渲染器 [98, 127] 即使在存在全局光照或遮挡时，也能够有效处理梯度. 我们的工作利用这些渲染器，但重点不同：在学习流程中将它们的梯度用作物理先验.

域随机化. 域随机化 [171, 135, 4, 153, 168] 背后的直觉是，模型通过在源域中见到大量随机数据，有望跨越域间差异. 这通常会带来在目标域上稳健但保守的性能. 域随机化的泛化能力来自一个更加稳健的模型：它试图通过采取保守行为来吸收域间差异；而我们方法的泛化能力来自一个更加准确的模型：它力求匹配一阶梯度信息.

2.3 方法

给定一段展示物理系统动态运动的视频，我们的目标是在仅部分了解物理模型及渲染条件的情况下，直接从视频中推断未知的状态、系统或控制参数. 具体而言，我们假设已知物理系统的控制方程（例如刚体系统的牛顿定律）和相机位置，但精确的系统参数、控制参数或渲染参数不可获取.

为解决这一问题，我们提出一个由两个部分组成的流程（图 2-2）：可微仿真与渲染引擎，以及 RISP 网络. 首先，利用引擎仿真并渲染物理系统的状态，输出不同渲染配置下的图像. 接着，RISP 网络学习从这些生成的图像中重建状态信息. 将这两个部分结合起来，就得到一个能够从渲染配置未曾见过的新视频中忠实恢复物理系统动力学信息的流程.

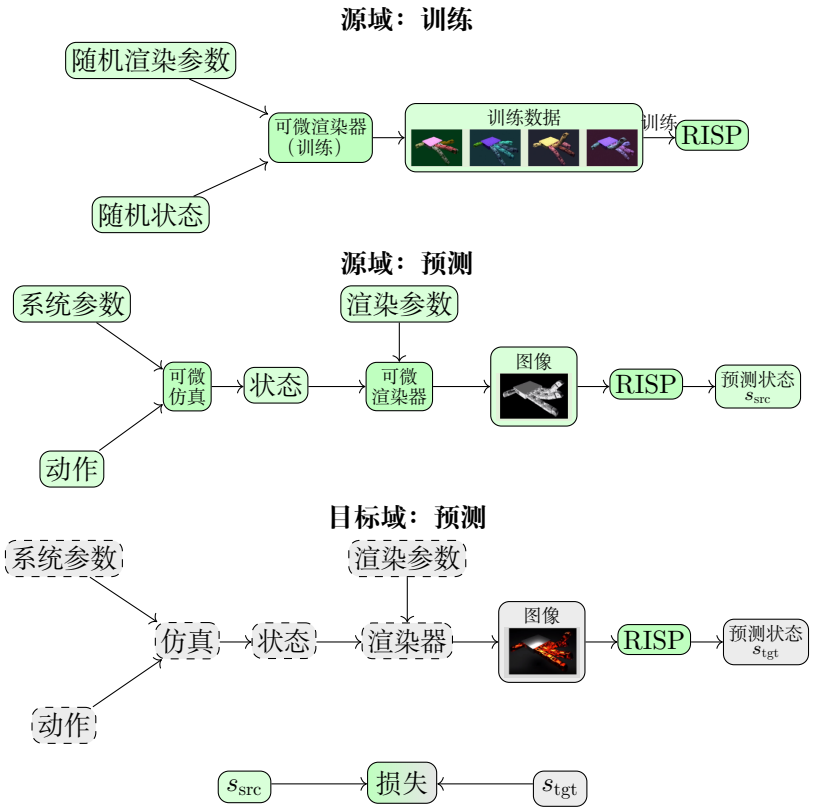


图 2-2: 我们的方法概览（第 2.3 节）. 首先使用以随机状态和渲染参数渲染的图像训练 RISP（上）. 然后将 RISP 接在可微渲染器的输出端，形成一条从系统和控制参数到由图像预测出的状态的完全可微流程（中）. 给定目标域中由未知参数（灰色虚线框）生成的参考图像（下），我们将其输入 RISP，并最小化预测状态之间的差异（下方绿灰色框），以重建底层系统参数、状态或动作. 各行分别标出源域训练、源域预测和目标域预测；浅绿及灰色节点分别对应源域和目标域，灰色虚线框依原图标记未知的参数及中间计算. 两路预测分别记为 s_{src} 和 s_{tgt}，共同输入下方损失框.

2.3.1 可微仿真与渲染引擎

给定动力学模型 $\mathcal{M}$ 已知的物理系统，我们首先对时间进行离散化，并使用可微仿真器根据每个时间步的动作输入模拟其状态：

$$\mathbf{s}_{i+1} = \mathcal{M}_{\varphi}(\mathbf{s}_i, \mathbf{a}_i), \qquad \forall i = 0, 1, \ldots, N-1, \tag{2.1}$$

其中 N 是一次物理仿真展开中的时间步数，$\mathbf{s}_i$、$\mathbf{s}_{i+1}$ 和 $\mathbf{a}_i$ 分别表示相应时间步的状态向量和动作向量. 向量 φ 编码模型的系统参数，例如质量、惯量和弹性. 接下来，我们应用可微渲染器 $\mathcal{R}$ 为每个状态 $\mathbf{s}_i$ 生成图像 $\mathbf{I}_i$：

$$\mathbf{I}_i = \mathcal{R}_{\psi}(\mathbf{s}_i), \qquad \forall i = 0, 1, \ldots, N. \tag{2.2}$$

这里，ψ 是编码渲染参数的向量，其梯度可由渲染器 $\mathcal{R}$ 获得. ψ 的例子包括光强、材料反射率或背景颜色. 略微滥用记号，我们将仿真与渲染引擎的流程重写为紧凑形式：

$$\{\mathbf{I}_i\} = \mathcal{R}_{\psi}\big[\underbrace{\mathcal{M}_{\varphi}(\mathbf{s}_0, \{\mathbf{a}_i\})}_{\{\mathbf{s}_i\}}\big]. \tag{2.3}$$

换言之，给定初始状态 $\mathbf{s}_0$ 和动作序列 $\{\mathbf{a}_i\}$，我们通过仿真生成状态序列 $\{\mathbf{s}_i\}$，并渲染对应的图像序列 $\{\mathbf{I}_i\}$. 从参考视频 $\{\mathbf{I}_i^{\text{ref}}\}$ 中恢复未知信息的任务可表述为：

$$\min_{\mathbf{s}_0, \{\mathbf{a}_i\}, \varphi, \psi} \quad \mathcal{L}(\{\mathbf{I}_i^{\text{ref}}\}, \{\mathbf{I}_i\}), \tag{2.4}$$

$$\text{s.t.} \quad \{\mathbf{I}_i\} = \mathcal{R}_{\psi}[\mathcal{M}_{\varphi}(\mathbf{s}_0, \{\mathbf{a}_i\})], \tag{2.5}$$

其中 $\mathcal{L}$ 是惩罚生成图像与参考图像之间差异的损失函数. 假设仿真器 $\mathcal{M}$ 和渲染器 $\mathcal{R}$ 关于各自输入可微，我们便可以运行基于梯度的优化算法求解式 (2.4). 这本质上就是 ∇Sim 提出的思想；∇Sim 是当时直接从视频输入辨识参数的先进方法 [126]. 具体而言，∇Sim 将 $\mathcal{L}$ 定义为逐像素差异的范数.

式 (2.4) 的一个主要局限是，为成功求解优化问题，初始图像 $\{\mathbf{I}_i\}$ 与参考图像 $\{\mathbf{I}_i^{\text{ref}}\}$ 需要足够相似. 事实上，由于仿真与渲染之间的耦合使优化问题高度非线性，如果两组图像不够接近，梯度下降等局部优化技术便容易陷入局部极小值. 虽然 ∇Sim 报告了在两组图像以中等差异的 ψ 渲染时取得的有前景结果，但我们在实验中发现，当两个渲染域差异极大时（图 2-1），直接优化定义在图像空间上的 $\mathcal{L}$ 很少奏效. 因此，我们认为需要一种根本不同的解决方案，这促使我们在方法中提出 RISP.

2.3.2 RISP 网络

式 (2.4) 难以推广到不同渲染域，部分原因在于损失 $\mathcal{L}$ 定义在图像空间的差异上，而图像空间对渲染配置的变化很敏感. 为解决这一问题，我们从许多可微仿真论文中注意到，状态空间中的损失函数对随机初始化相当稳健 [35, 104]，这启发我们在类似状态的空间中重新定义 $\mathcal{L}$. 更具体地说，我们引入 RISP 网络 $\mathcal{N}$，以图像 $\mathbf{I}$ 为输入，输出状态预测 $\hat{\mathbf{s}} = \mathcal{N}(\mathbf{I})$. 我们将式 (2.4) 中的优化问题重新定义如下（图 2-2）：

$$\min_{\mathbf{s}_0, \{\mathbf{a}_i\}, \varphi, \psi} \quad \mathcal{L}(\mathcal{N}_{\theta}(\{\mathbf{I}_i^{\text{ref}}\}), \mathcal{N}_{\theta}(\{\mathbf{I}_i\})), \tag{2.6}$$

$$\text{s.t.} \quad \{\mathbf{I}_i\} = \mathcal{R}_{\psi}[\mathcal{M}_{\varphi}(\mathbf{s}_0, \{\mathbf{a}_i\})]. \tag{2.7}$$

注意，由 θ 参数化的网络 $\mathcal{N}_\theta$ 经过预训练，并在该优化问题中保持固定. 本质上，式 (2.6) 将两个图像序列映射到预测状态空间，随后进行标准的梯度下降优化. 一个训练良好的网络 $\mathcal{N}$ 可以解释为 "逆渲染器" $\mathcal{R}^{-1}$：无论选择何种渲染参数 ψ，它都能恢复渲染不变的状态向量，使式 (2.6) 即使在图像序列 $\{\mathbf{I}_i\}$ 与 $\{\mathbf{I}_i^{\text{ref}}\}$ 由不同渲染器 $\mathcal{R}_\psi$ 生成时，也能匹配两个序列背后的信息. 下面介绍训练网络 $\mathcal{N}$ 的两个思路.

第一个思路：域随机化. 我们大量采样状态与渲染参数对 $(\mathbf{s}_j, \psi_j)$，并渲染相应图像 $\mathbf{I}_j = \mathcal{R}_{\psi_j}(\mathbf{s}_j)$，得到训练集 $\mathcal{D} = \{(\mathbf{s}_j, \psi_j, \mathbf{I}_j)\}$. 然后训练 $\mathcal{N}$ 以最小化预测误差：

$$\mathcal{L}^{\text{error}}(\theta, \mathcal{D}) = \sum_{(\mathbf{s}_j, \psi_j, \mathbf{I}_j) \in \mathcal{D}} \underbrace{\|\mathbf{s}_j - \mathcal{N}_\theta(\mathbf{I}_j)\|_1}_{\mathcal{L}_j^{\text{error}}}. \tag{2.8}$$

直觉很直接：由于 $\mathcal{N}_\theta$ 见到了用不同渲染参数 ψ 生成的图像，它学会在不同渲染配置之间泛化. 这正是域随机化的思想 [171]；我们借用它来解决跨不同渲染域的问题.

第二个思路：渲染梯度. 域随机化的一个主要瓶颈是需要覆盖渲染参数 ψ 整个分布的海量训练数据. 注意到完全渲染不变的 $\mathcal{N}$ 必须满足以下条件：

$$\frac{\partial \mathcal{N}_\theta(\mathcal{R}_\psi(\mathbf{s}))}{\partial \psi} \equiv 0, \qquad \forall \mathbf{s}, \psi, \tag{2.9}$$

我们考虑在训练损失中加入正则项：

$$\mathcal{L}^{\text{train}}(\theta, \mathcal{D}) = \mathcal{L}^{\text{error}} + \gamma \underbrace{\sum_{(\mathbf{s}_j, \psi_j, \mathbf{I}_j) \in \mathcal{D}} \left\| \frac{\partial \mathcal{N}_\theta(\mathcal{R}_{\psi_j}(\mathbf{s}_j))}{\partial \psi_j} \right\|_{\text{F}}}_{\mathcal{L}^{\text{reg}}}, \tag{2.10}$$

其中 $\|\cdot\|_{\text{F}}$ 表示 Frobenius 范数，γ 为正则化权重. 其直觉是：通过将这一雅可比矩阵压至零，我们促使网络 $\mathcal{N}$ 沿渲染参数 ψ 的维度展平其函数曲面，从而获得对渲染配置的不变性. 为实现该损失，我们应用链式法则：

$$\frac{\partial \mathcal{N}_\theta(\mathcal{R}_{\psi_j}(\mathbf{s}_j))}{\partial \psi_j} = \frac{\partial \mathcal{N}_\theta(\mathbf{I}_j)}{\partial \psi_j} = \frac{\partial \mathcal{N}_\theta(\mathbf{I}_j)}{\partial \mathbf{I}_j} \frac{\partial \mathbf{I}_j}{\partial \psi_j}, \tag{2.11}$$

其中第一项 $\partial \mathcal{N}_\theta(\mathbf{I}_j)/\partial \mathbf{I}_j$ 可在任何现代深度学习框架中获得，第二项 $\partial \mathbf{I}_j/\partial \psi_j$ 可由当时最先进的可微渲染器 [127] 获得. 现在可以更清楚地看到 RISP 背后的直觉：它要求网络对输入图像的敏感方向与渲染参数能够影响图像的方向正交，从而得到渲染不变的预测.

我们强调，式 (2.10) 中新损失的设计并非简单之事. 事实上，$\mathcal{L}^{\text{error}}$ 和 $\mathcal{L}^{\text{reg}}$ 各有其用途，必须结合使用：前者促使 $\mathcal{N}$ 的输出拟合各个不同状态，后者则试图沿 ψ 维度平滑其输出. 具体来说，不能将 $\mathcal{L}^{\text{reg}}$ 作为单独的损失来优化，因为这会导致 $\mathcal{N}$ 始终预测常数状态的平凡解. 将二者结合，迫使它们在准确预测状态和忽略渲染条件带来的噪声之间取得平衡，从而得到真正学会 "逆渲染器" $\mathcal{R}^{-1}$ 的网络 $\mathcal{N}$.

我们还需说明如何计算正则项 $\mathcal{L}^{\text{reg}}$ 关于网络参数 θ 的梯度；基于梯度的优化器需要该梯度来最小化新损失. 由于损失定义现在包含一阶导数，其梯度的计算涉及二阶偏导数；如果不加注意地使用多重循环实现，可能十分耗时. 我们最后一项贡献是提供一种高效计算这一梯度的方法，它完全可以通过现有框架实现（在我们的实验中为 PyTorch 和 mitsuba-2）：

定理 2.3.1： 假设渲染器 $\mathcal{R}$ 支持前向模式微分，网络 $\mathcal{N}$ 支持反向模式微分，我们可以使用占用 $\mathcal{O}(\sum_j |\psi_j||\mathbf{I}_j|)$ 空间的预计算数据，以每张图像 $\mathcal{O}(|\mathbf{s}||\theta|)$ 的时间计算随机梯度 $\partial\mathcal{L}^{\text{reg}}/\partial\theta$.

特别地，我们强调，计算 $\mathcal{L}^{\text{reg}}$ 的梯度不需要渲染器 $\mathcal{R}$ 中的二阶梯度；据我们所知，后者超出了所有现有可微渲染器的能力. 本定理的证明留在补充材料中.

进一步加速. 定理 2.3.1 表明，计算 $\mathcal{L}^{\text{reg}}$ 的梯度所需时间与网络大小和状态维数成线性关系. 对非常小的刚体系统（例如 $|\mathbf{s}| < 10$），$\mathcal{O}(|\mathbf{s}||\theta|)$ 的时间开销可以接受，但该方法难以扩展到更大的系统. 因此，在实现中我们使用略有不同的正则项：

$$\mathcal{L}^{\text{train}}(\theta, \mathcal{D}) = \mathcal{L}^{\text{error}} + \gamma \sum_{(\mathbf{s}_j, \psi_j, \mathbf{I}_j) \in \mathcal{D}} \left\| \frac{\partial \mathcal{L}_j^{\text{error}}}{\partial \psi_j} \right\|. \tag{2.12}$$

换言之，我们转而促使状态预测误差具有渲染不变性. 从定理 2.3.1 的证明可以看出，计算这一新正则项的梯度只需 $\mathcal{O}(\theta)$ 的时间；我们还从经验上发现，新正则项的性能与式 (2.10) 相当，但速度快得多. 我们将两个正则项的理论分析留待未来工作.

2.4 实验

本节开展多项实验，研究以下问题：

1. Q1：跨渲染域视频的逐像素损失是否足以预测参数？
2. Q2：如果逐像素损失不够好，是否存在其他能够与我们方法中的状态预测损失竞争的替代方案？
3. Q3：我们损失中的渲染梯度正则项是否必要？
4. Q4：我们的方法与直接优化状态差异相比表现如何？
5. Q5：所提出的方法是否适用于真实世界场景？

我们使用第 2.4.1 节所述的仿真环境回答前四个问题，并在本节末尾通过一个真实世界应用回答最后一个问题. 实验的更多细节，包括消融研究，见附录.

2.4.1 实验设置

环境. 我们实现了四种环境（图 2-1）：无接触的刚体环境（四旋翼飞行器）、有接触的刚体环境（立方体）、关节体（手）和可变形体环境（杆）. 每种环境都包含一个可微仿真器 [199, 37] 和一个可微渲染器 [98]. 我们刻意使用不同的渲染器 [127] 生成第 2.3 节中的训练集；采样渲染配置时，训练集和各环境也使用不同的分布.

任务. 我们考虑定义在所有环境的物理系统上的四类任务：状态估计、系统辨识、模仿学习和视觉运动控制. 状态估计任务（第 2.4.2 节）要求模型从给定图像预测物理系统的状态，是其他下游任务的前提. 系统辨识（第 2.4.3 节）和模仿学习（第 2.4.4 节）任务分别旨在从视频中恢复物理系统的系统参数和控制信号. 最后，在视觉运动控制任务（附录）中，我们用一张展示物理系统期望状态的目标图像替代视频，目标是找到合

适的控制信号，将系统引导至期望状态. 在所有任务中，我们使用照片级真实感渲染器 [138] 生成目标视频或图像.

基线. 我们考虑两个强基线. 逐像素损失（pixelwise-loss）基线由 ∇Sim [126] 使用；该方法是当时直接从视频输入辨识系统参数的先进方法. 我们通过从本方法中移除 RISP，并将图像逐像素损失通过可微渲染和仿真反向传播，来实现 ∇Sim. 我们运行这一基线，以分析逐像素损失在下游任务中的局限（Q1）. 第二个基线是感知损失（perceptual-loss）[79]，它将 ∇Sim 中的逐像素损失替换为基于预训练卷积神经网络提取的高层特征的损失函数. 将这一基线与我们的方法比较，可以说明我们为何选择让 RISP 预测状态，而不是其他感知特征（Q2）.

我们还纳入 ∇Sim 使用的两个弱基线：平均值（average）基线是一种确定性方法，总是返回训练集中观测量的平均值；随机（random）基线则从用于生成训练集的数据分布中随机抽取一个猜测值. 我们使用这两个弱基线，以避免设计出过于容易解决的环境和任务.

我们的方法. 我们考虑第 2.3 节中方法的两个版本：ours-no-grad 实现域随机化思想，但不使用所提出的正则项；ours 则是包含渲染梯度正则项的完整方法. 通过比较二者，我们希望更好地理解渲染梯度在所提方法中的价值（Q3）.

预言机. 在所有实验中，我们还考虑一种预言机方法：它直接最小化从仿真输出获得的状态差异，而不再进行渲染. 特别地，该预言机能够看到目标视频或图像中每一幅图像的真实状态，而所有基线和我们的方法都无法获取这些状态. 我们将这一方法视为预言机，因为它是我们的方法中仅涉及可微物理的子集，但需要一个完美的状态预测网络. 该预言机能够给出我们方法性能的上界（Q4）.

训练. 我们在修改版 ResNet-18 [60] 的基础上构建 RISP 网络和基线，并使用 Adam 优化器 [84] 训练. 网络架构、训练策略和超参数的更多细节见附录.

2.4.2 状态估计结果

在这项任务中，我们从随机生成的图像预测物理系统的状态，并在表 2.1 中报告状态预测误差的均值和标准差. 请注意，我们不包括感知损失和逐像素损失基线，因为它们不需要状态预测步骤. 总体而言，我们发现，我们的方法在各项状态估计结果上均显著优于所有基线. 两个弱基线表现不佳，证实这一状态估计任务不能轻易解决. 我们特别指出，带渲染梯度损失的方法在所有环境中都能预测最稳定、最准确的物理系统状态，这有力地表明 RISP 学会了做出不依赖各种渲染配置的预测.

	四旋翼	立方体	手	杆
average	0.5994 ± 0.0000	0.5920 ± 0.0000	0.2605 ± 0.0000	0.9792 ± 0.0000
random	0.9661 ± 0.7548	0.9655 ± 0.8119	0.8323 ± 0.5705	1.2730 ± 0.7197
ours-no-grad	0.3114 ± 0.3191	0.2805 ± 0.3199	0.1155 ± 0.0505	0.0201 ± 0.0087
ours	$\mathbf{0.1505 \pm 0.1163}$	$\mathbf{0.1642 \pm 0.1887}$	$\mathbf{0.0974 \pm 0.0255}$	$\mathbf{0.0194 \pm 0.0048}$

表 2.1: 状态估计结果（第 2.4.2 节）. 表中每项报告由四种渲染配置下的 800 张图像计算得到的状态估计误差的均值和标准差.

2.4.3 系统辨识结果

我们的系统辨识任务旨在通过观看动作序列已知的参考视频，预测物理系统的系统参数，例如质量、密度、刚度或弹性. 对每个环境，我们手动设计一个动作序列，并渲染其动态运动的参考视频. 接着，我们随机选择系统参数的初始猜测值，并使用所有基线、我们的方法和预言机运行基于梯度的优化. 我们以随机生成的渲染条件和初始猜测值重复该实验四次，并在表 2.2 中报告各系统参数的均值和标准差. 预言机近乎完美的表现表明，只要有可靠的状态估计，就可以解决这些系统辨识任务. 我们的两种方法都大幅优于几乎所有基线，有时甚至高出数个数量级. 这符合预期，因为前一项任务已经表明，我们的方法从目标视频预测状态的准确性远高于基线，而这是解决系统辨识问题的关键前提. 唯一的例外是立方体环境：逐像素损失的表现出人意料地好. 我们推测这可能是因为其几何形状相对简单，并且与背景的对比较高（图 2-1）.

	四旋翼 质量	立方体 刚度	手 关节刚度	杆 杨氏模量
random	9.22e−2 ± 3.83e−2	2.31e−1 ± 9.57e−2	5.70e−1 ± 1.62e−1	1.30e6 ± 1.35e6
pixelwise-loss	7.22e−2 ± 5.26e−2	**2.24e−3 ± 1.74e−3**	1.04e−1 ± 9.62e−2	8.50e5 ± 1.42e6
perceptual-loss	6.45e−2 ± 5.23e−2	1.16e−1 ± 5.83e−2	1.10e−1 ± 1.18e−1	8.32e5 ± 1.43e6
ours-no-grad	6.07e−2 ± 4.34e−2	1.16e−1 ± 6.20e−2	4.85e−2 ± **2.16e−2**	8.78e4 ± 1.52e5
ours	**1.18e−2 ± 1.93e−2**	6.76e−3 ± 7.23e−3	**3.96e−2** ± 2.73e−2	**9.31e1 ± 4.21e1**
oracle	2.36e−5 ± 2.41e−5	1.15e−3 ± 8.60e−4	3.92e−3 ± 4.40e−4	4.36e0 ± 3.57e0

表 2.2: 系统辨识结果（第 2.4.3 节）. 每一项报告由四组随机初始猜测值和渲染条件计算出的参数估计误差的均值和标准差.

2.4.4 模仿学习结果

我们的模仿学习任务考虑如何模仿参考视频中的动态运动. 实验设置与系统辨识任务相似，区别在于我们交换了环境中的已知和未知变量：现在系统参数已知，目标是从参考视频推断未知动作序列. 请注意，立方体环境因没有控制信号而被排除. 同前，我们在所有环境中以随机生成的渲染配置和动作初始猜测值重复实验四次. 结果见表 2.3. 我们发现，使用渲染梯度的方法（表中的 ours）取得了低得多的误差，表明我们更准确地模仿了视频中的运动. 逐像素损失的误差在不同渲染域之间变化较小，但大于我们的方法，说明它在全部四种渲染配置下都难以解决这一任务. 此外，预言机找到的动作序列产生的运动比我们的方法更相似，但它需要完整且准确的状态信息，而这很少能够从视频中获取. 图 2-3 将我们在手环境中的结果可视化.

2.4.5 真实世界应用

最后，我们在一个真实世界应用中评估方法的有效性：给定一段四旋翼飞行器飞行的视频，我们旨在通过推断合理的动作序列，建立其数字孪生体以复现视频中的运动. 这一从现实到仿真的迁移问题具有挑战性，原因有以下几点. 首先，真实视频包含我们未知的复杂纹理、材料和光照条件，而我们的可微渲染器也未见过这些条件. 其次，四

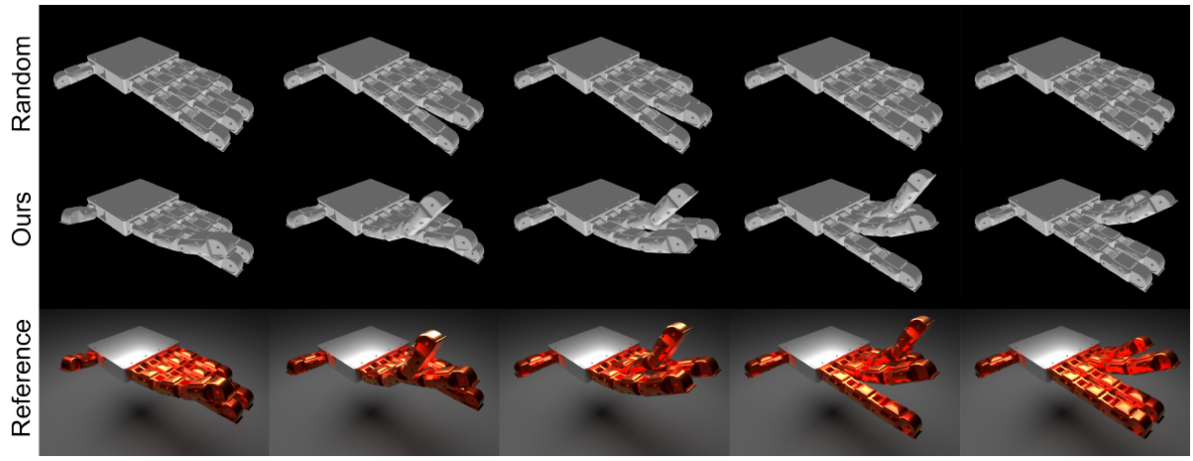


图 2-3: 手环境中的模仿学习（第 2.4.4 节）. 给定参考视频（下排，以五个中间帧显示），目标是重建一个动作序列，使其运动与之相似. 图中展示了以随机选择的动作初始猜测值（上排）和使用我们带渲染梯度方法优化后的动作（中排）生成的运动. 图内 Random、Ours、Reference 分别表示随机初始值、我们的方法和参考视频.

	四旋翼	手	杆
average	28.40 ± 0.00	7.62 ± 0.00	30.20 ± 0.00
random	1120 ± 113	10.27 ± 0.91	29.89 ± 0.61
pixelwise-loss	$12.65 \pm \mathbf{0.13}$	$6.71 \pm \mathbf{0.07}$	29.57 ± 0.85
perceptual-loss	N/A	5.10 ± 1.80	14.61 ± 5.13
ours-no-grad	25.07 ± 5.76	7.12 ± 0.71	$\mathbf{0.83} \pm 0.35$
ours	$\mathbf{2.63} \pm 1.86$	$\mathbf{1.52} \pm 0.18$	$1.05 \pm \mathbf{0.27}$
oracle	0.79 ± 0.44	0.02 ± 0.02	0.28 ± 0.16

表 2.3: 模仿学习结果（第 2.4.4 节）. 每一项报告由四组随机生成的初始猜测值和渲染条件计算出的状态差异的均值和标准差. N/A 表示优化后未能收敛.

旋翼飞行器的真实动力学受到环境噪声和复杂空气动力学效应的干扰，而这些效应被我们的可微仿真环境忽略. 尽管存在这些挑战，我们的方法使用标准可微刚体仿真器和渲染器，在定性上取得了良好结果，显示出对动力学模型和渲染配置中等程度差异的泛化能力（图 2-4）. 在此任务中，我们的方法大幅优于所有基线，细节见附录.

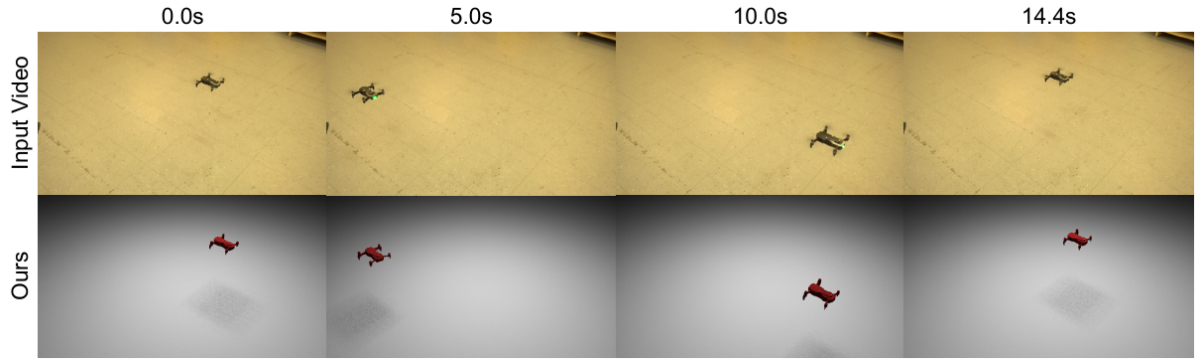


图 2-4: 真实世界实验中的模仿学习（第 2.4.5 节）. 给定参考视频（上排），目标是重建发送给虚拟四旋翼飞行器的动作序列，使其运动与参考视频相似. 下排展示了我们的方法重建的运动. 图内 Input Video 和 Ours 分别表示输入视频和我们的方法.

2.5 结论、局限性和未来工作

我们提出了一个框架，将渲染不变状态预测融入可微仿真和渲染，以进行跨域参数估计. 仿真和真实世界实验表明，在未见过的渲染配置下，我们的方法比逐像素损失或感知损失更加稳健. 额外的消融研究进一步证实，其效率和有效性来自在 RISP 网络中使用渲染梯度.

尽管取得了有前景的结果，RISP 仍有一些局限. 首先，我们需要能够捕捉对象动力学模型的可微仿真器，而对于动力学复杂的真实场景，例如流体，这种仿真器可能不可用. 第二个局限是，我们的方法需要知道相机参数，而真实场景中这些参数并不总能获得. 这一局限可以通过将现代可微渲染器提供的相机参数梯度纳入 RISP 来解决. 最后，我们假设可以获得中等精度的物体几何形状；这一假设有可能通过将 NeRF [122] 与我们的方法结合以推断几何形状而放宽.

Chapter 3

从运动观测中学习神经本构律以实现可泛化的偏微分方程动力学

3.1 引言

由偏微分方程控制的动力系统广泛存在于物理、化学和生物领域 [52]. 通过计算求解这些偏微分方程，例如使用有限元方法（FEM）[71]，已使科学与工程中的重要应用成为可能. 近期，研究界探索了机器学习（ML）在推进偏微分方程（PDE）建模中的作用 [81]. 某些方法，例如物理信息神经网络（PINN）[146]，假设整个 PDE 已知；另一些方法，例如图神经网络（GNN）[154, 137]，则不假设对底层 PDE 有任何了解，而是完全从数据中学习由 PDE 控制的动力学. 在对 PDE 没有任何了解的情况下，这些方法容易违反物理定律，且泛化能力有限. 本工作认为，与用神经方法学习整个 PDE 相比，仅将 PDE 的部分组成替换为神经组件、同时保持 PDE 的其余部分不变，具有一定优势. 例如，我们主张，弹性动力学方程 [50] 等被普遍接受的物理定律无需通过学习获得，而应显式施加，以保证物理正确性. 相比之下，我们认为 PDE 中那些研究人员传统上难以建模的部分尤其能够受益于学习方法. 具体而言，我们针对许多 PDE 中不可或缺的组成部分——本构律——探索一种机器学习方法.

本构律描述两个物理量之间的关系 [175]，例如固体中的应力与应变 [49]、流体中的剪切应力与剪切率 [174]，以及电磁学中的电位移场与电场 [92]. 这些本构律在弹性动力学方程、Navier–Stokes 方程和 Maxwell 方程等重要 PDE 中发挥关键作用.

传统上，本构律由领域专家手工设计. 它们应满足普遍接受的约束，例如标架无关性（旋转等变性）；但衡量本构律优劣的最终标准是它与实验观测数据的吻合程度，而这种关系往往是高度非线性的. 因此，为设计本构关系，研究人员依赖多种数据拟合工具，从高阶多项式 [128] 和样条 [198] 到指数模型 [61]. 近期，得益于神经网络强大的学习能力，基于神经网络的模型已被证明能够比经典的专家构造模型获得更高的准确度 [48, 94]. 此外，由于相同的网络架构可以用于捕捉多种力学行为，这些模型减少了手工设计函数形式的需要 [108]. 事实上，我们在本工作中表明，单一神经网络架构能够捕捉从水到弹性固体的多种本构行为，而这些行为以前需要逐一由专家设计本构模型.

通过神经网络学习本构律时，有两个突出的挑战. 第一，如何使这些神经本构律遵循已知物理先验，例如标架无关性？第二，如何获得用于监督训练的标记数据？例如，学习弹性时必须获得标记的应力–应变对 [8, 38, 44]. 然而，这些应力–应变对往往要么无法测量，例如体内医学成像 [1]，要么需要高度专门化的设备 [14].

为解决第一个问题，我们将物理先验作为归纳偏置引入网络架构，通过设计将这些先验直接内嵌其中. 这样，我们的网络无需大量数据增强便能自然满足必要的先验. 为解决标记数据的挑战，我们将可学习的本构模型嵌入基于 PDE 的可微物理仿真器 [126]，并对该物理仿真器的输出而非本构模型本身的输出进行监督训练. 具体来说，我们的方法使用运动观测数据（在连续介质力学文献中形式化定义为运动学信息）进行训练，例如物质点位置；这类数据比应力信息容易测量得多.

使用所提出的本构学习方法，我们展示了纯神经网络（NN）方法（例如 GNN）难以处理的可泛化 PDE 动力学. 这些泛化任务包括在仅用单条运动轨迹训练的情况下，跨时间范围、初始 / 边界条件、几何形状和多物理系统进行极端分布外泛化. 我们将相对纯 NN 方法的泛化优势归因于：我们只学习 PDE 的一个组成部分，同时保持 PDE 中被广泛接受的部分不变（例如质量和动量守恒）.

概括而言，本工作的贡献如下：

- 我们将可学习的本构律嵌入可微物理仿真器，并直接利用仿真器输出训练该本构律.
- 我们设计具有矩阵层面的旋转等变性和未变形状态平衡的网络架构，保证本构律满足物理先验.
- 我们在大变形 PDE 动力学上验证 NN–PDE 混合方法；该方法经单次训练后能够泛化到未见过的时间范围、初始条件、边界条件、几何形状和多物理系统，而以往的纯 NN 方法在这些任务上失败.

3.2 相关工作

本构律. 本构律可追溯至 1676 年，当时英国皇家学会会员 Robert Hooke 观察到弹簧力与变形之间的线性关系，即 $F_s = kx$ [170]. 此后，本构律成为物理学各分支的基础，包括弹性 [172, 43, 7]、塑性 [123, 34]、流体 [26] 和介电性质 [92]. 尽管这些本构律传统上通过非线性多项式基构造 [198, 197]，研究界已因神经网络的多功能性和稳健性而开始采用它们 [158, 169, 190, 186, 185, 42, 166, 184, 183, 87, 106]. 大多数基于神经网络的方法需要本构模型输入和输出的标记数据. 相比之下，我们的工作不需要这类数据，而是将神经本构律嵌入基于可微仿真的训练流程.

可微仿真. 可微仿真以可微方式实现传统 PDE 仿真（例如 FEM），使仿真能够用于基于梯度的优化流程 [55, 104, 114, 66, 67, 70, 37, 36, 142, 141, 35, 30, 47, 196, 32]. 值得注意的是，这些可微物理仿真能够直接从视频和运动观测中恢复本构参数 [126, 113, 22]，例如弹性的杨氏模量和泊松比. 以往的可微仿真工作假设专家设计的本构律先验已知. 然而，给定任意材料的运动观测序列，从业者识别其底层本构模型并非易事，更不用说恢复其参数. 相比之下，我们不假设材料模型可用，而是通过普遍适用的神经网络架构参数化本构律. Wang 等 [188] 和 Huang 等 [68] 也展示了直接从运动观测训练神经本构律的有前景结果，但其表述分别局限于各向同性弹性材料和二维准静态加载. 我们的通用表述既能处理各向同性和各向异性材料，也能处理随时间变化的三维弹塑性动力学.

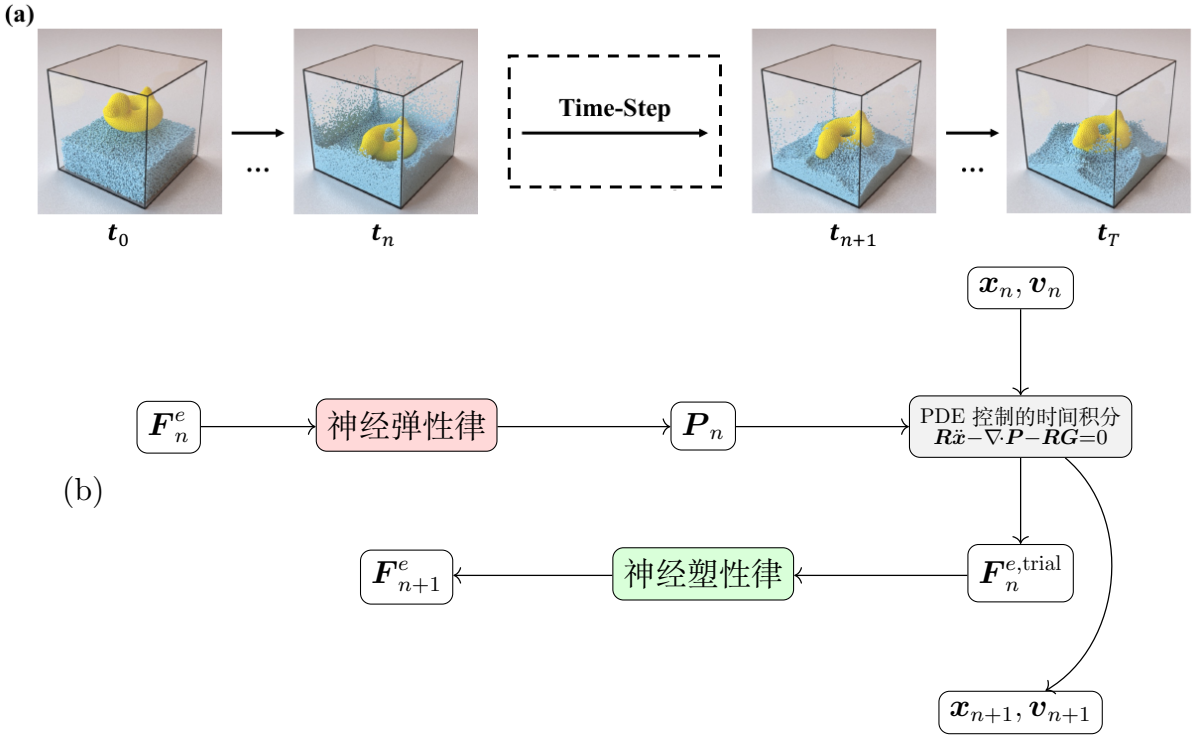


图 3-1: NN–PDE 混合时间步进. (a) 我们的方法通过连续 “时间步进” 求解动力系统. (b) 在时间步进算法内部，我们先使用神经弹性本构律得到应力，再求解控制 PDE（式 (3.1)）更新状态，最后通过神经塑性本构律得到新的弹性变形梯度. 相应伪代码见算法 2. 上方原图中 Time-Step 表示时间步进；下方图按原图重排，保留原图的速度符号 $\boldsymbol{v}$.

PDE 动力学的机器学习 (ML). 我们的工作属于更广泛的 PDE 机器学习研究. 物理信息神经网络 (PINN) [146] 用神经表征表示 PDE 的解, 并通过 PDE 信息损失引入物理先验. DeepONet [112] 和神经算子 [103] 学习从问题参数到解的映射, 目标是获得快速代理模型. 与这些方法不同, 我们的工作完全处于经典数值求解器框架内. 我们只为经典求解器的一个组成部分——PDE 的本构模型——提出机器学习方案. 还有许多优秀工作仅用神经网络替换 PDE 的一个部分, 尽管其背景并非本构律. 例如 Bar-Sinai 等 [10] 的数据驱动离散化、Yin 等 [204] 的 APHYNITY, 以及 Um 等 [178] 的求解器在环方法. 另一个稳健的 ML–PDE 框架是图神经网络 (GNN). GNN 直接学习每个时间步施加于离散粒子的力 [154, 137], 因而隐式地同时学习本构律 (材料模型) 和运动方程 (即弹性动力学方程). 因此, 我们的工作不同于 GNN: 我们引入了在标准连续介质力学条件下成立的运动方程先验 [50], 仅对本构模型采用 NN 方法. 相关地, Li 等 [100] 利用基于神经网络的塑性能量增强经典数值求解器, 旨在改进优化式时间积分器. 相比之下, 我们的工作聚焦于本构律, 而不是时间积分器.

3.3 方法

给定材料发生变形的运动序列, 我们的目标是获得由神经网络表示的本构模型. 这些本构律随后可用于预测训练运动序列中未见过的场景. 第 3.3.1 节首先回顾当时从运动观测辨识经典本构参数的先进框架; 第 3.3.2 和 3.3.3 节将介绍所提出的神经替代方案.

3.3.1 背景

连续 PDE. 本工作假设实验中的所有材料都遵循弹性动力学方程 [49]:

$$\rho_0 \ddot{\boldsymbol{\phi}} = \nabla \cdot \boldsymbol{P} + \rho_0 \boldsymbol{b}. \tag{3.1}$$

所关注的向量场是变形映射 $\boldsymbol{\phi}$, 它唯一确定初始位置为 $\boldsymbol{X}$ 的任意空间点在时刻 t 的当前位置 $\boldsymbol{x}$, 即 $\boldsymbol{x} = \boldsymbol{\phi}(\boldsymbol{X}, t)$. 此外, ρ_0 是初始密度, $\boldsymbol{P} = \hat{\boldsymbol{P}}(\boldsymbol{F}^e)$ 是第一 Piola–Kirchhoff 应力, $\boldsymbol{F}^e$ 是变形梯度 ($\nabla\boldsymbol{\phi}$) 的弹性部分, $\dot{\boldsymbol{\phi}}$ 和 $\ddot{\boldsymbol{\phi}}$ 分别是速度和加速度, $\boldsymbol{b}$ 是体力. 原则上, 我们的方法也可扩展到其他涉及时空梯度的 PDE, 例如 Navier–Stokes 方程.

本构律. 为使式 (3.1) 定义良好, 必须指定本构关系. 弹性本构律定义 $\boldsymbol{P}$ 和 $\boldsymbol{F}^e$ 之间的关系: $\boldsymbol{P} = \hat{\boldsymbol{P}}(\boldsymbol{F}^e)$. 塑性本构律定义一个不等式约束 $f_Y(\boldsymbol{P}) < 0$ (其中 f_Y 是屈服函数 [16]), 以及违反该约束时的流动法则. 这些形式的本构律已被证明能够描述从弹性固体到颗粒介质的多种材料.

离散化. 为数值求解式 (3.1), 我们在空间和时间上将其离散化, 得到动力系统 $\mathcal{M}$:

$$\boldsymbol{s}_{n+1} = \mathcal{M}_\mu(\boldsymbol{s}_n), \quad \forall n = 0, 1, \ldots, T, \tag{3.2}$$

其中 T 是仿真的总时间步数, $\boldsymbol{s}_n$ 和 $\boldsymbol{s}_{n+1}$ 是相应时间步的状态向量. 向量 μ 编码所有系统参数, 例如本构律的参数.

为获得动力系统 $\mathcal{M}$, 我们可以使用任何离散化技术, 例如 FEM、有限体积法 [39] 或光滑粒子流体动力学 [124]. 本工作采用物质点法 (MPM) [164, 76, 65], 因为它能够

灵活处理多种材料. 在 MPM 中，我们用 Q 个物质点离散化区域，$\mathcal{M}_\mu$ 即成为算法 1. 这里，状态向量 $\boldsymbol{s}_n = \{\boldsymbol{x}_n, \boldsymbol{b}_n, \boldsymbol{F}_n^e\}$ 包含所有离散物质点的当前位置、速度和弹性变形梯度. 时间积分方案 $\mathcal{I}$ 遵循标准 MPM 做法，由式 (3.1) 的弱形式导出. 附录 B.1 提供了更多 MPM 背景.

我们强调，本工作的主要贡献与离散化方法的选择正交. 时间积分方案 $\mathcal{I}$ 也可替换为其他由 PDE 控制、从弱形式导出的数值方法，例如 FEM. 我们的主要贡献是所有数值方法都需要的连续本构律.

事实上，时间步进方案涉及两个本构律：

$$\begin{aligned} \mathcal{E}_\mu &: \boldsymbol{F}^e \mapsto \boldsymbol{P} & \mathcal{P}_\mu &: \boldsymbol{F}^{e,\text{trial}} \mapsto \boldsymbol{F}^{e,\text{new}} \\ &: \mathbb{R}^{3\times3} \to \mathbb{R}^{3\times3} & &: \mathbb{R}^{3\times3} \to \mathbb{R}^{3\times3}. \end{aligned} \tag{3.3}$$

算法 1：时间步进，经典方法

输入： $\boldsymbol{s}_n = \{\boldsymbol{x}_n, \boldsymbol{b}_n, \boldsymbol{F}_n^e\}$

输出： $\boldsymbol{s}_{n+1} = \{\boldsymbol{x}_{n+1}, \boldsymbol{b}_{n+1}, \boldsymbol{F}_{n+1}^e\}$

1. 对 $i = 1, \ldots, Q$，$\boldsymbol{P}_n(i) = \mathcal{E}_\mu(\boldsymbol{F}_n^e(i))$.
2. $\boldsymbol{x}_{n+1}, \boldsymbol{b}_{n+1}, \boldsymbol{F}_{n+1}^{e,\text{trial}} = \mathcal{I}(\boldsymbol{x}_n, \boldsymbol{b}_n, \boldsymbol{F}_n^e, \boldsymbol{P}_n)$.
3. 对 $i = 1, \ldots, Q$，$\boldsymbol{F}_{n+1}^e(i) = \mathcal{P}_\mu(\boldsymbol{F}_{n+1}^{e,\text{trial}}(i))$.

弹性本构律 $\mathcal{E}_\mu$ 从弹性变形梯度 $\boldsymbol{F}_n^e$ 计算第一 Piola–Kirchhoff 应力 $\boldsymbol{P}_n$. 塑性返回映射方案 $\mathcal{P}_\mu$ [31] 将试探弹性变形梯度投影到塑性屈服约束 f_Y 上. $\mathcal{E}_\mu$ 和 $\mathcal{P}_\mu$ 的例子列于附录 B.2.

从运动观测进行系统辨识. 一旦获得由 PDE 控制的仿真，我们便能直接从运动观测中辨识本构参数 [113]. 具体地，我们可以基于真实观测的粒子位置 $\boldsymbol{x}_n^{\text{gt}}$ 定义损失函数：

$$\min_\mu \mathcal{L}(\{\boldsymbol{x}_n^{\text{gt}}\}_{n=0}^T, \{\boldsymbol{x}_n\}_{n=0}^T), \tag{3.4}$$

其中 $\mathcal{L}$ 是损失度量. 通过以可微方式实现仿真（算法 1），我们可以使用标准的基于梯度的优化技术求解式 (3.4)，优化与运动观测相匹配的本构参数.

3.3.2 神经本构律

本工作用基于神经网络的模型 $\mathcal{E}_{\theta_e}$ 和 $\mathcal{P}_{\theta_p}$ 替代经典弹性和塑性本构处理，其中 θ_e 和 θ_p 是神经网络权重. 此时时间步进方案变为算法 2. 图 3-1 示意了 NN–PDE 混合时间步进方案. 值得注意的是，我们只用神经网络表示本构律，而保持基于经典 PDE 的时间积分 $\mathcal{I}$ 不变. 时间积分方案 $\mathcal{I}$ 由应力散度计算力，并通过标准 Euler 方法 [9] 更新粒子位置和速度. 由于这些步骤遵循已知的物理定律，我们认为无需通过学习获得.

算法 2：时间步进，神经方法

输入： $\boldsymbol{s}_n = \{\boldsymbol{x}_n, \boldsymbol{b}_n, \boldsymbol{F}_n^e\}$

输出： $\boldsymbol{s}_{n+1} = \{\boldsymbol{x}_{n+1}, \boldsymbol{b}_{n+1}, \boldsymbol{F}_{n+1}^e\}$

1. 对 $i = 1, \ldots, Q$， $\boldsymbol{P}_n(i) = \mathcal{E}_{\theta_e}(\boldsymbol{F}_n^e(i))$.

2. $\boldsymbol{x}_{n+1}, \boldsymbol{b}_{n+1}, \boldsymbol{F}_{n+1}^{e,\text{trial}} = \mathcal{I}(\boldsymbol{x}_n, \boldsymbol{b}_n, \boldsymbol{F}_n^e, \boldsymbol{P}_n)$.

3. 对 $i = 1, \ldots, Q$， $\boldsymbol{F}_{n+1}^e(i) = \mathcal{P}_{\theta_p}(\boldsymbol{F}_{n+1}^{e,\text{trial}}(i))$.

为利用运动观测数据进行训练，我们也采用与式 (3.4) 相同的目标函数，但优化变量变为神经网络权重：

$$\min_{\theta_e, \theta_p} \mathcal{L}(\{\boldsymbol{x}_n^{\text{gt}}\}_{n=0}^T, \{\boldsymbol{x}_n\}_{n=0}^T). \tag{3.5}$$

我们还强调，对本构律不假设一般性的骨干形式. 假设某个特定骨干（例如 neo-Hookean 弹性）会妨碍我们捕捉其他类型的本构行为. 因此，我们选择不假设任何骨干，而是用 NN 表示整个本构律；NN 是具有更好表达能力和多功能性的数据驱动通用表征. 不过，本构律之外的框架可以视为基于 PDE 的通用骨干. 在这个意义上，我们与 Yin 等 [204] 利用 NN 增强物理模型的理念相似.

3.3.3 物理感知网络架构

我们的神经本构律必须满足基本的物理先验 [187]. 同时，我们也避免受到过强先验的约束，例如各向同性先验会妨碍模型捕捉各向异性材料 [188]. 本工作考虑两个普遍适用的物理先验.

旋转等变性. 物理世界中大多数材料的性质在旋转下保持不变，这一性质也称为标架无关性. 不同于以往聚焦向量数据（例如点云）的工作 [33]，我们必须保证变形梯度的旋转等变性，而变形梯度是方阵. 为此，我们将一组选定的旋转不变量输入神经网络，包括主伸长 $\boldsymbol{\Sigma}$（奇异值）、右 Cauchy–Green 张量 $\boldsymbol{F}^{eT}\boldsymbol{F}^e$，以及弹性变形梯度的行列式 $\det(\boldsymbol{F}^e)$. 然后，我们通过极分解 $\boldsymbol{F}^e = \boldsymbol{R}\boldsymbol{S}$ 从变形梯度中分离出旋转矩阵 $\boldsymbol{R}$，再将它乘回神经网络的输出，以保证旋转等变性. 附录 B.3 提供了详细算法，以及神经网络和神经本构律旋转不变性的证明.

未变形状态平衡. 另一个重要的本构先验是零载荷下保持静止形状. 我们通过以下方式保证未变形状态平衡：将输入不变量归一化，使其在零载荷下为零；并移除神经网络中的所有偏置层. 因此，当施加零载荷时，弹性网络产生零应力，而塑性网络保持试探弹性变形梯度不变.

3.4 实验

本节首先介绍实验设置（第 3.4.1 节），随后在重建（第 3.4.2 节）和泛化（第 3.4.3 节）任务中，将我们的方法与基线和预言机比较. 我们还在多物理环境（第 3.4.5 节）和真实世界实验（第 3.4.6 节）这两项进阶实验中研究方法. 读者可访问项目网站[2]，其中通过视频展示结果的效果最佳.

[2] https://sites.google.com/view/nclaw

3.4.1 实验设置

环境. 我们考虑四种不同的弹塑性材料，涵盖多种物理效应：纯弹性材料（Jell-O，果冻）、具有 Drucker–Prager 屈服条件的弹性材料（Sand，沙）、具有 von Mises 屈服条件的弹性材料（Plasticine，橡皮泥），以及弱可压缩流体（Water，水）. 这些真实材料模型的数学定义和实现详见附录 B.2.

基线. 我们将方法与三个强基线比较. spline [198] 用 Bézier 样条对弹性本构律建模，其中控制点是待优化参数. neural [188] 建立在 spline 基础上，但用神经网络替代 Bézier 样条. gnn [154] 使用图神经网络学习粒子动力学，不对底层 PDE 作任何假设.

我们的方法. 除充分训练后得到的结果（ours）外，我们还展示未使用式 (3.5) 训练时方法在初始阶段的表现（ours-init），以强调学习的有效性.

预言机. 我们考虑两个使用通常无法获取的额外知识的预言机. labeled [8] 使用带标记的真实 $\boldsymbol{P}$ 和 $\boldsymbol{F}^{\mathrm{e,new}}$ 进行监督，而不是使用运动观测. sys-id [113] 假定本构模型已给定，只辨识材料参数（例如杨氏模量）.

实现细节. 训练时，我们将一个包含 1000 个均匀物质点的立方体放在箱子中央，并赋予固定的线速度和角速度. 我们以 $5 \times 10^{-4}\,\mathrm{s}$ 的步长模拟立方体运动，共 1000 个时间步. 对每个环境，我们仅生成一条轨迹作为训练数据集. 神经网络使用三层多层感知机（MLP），每层 64 个神经元. 我们的定量结果报告每隔 5 帧的平均均方误差，因为按照文献 [154]，gnn 以 5 倍步长训练. 更多实现细节见附录 B.4.

3.4.2 重建

我们首先在每个环境的一条轨迹上训练所有方法，以重建运动序列. 表 3.1 定量比较了重建损失. 我们发现，spline 在所有环境中始终得到较差结果，这可能是由于不连续控制点搜索导致的优化困难，以及二次样条有限的表达能力. 其他基线取得了合理表现：neural 能很好地重建果冻和水，但由于缺乏塑性模型，在所有其他高度依赖塑性的环境中表现吃力；gnn 在所有环境中通常表现尚可. 与基线不稳定或次优的表现相比，ours 的重建损失一般低于 10^{-3}，并且除果冻环境中 neural 表现最佳外均排名第一. 原因是 neural 假设不存在塑性，而果冻恰好如此，从而使训练更容易. 我们还在 ours-init 中报告方法的初始表现，以强调我们的方法从零开始学习，不针对单个环境进行临时调参. 两个预言机 labeled 和 sys-id 在几乎所有环境中都取得近乎完美的结果. 唯一例外是 labeled 在沙环境中表现次优，该环境具有尤其困难的压力依赖塑性律. 因为 labeled 直接由真实的 $\boldsymbol{P}$ 和 $\boldsymbol{F}^{\mathrm{e,new}}$ 监督，我们将这一性能下降归因于缺少时间反向传播（BPTT）. BPTT 自动放大累积仿真损失，并引导神经网络学习长期稳定性.

3.4.3 泛化

在第 3.4.2 节的一条轨迹上训练后，我们直接将模型部署到具有分布外条件的四项任务，以评估方法的泛化能力：(a) 时间范围加倍，(b) 未见过的线速度和角速度，(c) 有挑战性的几何形状，以及 (d) 倾斜平面边界. 我们还在 (d) 中使用了更具挑战性的几何形状，并将 (c) 和 (d) 的粒子数增加至约 30000 个（训练时的 30 倍）. 我们强调，单次训练后的泛化并非易事：训练数据仅为 1000 个粒子在 1000 个时间步内的位置. 更多

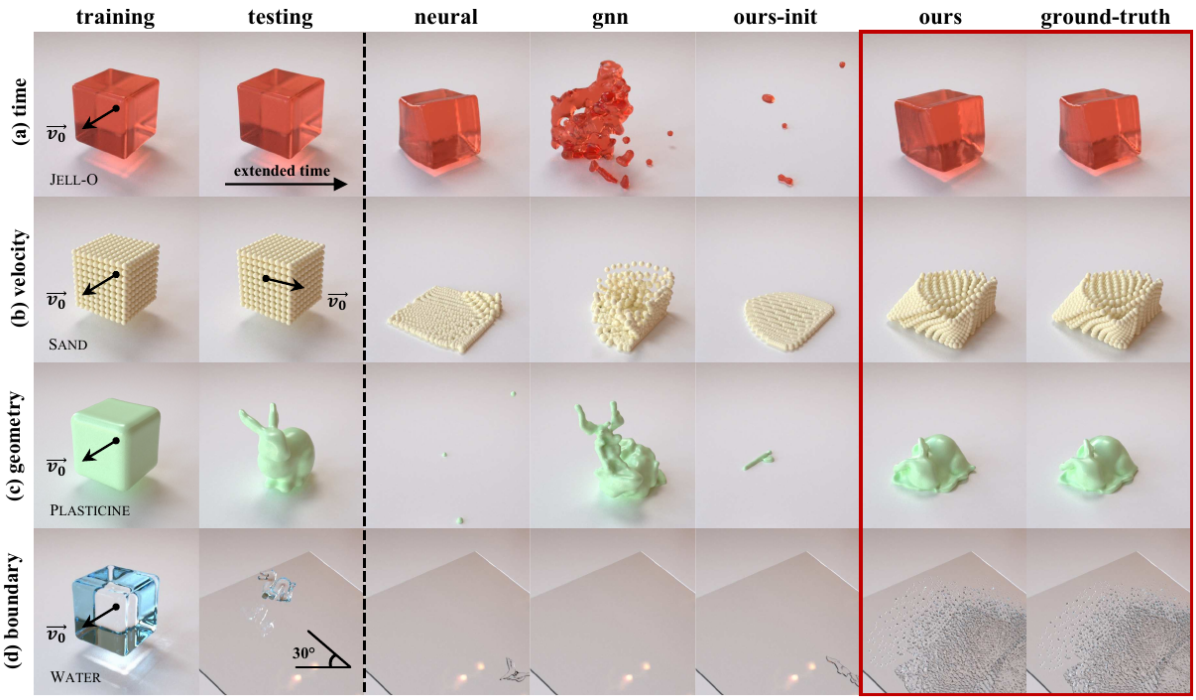


图 3-2: 泛化. 我们首先在 training 指定的环境中训练所有方法，初始速度由黑色箭头表示. 然后，在 testing 指定的四项任务中评估所有方法的泛化能力：(a) 延长时间，(b) 未见过的初始速度，(c) 有挑战性的几何形状，(d) 倾斜边界. 虚线左侧显示初始化，右侧显示经过一段时间后的仿真结果. 我们将方法（ours）与基线（neural 和 gnn）、我们方法的初始化（ours-init）及真实仿真（ground-truth）比较，并以红框标出 ours 和 ground-truth. 图中 training、testing 分别为训练和测试；time、velocity、geometry、boundary 分别为时间、速度、几何形状和边界；Jell-O、Sand、Plasticine、Water 分别为果冻、沙、橡皮泥和水.

Method	Jell-O	Sand	Plasticine	Water	**Overall**
spline	2.4e-1	3.2e-1	3.0e-1	3.2e-1	2.9e-1±3.5e-2
neural	1.2e-5	1.2e-2	1.9e-1	2.9e-2	5.7e-2±8.7e-2
gnn	2.1e-2	1.1e-2	8.7e-3	3.2e-2	1.8e-2±1.0e-2
ours-init	1.1e-1	4.8e-2	4.6e-2	6.8e-2	6.8e-2±2.8e-2
ours	2.4e-4	2.6e-5	6.5e-5	2.0e-5	8.6e-5±1.0e-4
labeled	8.0e-9	7.1e-2	5.5e-6	1.6e-7	1.8e-2±3.5e-2
sys-id	1.7e-8	2.7e-7	5.8e-10	1.7e-8	7.6e-8±1.3e-7

表 3.1: 重建. 不同方法的重建损失. 对每个环境，基线及我们的方法中排名第一者以深绿色表示，第二者以浅绿色表示；最后一列给出每种方法的统计量（均值 ± 标准差）. 图中 Method、Overall 分别表示方法和总体；材料名称译法见正文.

可视化见附录 B.6.

我们选择实验的代表性子集，在图 3-2 中给出定性比较. neural 的表现与重建结果一致：即使时间范围加倍，它在果冻上仍能很好地泛化，但在其他高度依赖塑性的任务上均失败. gnn 在不同于训练环境的条件下无法生成视觉上合理的仿真结果. 我们认为，由于 gnn 不假设任何底层 PDE 知识，训练出具有泛化能力的 gnn 需要比我们多得多的数据. 事实上，Sanchez-Gonzalez 等 [154] 在类似设置中使用至少 1000 条运动轨迹训练，而我们的方法只使用一条. 在所有任务上，ours 得到的视觉结果与 ground-truth 相似. 我们还展示 ours-init，以说明从随机初始化到训练后模型的显著改进.

表 3.2 定量比较了我们的方法和基线. 总体而言，ours 在大多数环境和任务中排名第一，并以数个数量级的优势超过其他基线. 与重建结果一致，我们的方法在果冻上的损失虽然足够小，但仍比 neural 差. 然而，如图 3-2 所示，这一差距很小，二者的视觉差异可以忽略.

Method	Jell-O			Sand			Plasticine			Water			Overall
	(a)	(b)	(c)	(a)	(b)	(c)	(a)	(b)	(c)	(a)	(b)	(c)	
spline	2.6e-1	2.4e-1	3.5e-1	3.5e-1	3.1e-1	3.6e-1	3.0e-1	3.0e-1	3.1e-1	3.7e-1	3.2e-1	3.3e-1	3.1e-1±4.0e-2
neural	2.9e-5	1.4e-5	5.6e-5	4.7e-2	6.2e-3	2.3e-1	2.4e-1	4.1e-3	2.8e-1	4.6e-2	1.7e-2	5.1e-2	4.9e-2±8.8e-2
gnn	3.3e-2	1.5e-2	1.6e-1	1.5e-2	1.7e-2	3.4e-1	1.1e-2	6.8e-3	3.7e-2	1.1e+0	5.8e-2	2.9e-1	1.2e-1±2.6e-1
ours-init	1.5e-1	6.6e-2	1.2e-1	1.5e-1	2.7e-2	5.0e-2	1.3e-1	2.7e-2	5.8e-2	3.0e-1	4.1e-2	1.0e-1	7.7e-2±6.9e-2
ours	9.8e-4	2.4e-4	4.1e-4	4.2e-5	6.5e-5	3.6e-4	1.4e-4	4.6e-5	2.3e-4	3.5e-4	1.9e-5	2.4e-4	1.9e-4±2.3e-4
labeled	1.1e-7	7.8e-8	1.7e-4	2.2e-1	1.8e-3	3.6e-1	1.5e-5	7.5e-6	1.1e-4	2.1e-6	2.2e-7	1.8e-6	2.9e-2±9.2e-2
sys-id	4.0e-8	6.1e-9	1.1e-8	3.7e-7	4.4e-8	1.6e-7	7.2e-10	1.3e-10	1.4e-9	2.6e-7	7.2e-9	1.4e-8	5.1e-8±9.9e-8

表 3.2: 泛化. 不同方法的泛化损失. 定量结果对应图 3-2 的三项主要任务：(a) 延长时间，(b) 未见过的初始速度，(c) 有挑战性的几何形状. 对任务 (b)，我们使用三个随机初始速度评估并报告平均损失，以进行更充分的验证. 对各环境中的每项任务，所有基线和我们的方法中排名第一者以深绿色表示，第二者以浅绿色表示；最后一列给出各方法的均值 ± 标准差. Method 为方法，Overall 为总体；材料名称译法见正文.

3.4.4 极端泛化

为研究方法的极限，我们设计实验对物理仿真中的两个重要因素进行压力测试：粒子数量和碰撞条件. 结果见图 3-3. 上排使用水环境，以龙形几何体作为初始形状；我们将仿真粒子数增加到超过 100 万，以压力测试方法的稳健性. 结果表明，即使经过长时间演化，我们的方法仍能产生与传统仿真的真实结果相符的预测. 我们强调，部署的神经网络仅用 1000 个粒子训练. 下排初始化了四个由果冻材料构成的橡胶玩具，并以较大的初速度使它们剧烈相撞；图中展示碰撞发生后的时刻. 由于本构律与仿真其余部分相互解耦，即使碰撞条件在训练时完全未见过，我们的方法仍能很好地泛化到该条件.

3.4.5 多物理泛化

由于模型仅使用包含单一材料的一条轨迹训练，预测多物理环境中的材料响应并非易事. 我们构造了图 3-4 左侧所示的两个具有挑战性的多物理环境，进一步检验这一能力. ground-truth 行展示传统仿真生成的真实运动，ours 行展示我们方法的预测.

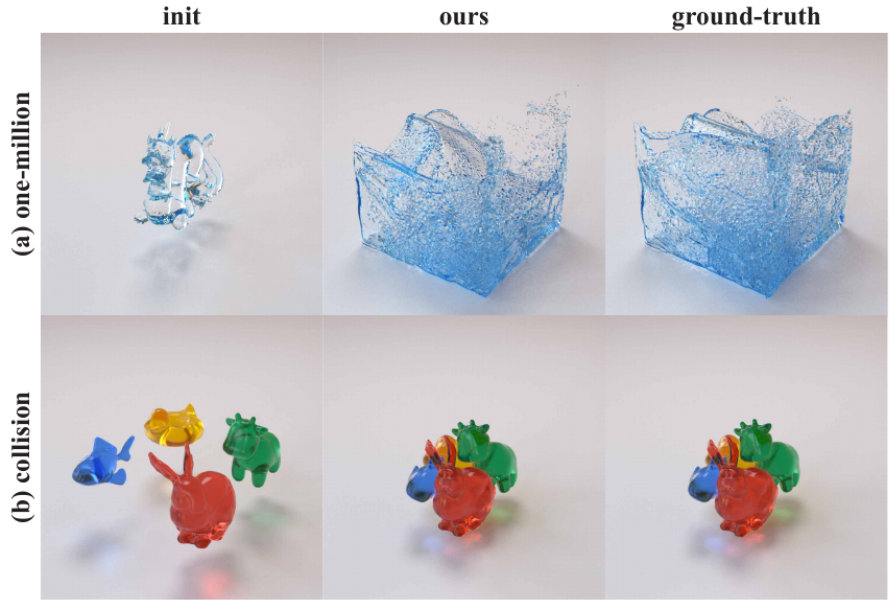


图 3-3: 极端泛化. 我们在两项极端泛化实验中比较初始状态、我们的方法以及传统仿真的真实结果：(a) 包含超过 100 万个粒子的复杂水龙；(b) 四个橡胶玩具相互及与地面剧烈碰撞. 图中 init、ours、ground-truth 分别表示初始状态、我们的方法和真实结果；one-million、collision 分别表示百万粒子和碰撞.

图 3-4(a) 中，我们用与果冻相同的材料模拟橡皮鸭，并将其投入一个大水池；二者通过 MPM 耦合. 橡皮鸭由 8086 个粒子建模，水由 94208 个粒子建模，总数超过 10 万. 几乎可以忽略的视觉差异表明，我们的方法在这一具有挑战性的大规模多物理环境中具有准确性. 图 3-4(b) 中，我们初始化一个由橡皮泥制成的冰淇淋，并自下而上逐渐将本构律转变为沙，以模仿 “融化” 效果. 即便存在随时间变化的相变，我们的方法仍保持稳定和准确，与真实结果的差异很小.

3.4.6 真实世界实验

真实材料通常呈现复杂行为，准确建模既困难又费力. 为进一步评估方法，我们选择真实面团作为测试材料，它在弹性和塑性方面均具有丰富行为. 如图 3-4 右侧所示，我们录制面团从高处落下的运动视频，并在面团上标记绿色点，以跟踪材料粒子尺度的变形. 我们直接依据带标记的粒子建立仿真，并以这些真实数据为监督训练方法. 注意，该环境是真正的三维环境，以反映面团的真实材料性质. 为仅用二维监督训练三维 NCLaw，我们引入额外先验：一个最小化深度方向速度大小的正则化损失. 图 3-4(c) 的实验表明，我们的方法重建了面团撞击地面的真实轨迹. 训练后，我们直接将模型应用于由相同材料制成、未见过的星形面团，如图 3-4(d) 所示. 我们的方法捕捉到了非平凡的弹塑性变形，并在这种新的复杂真实几何形状上实现单次训练后的泛化. 实验设置和实现详见附录 B.4.

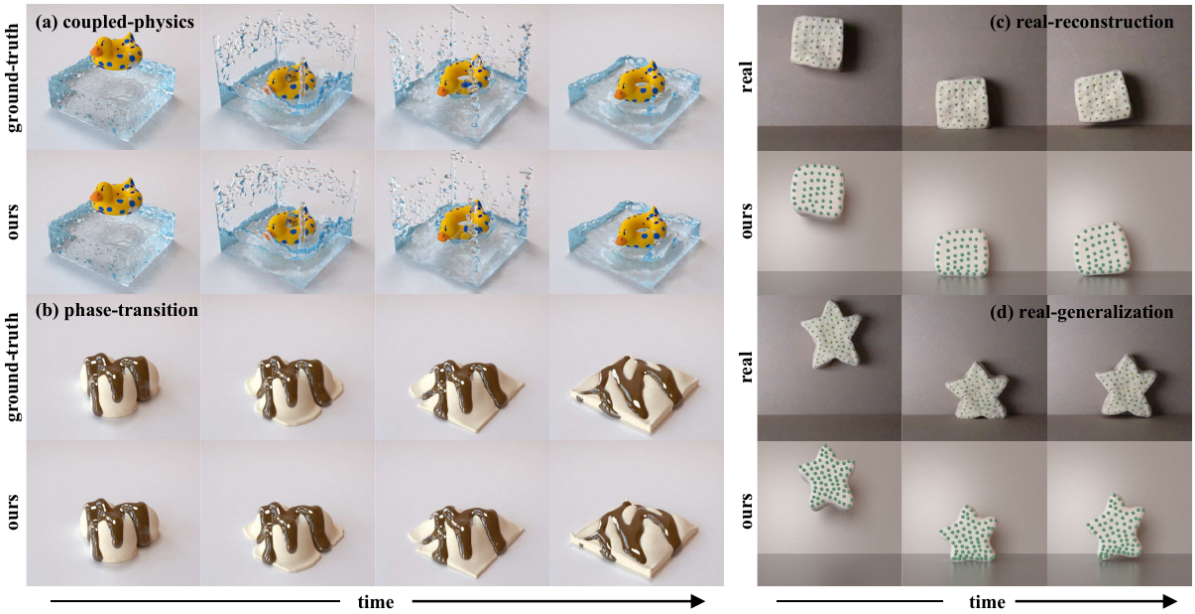
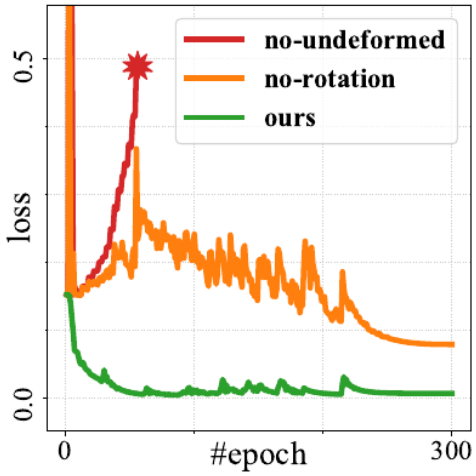


图 3-4: 进阶实验. 左: 在两个多物理环境中比较我们的方法与传统仿真的真实结果: (a) 橡皮鸭落入水池; (b) 融化的冰淇淋逐渐从一种材料转变为另一种材料. 右: 我们用真实面团下落数据训练方法, 学习面团的本构律: (c) 方法成功重建训练序列; (d) 方法能够很好地泛化到未见过的条件, 例如复杂几何形状. 图中 coupled-physics、phase-transition、real-reconstruction、real-generalization 分别为耦合物理、相变、真实重建和真实泛化; ground-truth、real、ours、time 分别为真实仿真、真实数据、我们的方法和时间.

3.4.7 消融研究

我们接着评估旋转等变性和未变形状态平衡在训练神经本构律中的作用. 为比较，我们设计方法的两个变体：no-undeformed 移除神经网络输入的归一化并重新加入偏置层，从而放松未变形状态平衡；no-rotation 直接将原始变形梯度输入神经网络，从而放松旋转等变性. 我们使用常规训练配置在橡皮泥上训练本方法及其变体. 如训练曲线图所示，我们的方法收敛更好、更快，而 no-rotation 的结果次优，no-undeformed 则由于引入不稳定的本构律导致仿真发散.



原文第 65 页的训练曲线：loss 为损失，#epoch 为训练轮数；ours 为我们的方法，两个变体的含义见正文.

除上述实验外，我们还在附录 B.6 中比较了本方法与 gnn 的运行时间性能，并给出训练数据集和泛化实验的完整可视化.

3.5 结论、局限性和未来工作

我们提出 NCLaw，一种从运动观测中学习可泛化 PDE 动力学的新型 NN–PDE 混合方法. 我们的方法在训练和部署阶段始终施加 PDE 约束，同时采用数据驱动的本构建模方式. 具体而言，我们将基于 NN 的本构律嵌入由 PDE 控制的可微仿真器，再通过基于仿真器输出与运动观测之差的损失函数训练 NN 模型. 借助基于物理精心设计的网络架构，NCLaw 保证常见的本构先验，例如旋转等变性和静止状态平衡. 在单条运动轨迹上训练后，NCLaw 实现了跨时间范围、初始 / 边界条件、复杂几何形状，乃至多物理场景的单次训练泛化. 真实世界实验证明了 NCLaw 从视频学习可泛化动力学的能力.

近期，我们计划将 NCLaw 扩展至多种物理现象，包括非均匀性、滞后和硬化 / 软化. 在本构设计中施加热力学第二定律 [50] 也是一个令人兴奋的方向. 在应用方面，我们计划将其推广到需要尽量缩小仿真到现实差距的任务，从机器人行走 / 操作到复杂物理中的一般控制 [191]. 我们还设想将 NCLaw 扩展到其他 PDE，例如湍流建模中的 Reynolds 应力张量 [105]，并计划将框架推广到其他离散化方案，例如 FEM. 此外，我们的 NN 方法可以受益于更好的可解释性以及训练后更方便的修改. 目前，我们假设

包括几何形状在内的初始条件已知；此外，还假设能够观测整个体积对象的运动（二维情况下为整个表面）. 未来工作可以考虑对初始条件的不确定性建模并处理部分观测. NCLaw 也可以扩展为从像素级视频数据中学习，而无需手动跟踪.

在更高层面上, NCLaw 同时受益于经典 PDE 和数据驱动方法. 通过施加经典 PDE 先验，NCLaw 在泛化任务上的准确度比不施加 PDE 约束的纯 NN 方法 [154] 高出数个数量级. 利用单一且富有表达能力的神经架构，NCLaw 能够捕捉从固体到流体的多种材料性质，而无需逐个由专家手工设计模型. 因此，我们相信 NCLaw 将为今后更多兼取两者之长的 NN–PDE 混合方案打开大门.

Chapter 4

将大语言模型与仿真作为双层优化器：推进物理科学发现的新范式

4.1 引言

在涵盖物理、化学、药理学等领域的物理科学中，多条研究路线旨在使科学发现自动化并加快其速度 [189]. 每条路线都在各自领域内创新，创造针对该领域特定挑战和细微差别的方法. 然而，这种做法往往缺少一种普遍适用的理念 [139, 41]；这样的理念对于普及先进研究工具、规范科学实践和提高跨学科效率可能至关重要. 我们的目标是超越特定领域，为物理科学提供统一的方法.

作为启发，我们观察人类科学家如何开展科学发现实验，并归纳出几条关键经验：迭代地提出假说，通过实验观测修正理论 [139]；将解分为离散部分，例如物理方程或分子结构，以及连续部分，例如物理参数和分子性质的参数 [189]；利用已有知识，同时不时积极探索新想法以追求突破 [195]；对于各类物理科学发现遵循通用的普遍原则，同时兼顾各学科的具体差别 [152].

大语言模型（LLM）作为拥有大量知识储备的通用工具 [2] 脱颖而出，近期因其广泛的知识库、先进的推理能力和对人友好的自然语言接口，在科学发现领域备受关注. 一条研究方向使用领域专属数据微调 LLM，使自然语言与科学信息对齐，例如化学结构 [27] 或药物结构 [110]；但这些方法受限于具体领域，若要更广泛地应用便需要大量数据. 另一条研究方向试图利用预训练 LLM 的固有能力，并通过互联网、编程或文档等外部资源加以增强. LLM 作为优化器或智能体 [69]，用于数学问题求解 [151]、开展化学实验 [15]，以及推进分子 [96] 和药物发现 [157]. 然而，这些方法受限于 LLM 的计算能力；而在物理科学中，根据物理定律假说计算数值结果、预测自然现象等任务对计算能力至关重要. 为解决这一限制，我们提出利用物理仿真增强 LLM，将 LLM 由知识驱动的抽象推理能力与仿真的计算结构和准确性结合起来.

为此，受人类科学家总体思维方式的启发，我们引入科学生成智能体（SGA），这是一种双层优化方法：外层使用 LLM 作为知识丰富、多才多艺的思考者，生成和修订科学假说；内层则使用仿真作为实验平台，提供观测反馈. 首先，我们利用 LLM 生成假说，再以这些假说指导仿真的执行；仿真产生的观测反馈反过来有助于修正和改进所提假说. 其次，我们引入双层优化框架：一层通过 LLM 对物理定律或分子结构等离散符号变量执行基于搜索的优化；另一层通过可微仿真，对材料刚度或分子坐标等连续参数执行基于梯度的优化. 第三，我们通过调整 LLM 的生成温度，为假说提出设计利用

与探索策略. 最后，我们展示该流程能够普遍应用于不同科学学科，只需对提示词等作最小修改.

在实证研究中，我们关注：(i) 分子设计，即依据分子的构象和量子力学性质发现分子结构及原子坐标；(ii) 本构律发现，即直接从记录的运动轨迹中发现材料本构方程及相应力学性质. 为给出具体例子，假设最初只有一个纯线性材料的代码，我们让模型通过优化该代码以拟合高度非线性的轨迹，发现更复杂的表征. 在这一任务中，我们的方法利用了双层优化的优势：外层使用 LLM 找出正确的符号材料本构方程，并提出可能有益的连续参数化方案（例如杨氏模量和泊松比）；内层通过可微仿真细化所提出的材料参数并提供有信息量的反馈. 总体而言，我们的方法能够发现所需的分子和本构律，优于其他基于 LLM 的基线；更有意思的是，它能够提出超出人类预期、但经领域专家分析后仍然合理的高性能方案. 总体贡献归纳如下：

- 我们提出一个结合 LLM 和物理仿真的通用物理科学发现框架.
- 我们提出双层优化：使用 LLM 在离散空间中进行基于搜索的优化，并使用可微仿真在连续空间中进行基于梯度的优化.
- 我们通过大量实验展示所提框架在物理定律发现和分子设计中的有效性与通用性；此外，还展示了从传统视角看出乎意料、但经领域专家检验被认为合理的新分子或本构律.

4.2 科学生成智能体

SGA 是一个双层优化框架：上层以 LLM 作为科学解的提出者，下层以仿真作为验证用的实验平台. 第 4.2.1 节形式化定义双层优化，第 4.2.2 和 4.2.3 节分别介绍外层和内层优化.

4.2.1 双层优化流程

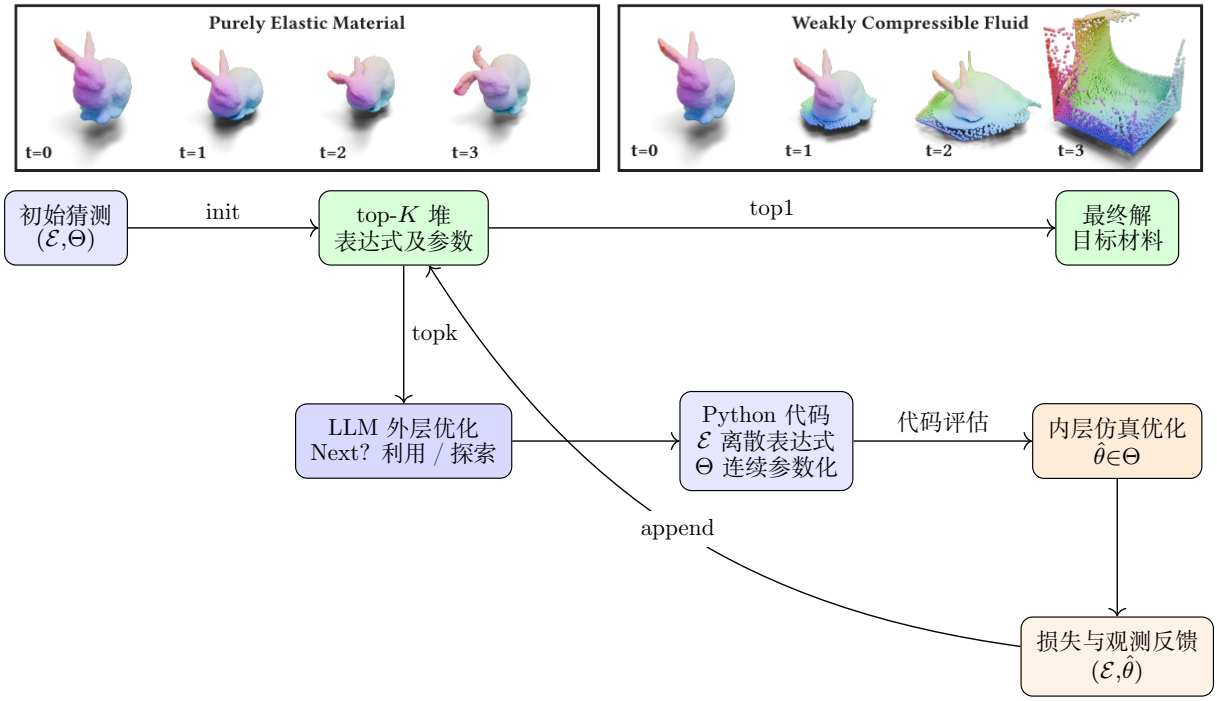
我们形式化描述方法流程，包括系统与底层子模块的输入 / 输出以及整体优化表述. 假设对于某个科学问题 $\mathcal{L}(y)$（例如重建某种力学行为），我们有一个评估物理现象 y（例如某种变形构型）的度量. 首先，将仿真（作为实验平台）描述为

$$y, z = \Phi(\theta; \mathcal{E}), \tag{4.1}$$

其中 Φ 是仿真器，以科学表达式 $\mathcal{E}$（例如本构方程）和连续部分 θ（例如材料参数）为输入，输出仿真的物理现象 y 和额外观测反馈 z（例如粒子轨迹）. 接着，通过提示词让 LLM 充当思考者，根据过去的仿真实验结果提出表达式 $\mathcal{E}$：

$$\mathcal{E}, \Theta = \mathrm{LLM}\big(\{\mathcal{L}(y_k), z_k, o_k, \mathcal{E}_k, \Theta_k\}_{k\in[K]}; \mathcal{P}\big), \tag{4.2}$$

其中集合 $[K]$ 概括了指向过去仿真结果的索引，结果包括科学问题的评价 $\mathcal{L}(y_k)$、其他物理反馈 (z_k, o_k) 和过去的提议 $(\mathcal{E}_k, \Theta_k)$；$o_k$ 概括内层优化的中间结果（详见第 4.2.3



堆中代码示例（Top-K Heap）

类头：

```
1  class Physics(nn.Module):
```

Θ 连续参数化：

```
2      def __init__(self):
3          super().__init__()
4          self.a = ...
```

$\mathcal{E}$ 离散表达式：

```
5      def forward(self, F):
6          F_new = self.a * F
7          return F_new
```

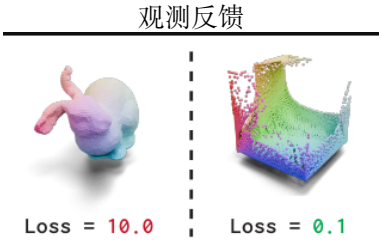
Python 代码修改示例

Θ 连续参数化：

```
4 -        self.a = ...
4 +        self.b = ...
```

$\mathcal{E}$ 离散表达式：

```
6 -        F_new = self.a * F
6 +        F_new = F / self.b
```



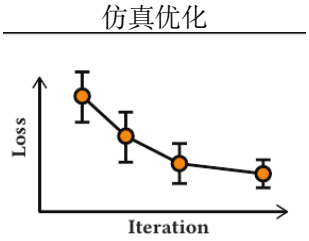


图 4-1: 科学生成智能体（SGA）的整体流程. 以本构律搜索为例，输入是一个初始猜测（纯弹性材料），输出是朝真实目标（弱可压缩流体）优化得到的另一个本构律. 初始猜测首先初始化一个存储解的 top-K 堆. 在外层优化中，LLM 以先前提出的 top-K 解为输入，在此基础上修改连续参数化 Θ 和离散表达式 $\mathcal{E}$，生成更好的解. 在内层优化中，基于梯度的优化通过仿真求解最优 Θ，并将优化后的解加入堆中. 经过若干轮双层优化后，堆返回排名第一的解作为最终解. 上方原图的 Purely Elastic Material 和 Weakly Compressible Fluid 分别为纯弹性材料和弱可压缩流体；下方流程按原图重排；Loss 和 Iteration 分别表示损失和迭代.

节）；Θ 确定内层优化决策变量的连续参数化（例如，在所提方程中优化哪些变量）；$\mathcal{P}$ 为提示词. 据此，我们将双层优化问题定义为

$$\min_{\mathcal{E},\Theta} \quad \mathcal{L}\big(y(\mathcal{E},\Theta,\hat{\theta};\Phi)\big) \tag{4.3a}$$

$$\text{s.t.} \quad G(\mathcal{E},\Theta;\Phi) \le 0 \tag{4.3b}$$

$$\hat{\theta} \in \arg\min_{\theta\in\Theta} \mathcal{L}\big(y(\theta;\Phi,\mathcal{E})\big), \tag{4.3c}$$

其中 $G(\cdot) \le 0$ 表示仿真的有效性（即表达式 $\mathcal{E}$ 能否用于仿真）. 外层优化搜索表达式 $\mathcal{E}$，它定义要进行的实验 $\Phi(\cdot;\mathcal{E})$；还搜索连续参数化 Θ，它定义内层连续优化 $\min_{\theta\in\Theta}$ 的搜索空间. 在依赖外层变量 $(\mathcal{E},\Theta)$ 的情况下，内层优化针对所提出的表达式，通过可微仿真搜索最优连续参数 $\hat{\theta}$.

算法 3：科学生成智能体

输入： 离散表达式与连续参数 $(\mathcal{E},\theta\in\Theta)$；利用次数 M_l，探索次数 M_h；利用温度 T_l，探索温度 T_h.

1. 使用堆存储排序后的（解，参数）：$\mathrm{H} \leftarrow \mathrm{heap}()$.
2. 连续优化：$\hat{\theta} \leftarrow \mathrm{optim}(\mathcal{E},\theta;\Phi)$.
3. $\mathrm{H.append}((\mathcal{E},\hat{\theta}))$.
4. 对 $i = 1,\ldots,N$：
   (a) 由 LLM 生成 M_l 个解：$(\mathcal{E},\Theta)[:M_l] \leftarrow \mathrm{LLM}(\mathrm{H.topk}(K),T_l)$.
   (b) 由 LLM 生成 M_h 个解：$(\mathcal{E},\Theta)[M_l:M_l+M_h] \leftarrow \mathrm{LLM}(\mathrm{H.topk}(K),T_h)$.
   (c) 对 $m = 1,\ldots,M_l+M_h$：
      i. 连续优化：$\hat{\theta} \leftarrow \mathrm{optim}(\mathcal{E},\theta\in\Theta;\Phi)$.
      ii. $\mathrm{H.append}((\mathcal{E},\hat{\theta}))$.

输出： $\mathrm{H.topk}(1)$，返回最佳解.

4.2.2 LLM 驱动的外层搜索

下面进一步介绍如何使用 LLM（式 (4.2)），以及它们如何与仿真交互以进行外层搜索（式 (4.3)）.

LLM 驱动的优化. 研究表明，LLM 是通用优化中有效的序贯决策者，通过提示和信息充分的上下文能够提供适当指导 [201, 151]. 我们设计提示词，以结构化方式指导 LLM，使其能够：分析过去的仿真实验结果，例如根据损失曲线判断应力张量的偏应力部分可能正确；制定高层计划，决定如何形成假说或改进以往实验，例如利用变形梯度行列式保证数值稳定性；提出可通过仿真作为实验执行、以检验假说的解，例如描述本构方程的代码片段. 针对式 (4.3a)，受文献 [117] 启发，我们采用进化搜索，在每次迭代中生成多个后代 $\{\mathcal{E}_m,\Theta_m\}_{m\in[M]}$（$M$ 为后代数），并保留最佳选择. 与之不同的是，我们的方法（算法 3）选择多个高性能候选，而不只选择最佳者；这既提高了假说在仿真中

的可行性（式 (4.3b)），也有利于进化交叉：LLM 从多次过去实验（“品系”）中产生新假说以更好地探索，这与文献 [151] 的发现类似.

与仿真的接口. 将 LLM 与仿真结合的主要挑战在于设计通信协议，使两个模块之间的通信既高效、结构化，又具有适应性. 我们观察到，物理科学中的解通常表示为数学表达式或结构化实体. 因而，在从 LLM 到仿真的方向，我们考虑两种设置：方程搜索和实体搜索，它们均统一抽象为式 (4.2) 中的 $(\mathcal{E}, \Theta)$. 在方程搜索中，LLM 可以提出方程 $\mathcal{E}$ 及内层连续优化的搜索空间 Θ；对实际使用者而言，以 PyTorch 为例，Θ 可对应通过 `nn.Parameter` 定义连续参数的 `__init__`，$\mathcal{E}$ 可对应定义方程计算过程的 `forward`（见图 4-1）. 在实体搜索中，LLM 提出结构描述 $\mathcal{E}$（例如原子如何连接形成分子），Θ 则简化为常量（例如每个原子均有待优化的三维坐标），并从优化式 (4.3a) 的决策变量中省略.

另一方面，在从仿真到 LLM 的方向，我们利用领域专家的知识设计函数，提取精简且相关的信息 z 作为观测反馈；这一过程类似经验丰富的科学家指导年轻同事如何有效记录实验发现. 例如，人类专家常常监测特定物体区域的运动，以推导本构律. 因此，为辅助这一过程，我们在仿真中加入记录粒子轨迹的函数. 最后，第 4.2.3 节将深入解释记作 o 的内层优化结果；这些结果用作从仿真反馈给 LLM 的信息.

利用与探索. 受人类科学家通过巧妙平衡谨慎推进与大胆探索取得突破的启发，我们通过调整 LLM 的解码温度 [201] 设计利用与探索策略. 在式 (4.3a) 中生成后代 $\{\mathcal{E}_m, \Theta_m\}_{m\in[M]}$ 时，我们将其分为两组：一组（$m \in \mathcal{M}_{\text{exploit}}$）是谨慎的跟随者，保持“梯度”并保守地沿用先前的解；另一组（$m \in \mathcal{M}_{\text{explore}}$）是大胆的探索者，承担风险并提出独特的解. 根据经验，我们观察到：$\mathcal{M}_{\text{exploit}}$ 常包含与之前迭代重复的解；$\mathcal{M}_{\text{explore}}$ 则倾向于产生过于随机、无法提供优化指导信息的解，或者无效的解（即违反式 (4.3b)），因此几乎不提供反馈信号. 作为经验法则，我们发现 $\mathcal{M}_{\text{exploit}}$ 与 $\mathcal{M}_{\text{explore}}$ 取 $1:3$ 的比例是有效的.

4.2.3 可微内层优化

在外层给定搜索空间 Θ 和仿真表达式 $\mathcal{E}$ 的情况下，内层优化（式 (4.3c)）通过可微仿真（式 (4.1)）进行基于梯度的优化，求解最优连续参数 $\hat{\theta} \in \Theta$. 本质上，领域专属知识通过仿真的梯度 $\nabla_\theta \Phi(\theta; \mathcal{E})$ 提炼到中间优化结果 o（如损失曲线）中. 随后将 $(\hat{y}, o)$ 反馈给 LLM，以修订解. 注意，o 可以包含相对于目标度量 $\mathcal{L}$ 的损失曲线，以及优化过程中其他辅助记录，这些信息说明如何从多个方面改进解；例如，当 $\mathcal{L}$ 表示位置位移时，o 可以包含内层优化各次迭代中的速度.

4.3 实验

4.3.1 问题定义

本构律发现. 从运动观测辨识本构律是物理学、材料科学和机械工程等领域中最困难的挑战之一. 我们遵循物理仿真的近期进展，将本构律发现任务表述为一个优化问题 [115]，并使用可微物质点法（MPM）仿真器 [164, 76]. 注意，我们的方法并非专为 MPM 仿真器设计，而适用于任何物理仿真. 任务目标是从粒子位置的真实轨迹 $\hat{X}_{t\in[1,\ldots,T]}$ 中，

同时辨识本构律的离散表达式和连续参数，具体为符号材料模型 $\varphi(\cdot)$ 及相应材料参数 θ，其中 T 表示步数. 我们考虑两类本构律 $\varphi_E(\cdot;\theta_E)$ 和 $\varphi_P(\cdot;\theta_P)$，分别对弹性和塑性材料建模，其形式定义为

$$\varphi_E(\mathbf{F};\theta_E) \mapsto \boldsymbol{\tau}, \tag{4.4a}$$

$$\varphi_P(\mathbf{F};\theta_P) \mapsto \mathbf{F}^{\text{corrected}}, \tag{4.4b}$$

其中 $\mathbf{F} \in \mathbb{R}^{3\times3}$ 是变形梯度，$\boldsymbol{\tau} \in \mathbb{R}^{3\times3}$ 是 Kirchhoff 应力张量，$\mathbf{F}^{\text{corrected}} \in \mathbb{R}^{3\times3}$ 是经塑性返回映射修正后的变形梯度，θ_E 和 θ_P 分别是弹性和塑性本构律的连续材料参数. 给定特定本构律，将其输入可微仿真，得到粒子位置轨迹：

$$X_{t\in[1,\dots,T]} = \text{sim}(\varphi(\cdot;\theta)), \tag{4.5}$$

然后通过将输出轨迹拟合真实轨迹 $\hat{X}_{t\in[1,\dots,T]}$ 来优化本构律.

分子设计. 本研究聚焦分子设计中一个普遍任务：发现具有特定量子力学性质的分子. 我们的目标是确定最优分子结构及其三维构象，使其匹配预先定义的目标量子力学性质. 设计过程既涉及离散表达式——由 SMILES 字符串 [194] 表示的分子结构，也涉及连续参数——分子中各原子的三维坐标. 该方法包含两个循环：外循环中，LLM 生成作为 SMILES 字符串的初始分子结构，同时给出三维原子坐标的初步猜测；内循环中，同时优化分子的三维构象和量子力学性质，二者均由三维原子位置决定.

为生成三维构象，我们使用 ETKGD 算法 [150]，再使用 Merck 分子力场（MMFF）[56] 优化；二者均在 RDKit [93] 中实现. 为获得量子力学性质值，我们使用基于 transformer 的预训练大模型 Uni-Mol [212]，它已在 QM9 数据集 [148] 上微调.

4.3.2 实验设置

任务设计. 我们设计了一组多样且具有挑战性的评估任务. 对本构律发现，提出四项任务：(a) 从线性弹性材料出发拟合非线性弹性材料；(b) 从纯弹性材料出发拟合 von Mises 塑性材料；(c) 从纯弹性材料出发拟合颗粒材料；(d) 从纯弹性材料出发拟合弱可压缩流体. 对分子设计，我们考虑四个常用任务，围绕三个常见的量子力学评估性质 [40, 212]，并分别设定目标值：(e) HOMO（最高占据分子轨道）设为 0；(f) LUMO（最低未占据分子轨道）设为 0；(g) HOMO–LUMO 能隙设为 0；(h) HOMO–LUMO 能隙设为 -2. 所有这些值均基于 QM9 数据集的全部数据归一化.

实现细节. 按照先前做法 [117]，我们以不同随机种子将全部实验运行五次. 由于任务复杂，我们提供一个有效设计的简单引导示例，以确保成功率. 可微 MPM 仿真使用 warp [118]，内层优化基于 PyTorch [134] 开发. 所有实验均使用均方误差作为评价准则，并使用 Adam 优化器 [84]. 我们选择 gpt-4-turbo-preview 作为 LLM 骨干模型，暂定利用温度 $T_l = 0.5$，探索温度 $T_h = 1.0$.

4.3.3 物理科学发现

我们考虑六个强基线进行评估：

1. 思维链（CoT）提示 [193] 通过查看示例中的分步解来解决问题. 我们为 CoT 提供五个带解释的例子作为初始解.

Method	#Iter.	#Hist.	#Exploit / #Explore	Bilevel	Constitutive Law Search				Molecule Design			
					(a) ↓	(b) ↓	(c) ↓	(d) ↓	(e) ↓	(f) ↓	(g) ↓	(h) ↓
CoT	1	5	N/A	✗	298.5	1462.3	150.0	384.1	3.0	32.1	18.6	6.0
FunSearch	20	2	0 / 4	✗	210.3	872.2	82.8	139.5	1.1	7.1	8.3	1.1
Eureka	5	1	0 / 16	✗	128.0	531.0	101.7	150.1	4.3	9.8	3.3	9.7e-1
OPRO	5	5	0 / 16	✗	136.2	508.3	99.2	128.8	2.4	9.4	3.1	1.3
Ours (no bilevel)	5	5	4 / 12	✗	90.2	517.0	83.6	68.4	8.6e-1	9.1	1.8	1.4
Ours (no exploit)	5	5	0 / 16	✓	3.0e-3	3.9e-1	6.6e-2	**1.4e-12**	4.0e-4	1.5e-1	6.1e-1	**2.8e-5**
Ours	5	5	4 / 12	✓	**5.2e-5**	**2.1e-1**	**6.0e-2**	**1.4e-12**	**1.3e-4**	**1.1e-1**	**5.4e-1**	3.6e-5

表 4.1: 基准测试. 我们将方法与四个基线及本方法的两个变体比较，并列出架构或超参数差异. #Iter. 为迭代次数，#Hist. 为历史优化步骤中 top-k 检索的 K 值，#Exploit/#Explore 为利用与探索的后代数，Bilevel 表示是否启用双层优化. 八个任务分为本构律搜索 (a)–(d) 和分子设计 (e)–(h). 所有任务均以较低损失为佳，最低损失的最佳方法用粗体标出. 图内 Method、Constitutive Law Search 和 Molecule Design 分别为方法、本构律搜索和分子设计；Ours (no bilevel)、Ours (no exploit) 为无双层优化和无利用的变体.

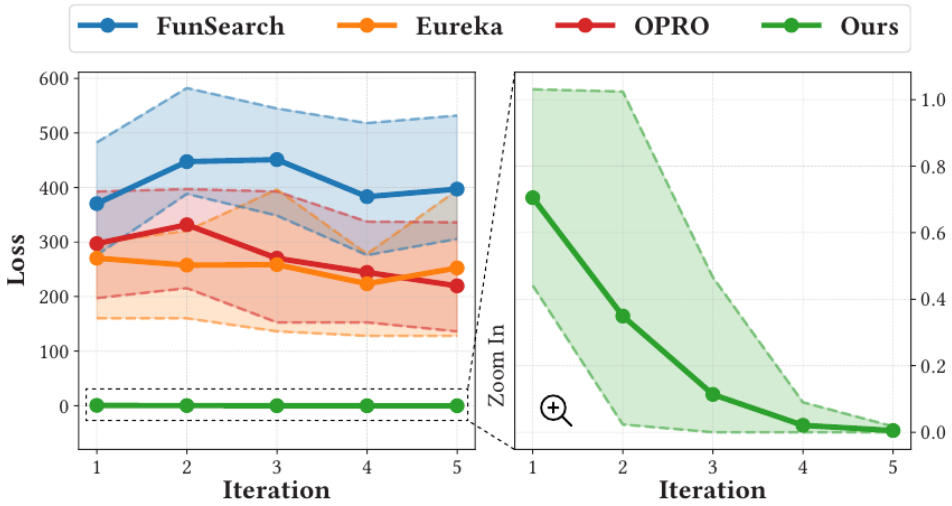


图 4-2: 损失趋势比较. 图示 LLM 驱动优化的不同迭代中，最佳解的损失在不同随机种子上的平均值；阴影表示最小 / 最大值. 图内 Loss、Iteration、Zoom In、Ours 分别表示损失、迭代、放大和我们的方法.

2. FunSearch [151] 使用进化策略避免局部最优. 我们采用原始实现给定的超参数，使用两条优化历史和四个探索者. 为与其他方法公平比较，我们将迭代次数设为 20，使被评估的解的数量相同.

3. Eureka [117] 在每次迭代中生成多个解，以提高所生成代码的成功率. 我们保留原始实现的超参数.

4. Optimization by PROmpting（OPRO）[201] 强调使用排序后的优化轨迹的优势. 除历史优化步数外，我们将其超参数设为与 Eureka 相同.

在上述工作中，我们注意到 LLM 推理温度均为 1.0，与本方法的探索温度相同，因此将其标记为零个利用者. 我们还考虑本方法的两个变体：(v) Ours (no bilevel) 移除双层优化，只使用 LLM 搜索；(vi) Ours (no exploit) 通过始终将温度设为 1.0 来移除利用.

我们针对设计的八项任务进行实验，结果见表 4.1. 与基线 (i)–(iv) 相比，我们的方法显著优于它们，优势达到若干数量级. 从本方法中移除双层优化时，性能急剧下降，但在统计上仍优于基线 (i)–(iv)，表明超参数选择以及引入利用对任务有所帮助. 移除利用但恢复双层优化时，性能重新提高：在任务 (d) 上与本方法相当，在 (h) 上甚至更好. 然而，在某些任务中，尤其是在现实中更受关注的困难任务（例如 (b) 和 (f)）上，性能差距超过 50%，表明利用与探索策略有效. 图 4-2 展示了任务 (a) 的损失趋势；我们的方法因损失低得多且呈收敛趋势而突出.

我们还将方法与各特定领域的传统方法比较，以展示其通用性. 首先，我们将本构律搜索任务 (a) 重构为符号回归任务：记录真实输出（应力张量）作为监督，并将九个输出维度分为九个独立问题，再将其组合进行评估. 注意，这些修改大幅简化了原任务：我们移除了时间反向传播（BPTT），直接发现本构律而无需代理损失. 我们评估 SRBench [91] 中的 14 个传统基线和三个数据驱动的预训练基线. 表 4.2 列出其中表现最佳的若干基线，其余结果见附录 C.3.1. 如表所示，即使处于更具挑战性的设置下，我们的方法在这一任务中仍排名第一. 此外，由于方法依赖 LLM 的上下文学习能力，它受到的变量数量限制比数据驱动的预训练基线更少. 对分子设计任务，我们还将方法与采用基于种群的分子设计算法的 GhemGE [205] 比较. 表 4.3 显示，我们的方法损失低得多，展示了方法的总体有效性.

4.3.4 消融研究

泛化还是记忆. 为弄清方法带来的改进是否仅仅因为 LLM 在训练阶段见过这些解，我们设计了一项消融实验，让它发明一种地球上不存在的虚构本构律. 我们按 50%、30% 和 20% 的比例混合 von Mises 塑性、颗粒材料和弱可压缩流体的本构律，使新本构律表示一种行为极其复杂的虚构材料. 我们重复图 4-1 中的实验设置[3]，将方法与基线比较，性能见表 4.4. 如表所示，我们的方法仍能以较低的定量损失发现本构律. 据我们观察，真实材料与优化后的本构律之间的视觉差异很小. 发现的本构律见附录 C.4.5.

双层优化是关键. 我们在图 4-3 中使用任务 (h) 评估双层优化的重要性. 比较蓝色三角形曲线和红色圆点曲线，它们分别表示 LLM 驱动的外层优化和仿真驱动的内层优

[3]译注：原文此处写作 Figure 1；译文依据上下文推定其指本章图 4-1，这一对应并非原文明示.

方法	$R^2 \uparrow$	MSE ↓	MAE ↓	符号解
FFX	0.9824	4.5e5	3.7e2	是
MLP	0.9876	3.2e5	3.4e2	否
FEAT	0.9964	9.2e4	1.7e2	是
DSO	0.9968	8.2e4	9.2e1	是
Operon	0.9988	2.8e4	9.8e1	是
SymbolicGPT	0.5233	6.9e6	1.7e3	是
NeSymReS	超过 3 个变量时不适用			是
T-JSL	超过 2 个变量时不适用			是
Ours	**0.9990**	**1.7e4**	**8.6e1**	是

表 4.2: 与符号回归比较. 我们将方法与 SRBench 中表现最好的五种方法以及三种预训练符号回归方法比较. 最后一列表示结果是否为符号表达式.

方法	(e) ↓	(f) ↓	(g) ↓	(h) ↓
GhemGE	4.8e−3	1.8	1.5	9.8e−5
Ours	**1.3e−4**	**1.1e−1**	**5.4e−1**	**3.6e−5**

表 4.3: 与基于种群的分子设计方法比较. 我们将方法与传统的基于种群的分子设计方法 GhemGE 比较，报告分子设计任务 (e)–(h) 的结果.

方法	FunSearch	Eureka	OPRO	Ours
损失	105.0	89.1	98.0	**1.3e−3**

表 4.4: 虚构本构律实验. 我们构造一种虚构本构律，以防止 LLM 通过记忆取巧，并报告本方法和基线的结果.

化，可以容易地得出采用双层优化时损失表现更好的结论. 然而，我们还关心双层优化在每一步优化内部如何工作，以及 LLM 与仿真分别提供多少帮助. 如锯齿形曲线所示，我们发现 LLM 与仿真在所有优化步骤中相互帮助：仿真优化的结果会使 LLM 下一次提出的解更好，反之亦然. 我们认为二者具有不同专长：LLM 是拥有跨学科知识的通才科学家，而仿真是拥有专门知识的领域专家.

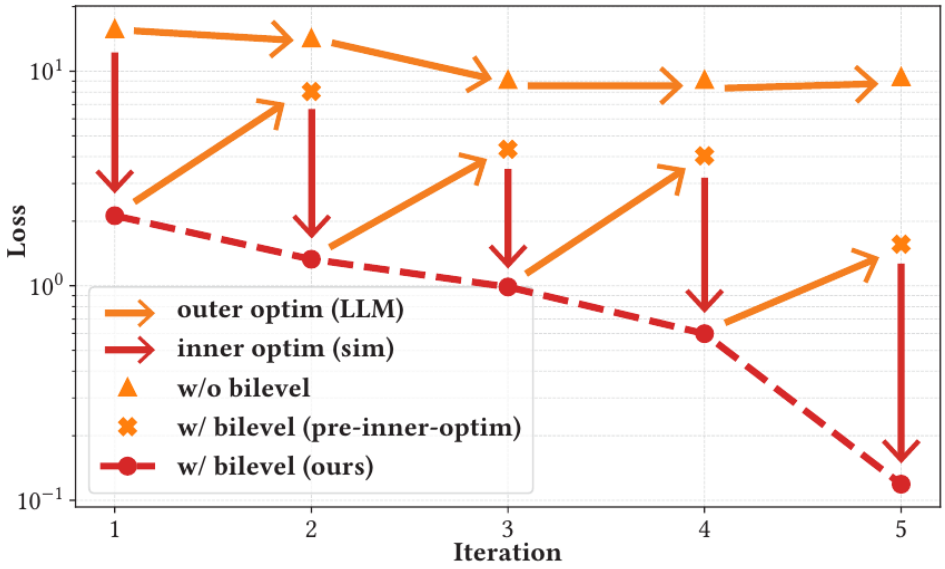


图 4-3: 双层优化的消融实验. 有和无双层优化的优化轨迹分别以红色圆点和橙色三角形表示；本方法在内层优化之前的中间步骤以橙色叉号表示. 外层 LLM 优化与内层仿真优化分别以橙色和红色箭头突出显示. 图内 Loss、Iteration 分别为损失和迭代；w/o bilevel、w/ bilevel (pre-inner-optim)、w/ bilevel (ours) 分别表示无双层优化、双层优化的内层优化前步骤以及采用双层优化的本方法.

译注：上一段正文称蓝色三角曲线和红色圆点曲线分别表示外层与内层优化；图例及图注则以橙色三角表示无双层优化、红色圆点表示有双层优化，并以箭头区分外层和内层. 两处说法按原文保留.

LLM 骨干模型. 除 GPT-4 [130] 外，我们还使用三个额外的 LLM 骨干模型重复表 4.1 中的实验：(i) GPT-3.5 [133]，(ii) Claude-3-Sonnet [5]，(iii) Mixtral-8x7B [74]，并在图 4-4 中报告排名. 面积最大表明，我们选择的 GPT-4 在统计上优于其他方法. 有趣的是，我们发现 Claude-3-Sonnet 在大多数本构律搜索任务中排名第二，而 Mixtral-8x7B 甚至在两项分子设计任务中排名第一. 因此，我们的流程对其他 LLM 也有效；不过，我们建议实际使用者首先尝试 GPT-4，同时也可因预算或可定制性考虑开源模型（例如 Mixtral-8x7B）.

利用与探索. 我们使用实验中最具挑战性的任务之一 (b)，在图 4-5(a) 中将仿真执行状态的统计结果可视化. 移除利用时，错误率显著上升，表现为绿色柱减少；这导致具有利用的那些方法的性能退化，如图 4-5(b) 所示. 然而，即便成功率仍然很高，移除探索时的优化结果依然比同时保留二者更差. 我们认为，当优化问题具有挑战性时，探索十分重要；在本例中尤其如此，因为搜索空间高度非线性且无结构，因而产生大量局

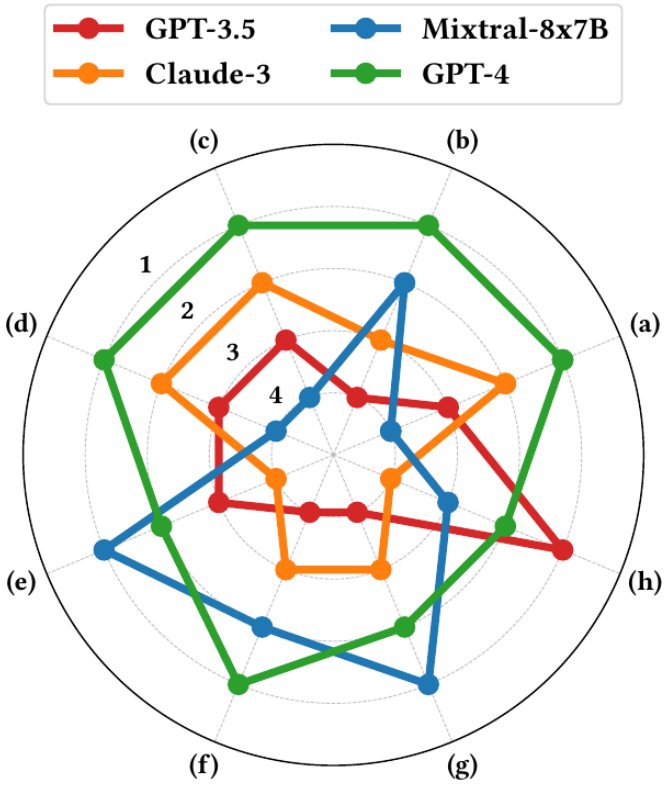


图 4-4: LLM 骨干模型消融实验. 我们比较四个选定骨干 LLM 的表现并报告其排名. 越靠外的曲线表示性能越好；(a)–(h) 对应实验中的八项任务.

部最优.

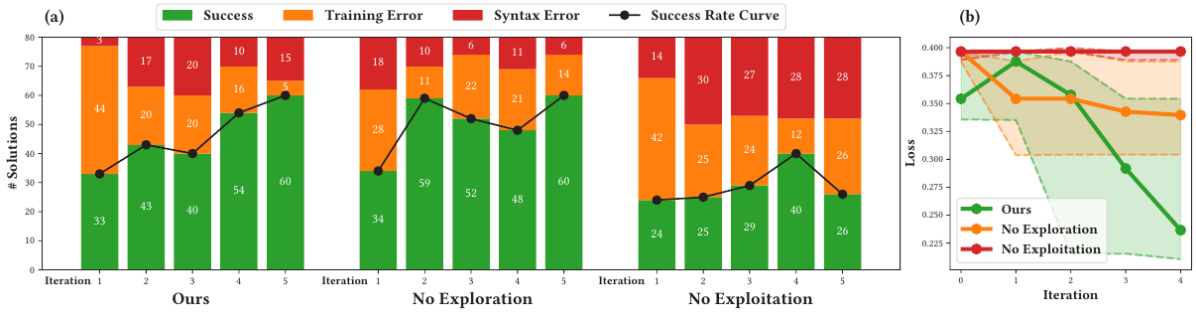


图 4-5: 探索与利用的消融实验. (a) 不同迭代中可用于仿真的有效解的直方图. (b) 不同迭代中最佳解的损失在不同随机种子上的平均值，阴影表示最小 / 最大值. 图内 Success、Training Error、Syntax Error、Success Rate Curve 分别为成功、训练错误、语法错误和成功率曲线；Ours、No Exploration、No Exploitation 分别为本方法、无探索和无利用；Loss、Iteration、#Solutions 分别为损失、迭代和解的数量.

4.3.5 案例研究

译注：上一段在说移除利用之后，又称具有利用的方法性能退化，原文前后表述存在矛盾；译文未擅自改写.

本构律搜索. 对任务 (a)，即给定高度非线性材料的轨迹进行拟合，我们在图 4-6(a) 中给出搜索所得本构律的精简代码片段. 我们对代码略作重排以适应正文，完整示例见附录. 从线性材料出发，我们的方法能够自动生成含二次偏应力项的本构律. 注意，本方法还给出了 `__init__` 函数的具体实现，该函数在计算图中定义连续参数，以供后续内层优化使用.

分子设计. 根据图 4-6(b) 中 LLM 的优化结果，比较两个分子的 HOMO–LUMO 能隙时，我们观察到它们各自不同的特征：分子 A（gap-0）含有连接在环上的硫原子和氯原子，以及一个三氟甲基，从而引入吸电子效应；分子 B（gap-2）的环结构中含有氧（尤其是在醚中）和硫，引入局域化的非键电子对. 此外，分子 B 的整体结构比分子 A 更复杂，包含多个环. 分子 B 有一个可能起初不符合预期的有趣之处：它含有一个氟原子. 氟的高电负性通常导致电子密度被吸引，影响能隙值. 然而，由于分子 B 结构复杂，氟原子的影响在一定程度上是局域的，因此不会显著改变能隙值.

4.4 相关工作

4.4.1 自动化科学发现

机器学习方法增强的自动化科学发现是强有力的研究加速器，使科学家能够生成假说、设计实验、解释海量数据集，并发现传统科学方法可能忽略的见解 [2, 89, 189]. 这一多方面的过程通过两个协同关联的阶段展开：假说形成，以及实验数据的收集与分析. 自动化系统的整合不仅增强科学探究过程，还简化从概念形成到实证验证的发现流程.

(a)

```python
class Physics(nn.Module):
  def __init__(self, youngs_modulus_log: float = 13.03,
                     poissons_ratio_sigmoid: float = -1.99):
    super().__init__()
    self.youngs_modulus_log = nn.Parameter(
      torch.tensor(youngs_modulus_log))  # Log of Young's modulus
    self.poissons_ratio_sigmoid = nn.Parameter(
      torch.tensor(poissons_ratio_sigmoid))  # Sigmoid of Poisson's ratio
  def forward(self, F: torch.Tensor) -> torch.Tensor:
    youngs_modulus = self.youngs_modulus_log.exp()
    poissons_ratio = torch.sigmoid(self.poissons_ratio_sigmoid) * 0.49
    mu = youngs_modulus / (2 * (1 + poissons_ratio))  # Shear modulus
    lam = youngs_modulus * poissons_ratio / (
      (1 + poissons_ratio) * (1 - 2 * poissons_ratio))
    # Deformation gradient determinant J
    J = F.det().view(-1, 1, 1)
    F_invT = F.inverse().transpose(1, 2)
    # Volumetric part
    P_vol = lam * (J - 1) * F_invT
    # Deviatoric part
    P_dev = mu * (F - (1 / J) * F_invT)
    # Compute Kirchhoff stress tensor
    kirchhoff_stress = P_vol + P_dev @ F.transpose(1, 2)
    return kirchhoff_stress
```

(b)

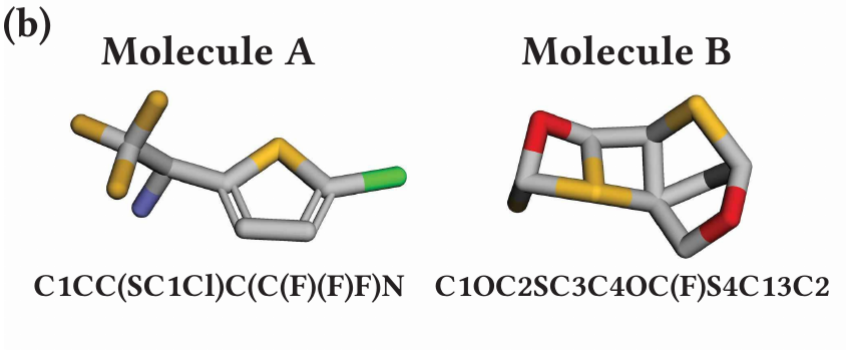


图 4-6: 案例研究. (a) 搜索所得本构律的具体示例；(b) 两个针对不同目标优化的新分子及其 SMILES 字符串. 图中 Molecule A、Molecule B 分别为分子 A、分子 B；代码原样保留，注释依次表示杨氏模量的对数、泊松比的 sigmoid、剪切模量、变形梯度行列式 J、体积部分、偏量部分及计算 Kirchhoff 应力张量.

本文特别关注但不限于本构律发现和分子设计；这些领域体现了自动化在揭示复杂材料行为，以及创新设计具有定制性质的分子方面的深远影响. 本构材料模型的自动辨识是长期存在的问题，近期工作利用可微仿真 [37, 115, 113] 将其作为系统辨识问题处理. 借助机器学习和人工智能，研究人员能够预测分子行为、针对特定功能优化化学结构，从而迅速加快新药、材料和化学品的开发 [77, 213, 156].

4.4.2 大语言模型与智能体

ChatGPT 和 GPT-4 等大语言模型（LLM）的进步，引发了人们对其作为自主智能体的潜力的广泛兴趣 [18, 129, 130]. 近期进展表明，可以通过让 LLM 创建并使用自己的工具来增强其解决复杂问题的能力，如 LATM 框架所示 [165]；也可以让它们在没有梯度的情况下充当优化器，如 OPRO 方法所示 [201]. 这些方法标志着向更独立、更多功能的 LLM 智能体转变：它们能够通过自行构造的工具和优化技术生成解决方案 [20, 203, 202]，展示出不断发展的解决问题能力. 在科学发现领域，LLM 已开始作出重要贡献，尤其是在数学和计算问题中. FunSearch 方法 [151] 将 LLM 与评估器结合，在极值组合学和在线装箱问题上超越已知结果，说明 LLM 能够为已有问题发现新解. 类似地，AlphaGeometry [173] 在没有人类演示的情况下成功解决奥林匹克几何问题，突显了 LLM 实现复杂推理任务自动化的潜力. 这些例子强调了 LLM 推进科学探究和自动推理边界的变革性影响.

4.4.3 双层优化

双层优化具有由两层优化问题组成的层次结构，其中上层问题的解取决于下层问题的结果 [28]. 由于优化任务的嵌套性质及其复杂的相互依赖关系，双层优化问题本质上比单层问题更加复杂. 近期进展聚焦于开发高效算法，包括进化算法 [160]、基于梯度的方法 [109] 和近似技术 [159]，以应对许多双层问题的非凸及不可微特征带来的计算挑战. 在双层优化广泛的应用领域中，神经架构搜索（NAS）[107, 12, 19, 200] 尤为突出，也与本文的问题设置接近：上层优化离散的神经网络架构，下层优化神经网络的连续权重. 然而，典型 NAS 方法需要预先定义搜索空间，将离散网络架构的探索限制在人工指定的边界内. 我们的框架使用编码了通用知识的 LLM，摆脱人工设计约束所施加的限制，因而有所不同.

4.5 结论

我们考虑若干局限和未来方向. 首先，尽管我们提示 LLM 生成伪代码计划和注释，通常仍难以保证 LLM 所生成解的可解释性. 第二，由于在本应用中 LLM 生成的代码未经任何过滤便直接执行，存在危及操作系统的潜在 AI 安全风险. 第三，我们的方法仅利用 LLM 的内部知识作为先验，而现实中人们会设计人工约束和规则来正则化并改进优化 [176]；我们将这些领域专属应用和基于人类反馈的正则化方法留作未来工作. 第四，方法性能高度依赖生成代码的可微性；但由于连续参数数量相对有限，零阶优化器 [59] 也应当能够发挥作用. 第五，LLM 推理需要大量计算资源，因而增加费用. 例如，使用 GPT-4 时，我们的方法完成一项任务约需 10 美元；随着迭代次数增长，这将越来

越难以接受. 第六，由于所提 top-k 堆重用先前生成的解，LLM 中相邻迭代的 KV 缓存会高度相似，这为近期 KV 缓存优化方法 [210] 通过重用 KV 缓存加速本方法打开了可能.

总之，我们提出科学生成智能体这一双层优化框架：LLM 充当知识丰富、适应性强的思考者，构造物理方程或分子结构等科学解；与此同时，仿真充当实验平台，提供观测反馈并优化物理参数等连续部分. 我们聚焦于本构律搜索和分子设计这两个科学问题. 我们的方法优于其他基于 LLM 的基准方法，带来一致、稳健且近乎单调的改进. 此外，它在辨识未知的真实本构律和分子结构方面表现出突出的能力. 值得注意的是，我们的系统生成了创新性解：尽管不符合常规，但经各领域专家充分分析后被认为合理. 我们认为这一过程具有开拓性，为进一步推动物理科学发现建立了新范式.

Chapter 5

结论

本论文旨在展示基于物理的世界模型及使用神经物理构建此类模型的巨大潜力. 通过引入渲染不变状态预测器 (RISP)、神经本构律 (NCLaw) 和科学生成智能体 (SGA), 论文同时说明了三个关键点: 基于物理的世界模型具有有价值的功能; 使用可微神经物理模块构建这些模型是高效的; 它们在下游科学任务中具有巨大的应用潜力.

RISP 引入作用于渲染输出的神经模块, 显著增强可微物理的实用性, 从而大幅缩小底层世界模型的仿真到现实差距. 它回应了关于如何最好地利用可微物理这一长期问题, 并提出了训练神经模块的优雅方案. RISP 以独特方式展示了神经物理出人意料的潜力: 它超越单纯的物理建模, 使世界模型的效率和真实性达到新的水平.

在 RISP 展示神经物理对世界模型感知建模的效率和有效性后, NCLaw 更进一步, 使用精心设计的神经网络参数化本构律——区分不同材料的核心机制. NCLaw 为从弹性体到弱可压缩流体的多种复杂物理现象提供统一表征. 为保证物理性质并稳定训练, 旋转等变性和静态平衡等若干基本不变性被嵌入神经架构. NCLaw 为更准确的仿真到现实校准, 以及更高效的基于物理的数据驱动仿真铺平了道路.

SGA 直接建立在 NCLaw 的基础设施之上, 但使用大语言模型生成的 token 替代人工设计的神经模块. 它不再直接依据神经参数优化世界模型以匹配物理轨迹, 而是将可微仿真和语言模型用作双层优化器, 同时更新物理定律的离散表达式和连续参数化. 除解决 NCLaw 中的本构律搜索问题外, SGA 还在分子设计挑战中探索跨学科能力. 实验突出展示了 SGA 在科学发现中的优异表现, 暗示了通向完全自动化科学与工程进展的一条潜在路径.

5.1 主要认识

通过使用上述三种方法以神经物理构建世界模型, 我们归纳出本论文获得的经验:

- 在物理世界模型流程中合理嵌入神经模块, 可以提高其效率、准确性和泛化能力. 这种有策略的整合发挥神经网络的优势, 并补充传统建模技术, 尤其适用于传统上较多依赖经验设计的组成部分.

- 在神经物理中, 设计恰当的不变性或等变性对提高性能和稳定训练至关重要. 此外, 鉴于偏微分方程在神经物理中广泛存在, 使用高效的高阶算子有助于对这些不变性建模.

- 经广泛验证可靠、较不易出错的物理定律, 应以其原有形式保留, 而不是通过参数化替换. 例如, 保持不同模块之间的固有连接, 可以避免世界模型引入不必要的先验, 从而提高其稳健性和效率.

- 物理世界模型的发展不仅有利于物理建模本身，也有利于其他领域. 智能体与这些世界模型之间的交互有助于更高效的科学发现，并促进智能的演化，进而推动多个领域产生更深入的认识和进展.

5.2 局限性和未来方向

虽然本论文展示了以神经物理构建世界模型的方法和潜力，但有必要指出当前基于神经物理的世界模型的局限. 这些局限自然引出了未来研究和发展的方向.

- **模块化有限.** 模块化是现代系统的重要方面. 虽然论文中的工作遵循高层模块化原则，但构建世界模型时完全用神经模块替换特定模块，而该神经模块本身并未明确模块化. 此外，这些工作主要针对感知模块和本构律的替换，忽略了物理系统的其他重要组成，包括但不限于接触模型、边界条件、空间和时间积分器以及几何表征. 未来研究应考虑探索这些额外的模块化方案.

- **与多模态表征的接口不完整.** 世界模型的关键特征之一是具有多模态扩展能力. 全面有效的世界模型将其潜在动力学投射到完整范围的表征中，包括视觉和音频形式. RISP 使用可微物理渲染获取视觉表征；但这种方法既低效又受到范围限制. 探索将动力学模型与 NeRF [122]、Gaussian splatting [83]、视频生成模型 [17] 或其他多模态模块结合，具有很大潜力. 这种结合可能提高世界模型多模态表征能力的效率和广度.

- **数据获取效率低.** 数据规模和质量是影响可扩展性的关键因素. 神经物理通过施加物理上正确的连接和不变性，有助于减少训练世界模型所需的数据量；然而，这种方法也将质量要求提高到状态层级. RISP 通过引入可微渲染器，将监督从状态层级扩展至像素层级. 尽管如此，优化 RISP 仍依赖状态层级的标注. 未来研究可以探索使用对比学习 [25] 进行自监督数据获取，或利用 DINOv2 [132, 29]、SAM [85] 等基础视觉模型. 这些方法可能在保持或提高数据质量的同时改善可扩展性.

- **引导性和可控性探索不足.** 在预训练和后训练阶段，引导和控制世界模型的演化都具有显著挑战. 与传统物理仿真一致，以神经物理构建的世界模型通常通过边界条件和碰撞纳入交互. 虽然这些方法提供精确、全面的运动描述，但往往不够方便用户使用，也不够有创造性. 条件图像生成的近期进展通过文本提示 [63] 和视觉提示 [208] 等技术，在这一方面带来了新可能. 将这些模块融入世界模型可能是一个令人兴奋的发展方向，为建模过程提供更高的灵活性和用户交互能力.

Appendix A

RISP 附录：利用可微仿真与渲染进行跨域参数估计的渲染不变状态预测器

A.1 定理的证明

为简洁起见，我们去掉 $\mathcal{L}^{\text{reg}}$ 中的求和并略去下标 j，专注于推导 Frobenius 项关于网络参数 θ 的梯度. 令 $\mathbf{G}$ 为 Frobenius 范数中的雅可比矩阵，i 和 j 为其行、列指标.

现在如下推导 $\mathcal{L}^{\text{reg}}$ 关于 θ 中第 k 个参数的梯度：

$$\frac{\partial \mathcal{L}^{\text{reg}}}{\partial \theta_k} = \sum_{ij} 2\mathbf{G}_{ij} \frac{\partial \mathbf{G}_{ij}}{\partial \theta_k} \tag{A.1}$$

$$= \sum_{ij} 2\mathbf{G}_{ij} \frac{\partial}{\partial \theta_k} \left[\frac{\partial \mathcal{N}_\theta(\mathbf{I})_i}{\partial \mathbf{I}} : \frac{\partial \mathbf{I}}{\partial \psi_j} \right], \tag{A.2}$$

$$= \sum_{ij} 2\mathbf{G}_{ij} \left[\frac{\partial^2 \mathcal{N}_\theta(\mathbf{I})_i}{\partial \mathbf{I} \partial \theta_k} : \frac{\partial \mathbf{I}}{\partial \psi_j} \right] \tag{A.3}$$

$$= \sum_{i} \frac{\partial^2 \mathcal{N}_\theta(\mathbf{I})_i}{\partial \mathbf{I} \partial \theta_k} : \left(\sum_j 2\mathbf{G}_{ij} \frac{\partial \mathbf{I}}{\partial \psi_j} \right) \tag{A.4}$$

$$= \sum_{i} \frac{\partial^2 \mathcal{N}_\theta(\mathbf{I})_i}{\partial \mathbf{I} \partial \theta_k} : 2 \sum_j \left(\frac{\partial \mathcal{N}_\theta(\mathbf{I})_i}{\partial \mathbf{I}} : \frac{\partial \mathbf{I}}{\partial \psi_j} \right) \frac{\partial \mathbf{I}}{\partial \psi_j}. \tag{A.5}$$

该推导表明，假设所有 $\partial \mathbf{I}/\partial \psi_j$ 均可用，我们可以遍历状态维度（由 i 索引）并运行反向传播，得到 $\partial \mathcal{L}^{\text{reg}}/\partial \theta$. 事实上，$\partial \mathbf{I}/\partial \psi_j$ 只依赖训练数据，在整个优化过程中保持不变，因此可以在训练集中预先计算，占用额外空间 $\mathcal{O}(\sum |\psi||\mathbf{I}|)$. 通常 $|\psi_j|$ 是一个较小的数（在大多数实验中小于 10），因此将其与数据集中生成的图像一同存储是可以承受的. 这也是我们偏好提供前向模式微分的可微渲染器的原因.

假设所有 $\partial \mathbf{I}/\partial \psi_j$ 均已备好，我们可以对每个状态维度 i 使用两次反向传播来实现式 (A.5). 首先，通过 $\mathcal{N}$ 反向传播，得到 $\partial \mathcal{N}_\theta(\mathbf{I})_i/\partial \mathbf{I}$. 接着，使用 $\partial \mathbf{I}/\partial \psi_j$ 组装伴随向量（式 (A.5) 中的第二个求和）. 最后，使用该伴随向量，再次通过二阶偏导数 $\partial^2 \mathcal{N}_\theta(\mathbf{I})_i/(\partial \mathbf{I} \partial \theta)$ 反向传播. 注意，第二次反向传播无需遍历 k. 因此总时间开销为 $\mathcal{O}(|\mathbf{s}||\theta|)$.

译注（原文疑点） 正文式 (2.10) 给出的是未平方的 Frobenius 范数，而式 (A.1) 的 $2G_{ij}$ 因子对应平方范数的求导形式. 两者之间的关系尚待核对作者材料；此处保留原文，不据此改动公式.

A.2 实现细节

A.2.1 环境

四旋翼飞行器.

状态空间. 我们将四旋翼飞行器的物理系统定义为无接触刚体系统. 当其中心在任意时刻到原点 $(0,0,0)$ 的欧氏距离超过 1000 时，我们明确将任务标记为“失败”. 飞行器状态定义为世界坐标位置 (x,y,z) 和旋转角（yaw、pitch、roll，即偏航、俯仰、横滚）. 为减轻旋转角不连续性的负面影响，我们将网络输出设计为

$$(x,y,z,\sin(\text{yaw}),\sin(\text{pitch}),\sin(\text{roll}),\cos(\text{yaw}),\cos(\text{pitch}),\cos(\text{roll})), \tag{A.6}$$

随后使用 atan2 还原旋转角.

动作空间. 控制信号为四维，表示施加在四个螺旋桨上的力矩. 控制信号的幅值被限制在 $[-500,500]$ 内.

参数空间. 在系统辨识实验中，我们在 $[2.8,3.2)$ 内均匀采样飞行器质量，其真实值为 3.

立方体.

状态空间. 我们将立方体的物理系统定义为有接触的刚体系统. 与四旋翼环境类似，网络输出按照式 (A.6) 设计为九维向量.

参数空间. 在立方体环境中，我们采用由弹簧刚度 k_n 参数化的软接触模型. 为提高数值稳定性和性能，系统辨识实验训练 $\lambda_k = \log_{10} k_n$. 我们在 $[5.5,6.5)$ 内均匀采样 λ_k，其真实值为 6.

手.

状态空间. 我们将手的物理系统定义为由文献 [199] 支持的关节体系统. 手掌固定在世界坐标系中，各关节建模为转动关节. 该环境有 13 个关节，每个关节均被限制在 $[-\pi/4,\pi/4]$ 内. 手的状态空间定义为表示各关节角度 $\{\theta_{1\ldots13}\}$ 的 13 维向量.

动作空间. 通过对各关节施加力矩控制手. 在时刻 t，我们将每根手指的控制信号参数化为

$$u_t = m_j \sin\left(2\pi \min\left(1.0, \max\left(0.0, \frac{1}{T}(t-\delta_j)\right)\right)\right),$$

其中 $0 \le m_j \le 1$ 和 $0 \le \delta_j \le 40$ 表示第 j 根手指的动作幅值和动作偏移，$T = 40$ 为周期.

参数空间. 对每根手指，我们让该手指所有关节的关节刚度可供训练，使参数空间成为五维空间. 我们在 $[0.0,1.0)$ 内均匀采样关节刚度，其真实值均为 1.0.

杆.

状态空间. 我们将杆的物理系统定义为由文献 [37] 支持的软体动力学系统. 固定杆的两端并对其施加重力. 沿中心轴线均匀选择 10 个传感器，跟踪它们在竖直方向的位置，由此定义十维状态空间.

动作空间. 我们在杆中心竖直施加外力作为动作信号，动作被限制在 $[-500,500)$ 内.

参数空间. 我们用杨氏模量 E 参数化杆环境. 为提高数值稳定性和性能，训练 $\lambda_E = \log_{10} E$. 我们在 $[4,7)$ 内均匀采样 λ_E，其真实值为 5.

A.2.2 RISP 网络

网络设置. 我们在修改版 ResNet-18 [60] 的基础上构建网络. 所有层后均接不带仿射参数的实例归一化 [177]，遵循 James 等 [72] 的做法. 由于 ReLU 使二阶导数退化为常数值，我们将其替换为 Swish 非线性函数 [147]. 我们将 ResNet-18 骨干的最后一层改为线性投影层，其输出数与状态维数相同.

训练. 我们使用学习率为 10^{-2}、权重衰减为 10^{-6} 的 Adam 优化器 [84]. 学习率采用余弦衰减调度 [111]. 网络训练 100 个 epoch，默认批量大小为 16. 为避免训练初期模型坍缩，我们将渲染梯度的幅值系数从 0 线性增加至依环境而定的值：四旋翼为 1，立方体为 20，手为 100，杆为 30.

A.2.3 训练细节

模仿学习和视觉运动控制实验使用相同的实验设置. 我们使用学习率为 3×10^{-3}、动量为 0.5 的 RMSProp 优化器，通过 100 次迭代优化系统参数或动作参数.

A.3 更多实验

A.3.1 视觉运动控制

我们考虑如下视觉运动控制任务：给定展示物理系统期望状态的目标图像，我们优化一个动作序列，将物理系统从随机生成的初始状态引导至目标状态. 目标状态从模仿学习任务的真实状态中选取. 表 A.1 报告在不同渲染配置和初始猜测下重复实验得到的状态误差. 状态误差定义为期望状态与仿真器最终状态之间的 L^1 距离. 表 A.1 中，我们方法较小的状态误差和标准差显示出相对于其他基线的优势. 与前文一样，所有方法的性能都以预言机为上限，而预言机需要更多有关真实状态的知识.

	四旋翼	手	杆
average	1.38 ± 0.00	0.23 ± 0.00	0.89 ± 0.00
random	63.93 ± 9.44	0.32 ± 0.06	0.87 ± 0.06
pixelwise-loss	3.02 ± 1.35	0.25 ± 0.05	0.05 ± 0.01
perceptual-loss	1.59 ± 0.26	0.28 ± 0.05	0.11 ± 0.06
ours-no-grad	$1.30 \pm \mathbf{0.15}$	0.23 ± 0.06	$\mathbf{0.04} \pm \mathbf{0.0002}$
ours	$\mathbf{0.54} \pm 0.25$	$\mathbf{0.11} \pm \mathbf{0.02}$	$\mathbf{0.04} \pm 0.0006$
oracle	0.15 ± 0.09	0.002 ± 0.00	0.01 ± 0.02

表 A.1: 视觉运动控制结果. 每项报告由四组随机生成的初始猜测和渲染条件计算的状态差异均值及标准差.

A.3.2 真实世界实验

问题设置. 我们考虑真实四旋翼飞行器的模仿学习任务. 输入包括相机内外参数、该相机拍摄的真实飞行器视频片段、飞行器系统参数的经验值，以及以网格文件表示的

飞行器几何形状. 目标是恢复一个动作序列，使仿真飞行器尽可能接近输入视频中的运动.

实验设置. 为获取真实视频，我们设置了一台内外参数已知且经过标定的相机，整个录制过程保持位置和姿态固定. 获得原始视频后，裁去首尾运动很少的帧，生成时长为 14.4 秒、频率为 10 Hz 的预处理视频. 为模拟潜在部署，我们选择一款用户可公开购买的商用四旋翼飞行器，手动控制它沿平滑轨迹运动.

为评估各方法，我们还建立了动作捕捉（MoCap）系统以记录飞行器的位置和旋转. 我们在飞行器上附加六个反光标记，使动作捕捉系统跟踪由这些标记张成的局部坐标系. 注意，我们仅将动作捕捉数据用于测试；它并非本流程的必要步骤，我们的方法均不依赖动作捕捉数据.

为与其他实验保持一致，RISP 使用与四旋翼环境相同的网络架构. 我们均匀采样位置和姿态，确保飞行器在画面内可见；每个采样状态对应不同的渲染配置. 共生成一万个数据点，按 8:2 划分训练集与测试集.

模仿学习. 我们从真实视频片段重建动作序列. 这比仿真任务困难得多：真实物理中存在不可预测的噪声，可微仿真无法完美建模；真实视频具有未见过的外观，包括但不限于光源和地板纹理；视频很长，也显著增加动作序列的维度.

为应对这些挑战，我们为真实实验设计了略有不同的 RISP 模仿学习流程. 首先在输入视频上运行 RISP，生成飞行器的目标轨迹；然后以每一视频帧的动作为决策变量，最小化目标轨迹与可微四旋翼仿真器生成轨迹之间差异定义的损失函数. 由于优化涉及许多自由度和较长时间范围，良好的初始猜测至关重要. 我们使用试图跟踪目标轨迹的手工设计控制器的输出初始化动作序列. 这样的初始猜测保证飞行器留在相机视野内，并由本方法和基线共同使用.

结果. 图 A-1 通过展示代表性帧子集给出真实实验结果.

RISP. 为验证稳健性，我们大幅随机化渲染配置，使之与输入视频显著不同. 我们的方法在无法获得输入视频渲染配置或精确动力学模型的情况下，重建了相似的动态运动.

GradSim. 我们在完全相同的渲染配置和初始猜测下，将 ∇Sim [126] 的逐像素损失与本方法比较. 我们观察到逐像素损失无法为优化提供有效指导，飞行器立即飞出画面. 我们推测，逐像素损失退化的主要原因是它假设目标域的渲染配置已知；但从真实视频获取这种信息通常并不容易.

GradSim-Enhanced. 基于上述推测，我们提出两项改进：首先手动调整渲染配置，使其尽可能接近真实视频；其次提供依据动作捕捉系统记录的真实轨迹计算得到的良好动作序列初始猜测. 注意，在真实应用中通常无法获得动作捕捉系统提供的良好初始猜测，RISP 和 GradSim 均未使用它. 即使获得这些强有利条件，GradSim-Enhanced 也只能恢复动作序列最初的一小段，最终仍不可控制地漂出画面.

A.3.3 消融研究

渲染梯度. 我们比较 ours 和 ours-no-grad，研究渲染梯度对数据效率的影响. 我们复用四旋翼状态估计数据集（表 2.1），但将渲染配置数量在 1 到 10 之间变化. 随后将两种方法各训练 100 个 epoch，并在包含 200 个随机状态的测试集上报告性能；每个状

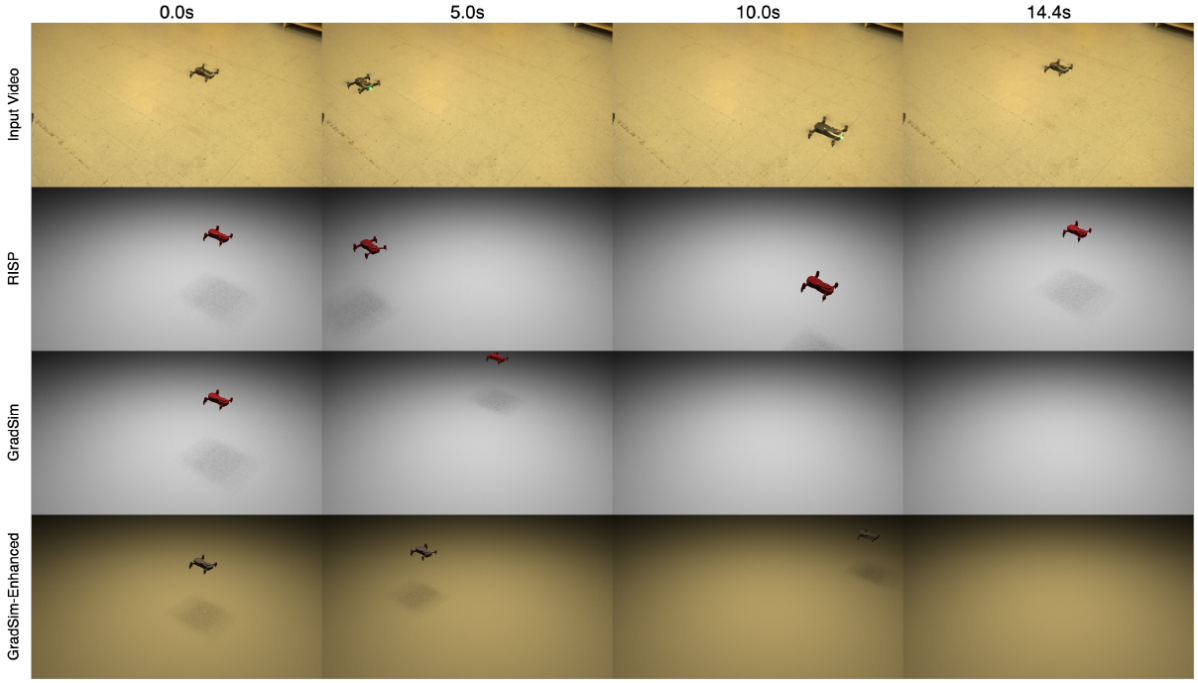
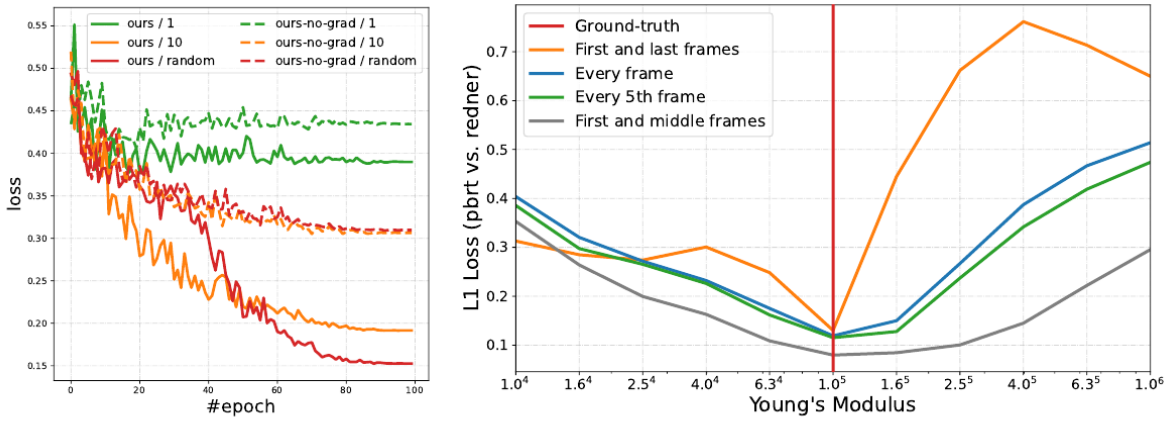


图 A-1: 真实实验中的模仿学习. 给定参考视频（上排），目标是重建一个与其运动相似的动作序列. 图示本方法（上中排）、逐像素损失（下中排）及其增强变体（下排）重建的运动. Input Video 为输入视频，RISP 为本方法，GradSim 和 GradSim-Enhanced 分别为逐像素损失方法及其增强变体.

态均以 10 个未见过的渲染配置增强. 结果见图 A-2(a). 右侧图例概括了不同渲染配置数量下 ours（实线）和 ours-no-grad（虚线）的表现，绿色、橙色和红色分别对应使用 1 个、10 个及随机采样渲染配置训练的结果. 所有实线均达到比对应虚线更低的状态估计损失，表明我们的渲染梯度能从相同数量的渲染配置中挖掘出更多信息. 值得注意的是，仅用 10 个渲染配置（橙色实线），带渲染梯度的方法所达到的损失就低于无梯度但采用随机采样渲染配置的方法（红色虚线），体现出更高的数据效率.

通过比较表 2.1、2.2、2.3 和 A.1 中的 ours 与 ours-no-grad，可以看到渲染梯度对方法性能的大幅改善至关重要. 我们强调，渲染不变的状态估计是本方法泛化能力的核心来源，也是许多下游任务成功的关键.



(a) **(b)**

图 A-2: (a) epoch 数与损失的曲线. 实线和虚线分别为 ours 和 ours-no-grad 的训练曲线. 颜色表示训练集中的渲染配置数量：绿色、橙色分别为 1 和 10，红色曲线对每个训练数据采样不同的渲染配置. (b) 损失与杨氏模量的曲线. 橙、蓝、绿、灰曲线表示不同时间采样率下关于杨氏模量的损失曲面，红色竖线为真实杨氏模量. 图内 loss、#epoch、Young's Modulus 分别为损失、训练轮数和杨氏模量；First and last frames、Every frame、Every 5th frame、First and middle frames 分别表示仅首末帧、每帧、每五帧及仅首帧和中间帧.

时间采样率. 我们使用四种采样策略研究时间采样率对 RISP 性能的影响：密集采样每一帧、稀疏采样每五帧、仅采样首末帧，以及仅采样首帧和中间帧. 我们复用杆环境中的系统辨识任务，在图 A-2(b) 中绘制真实杨氏模量附近的损失曲面. 我们观察到所有曲线具有相同的全局最优点，因此 RISP 对不同采样率具有稳健性. 此外，除信息大量丢失的极端情况（首末帧）外，大多数曲线是只有一个局部极小值的单峰曲线，意味着优化问题更容易. 因此，我们认为可以合理期待本方法配合标准的基于梯度的优化器，在不同时间采样率下取得成功.

Appendix B

从运动观测中学习神经本构律以实现可泛化 PDE 动力学：附录

B.1 物质点法

本节旨在展示如何从控制 PDE 系统地推导 MPM，即算法 1 中 $\mathcal{I}$ 的时间积分方案. 更多推导细节见 Sulsky 等 [164]、Stomakhin 等 [162]、Yue 等 [206] 和 Jiang 等 [76, 75].

推导 MPM 时，我们从质量守恒和动量守恒的 Euler 形式出发：

$$\frac{d\rho}{dt} = -\rho\nabla\cdot\boldsymbol{v}, \tag{B.1}$$

$$\rho\boldsymbol{a} = \nabla\cdot\boldsymbol{\sigma} + \rho\boldsymbol{b}, \tag{B.2}$$

其中 ρ 是密度，$\boldsymbol{a}$ 是加速度，$\boldsymbol{\sigma}$ 是 Cauchy 应力张量，$\boldsymbol{b}$ 是体力. 注意，质量守恒式 (B.1) 无需特殊处理，因为随 Lagrange MPM 粒子平流时它会自动满足.

B.1.1 弱形式

在运动方程 (B.2) 两侧乘以测试函数 $\boldsymbol{w}$，并在任意区域 Ω 上积分，得到弱形式：

$$\int_\Omega \rho\boldsymbol{a}\cdot\boldsymbol{w}\,d\Omega = \int_\Omega (\nabla\cdot\boldsymbol{\sigma})\cdot\boldsymbol{w}\,d\Omega + \int_\Omega \rho\boldsymbol{b}\cdot\boldsymbol{w}\,d\Omega. \tag{B.3}$$

这应对任意测试函数 $\boldsymbol{w}$ 和任意区域 Ω 成立. 分部积分后，上式变为

$$\int_\Omega \rho\boldsymbol{a}\cdot\boldsymbol{w}\,d\Omega = -\int_\Omega \boldsymbol{\sigma}:\nabla\boldsymbol{w}\,d\Omega + \int_{\partial\Omega_T}\boldsymbol{w}\cdot\boldsymbol{T}\,dS + \int_\Omega \rho\boldsymbol{b}\cdot\boldsymbol{w}\,d\Omega, \tag{B.4}$$

其中 $\boldsymbol{T}$ 是表面力，$\partial\Omega_T$ 为施加表面力的边界.

B.1.2 空间离散化

MPM 使用两组基函数，分别用于物质点和网格，对弱形式进行空间离散化，得到以下离散系统：

$$\sum_{b=1}^{G} M_{ab}\boldsymbol{a}_b = -\sum_{i=1}^{Q} V_i^0\boldsymbol{\tau}_i\nabla N_a(\boldsymbol{x}_i) + \sum_{i=1}^{Q} M_i N_a(\boldsymbol{x}_i)\boldsymbol{b}_i, \tag{B.5}$$

其中 $M_{ab} = \sum_{i=1}^{Q} M_i N_a(\boldsymbol{x}_i) \cdot N_b(\boldsymbol{x}_i)$ 是完整质量矩阵；V_i^0、M_i、$\boldsymbol{\tau}_i$、$\boldsymbol{x}_i$、$\boldsymbol{b}_i$ 分别是物质点 i 的初始体积、质量、Kirchhoff 应力、当前位置和外力；Q 是物质点数；N_b 和 $\boldsymbol{a}_b$ 是 Euler 网格节点 b 上的基函数和加速度；G 是 Euler 基函数的数量. 更多推导细节见 Sulsky 等 [164].

B.1.3 时间离散化

使用时间步长为 Δt 的显式 Euler 方案，可以进行时间离散化：

$$\sum_{b=1}^{G} M_{ab} \frac{\boldsymbol{v}_b^{n+1} - \boldsymbol{v}_b^n}{\Delta t} = -\sum_{i=1}^{Q} V_i^0 \boldsymbol{\tau}_i^n \nabla N_a(\boldsymbol{x}_i^n) + \sum_{i=1}^{Q} M_i N_a(\boldsymbol{x}_i^n) \boldsymbol{b}_i^n. \tag{B.6}$$

B.1.4 MPM 伪代码

根据上述空间和时间离散化，我们得到算法 4 中的伪代码，并在 Hu 等 [65] 的 MLS-MPM 框架下实现.

算法 4：MPM 算法

输入： 时刻 t_n 各物质点 $i = 1, \ldots, Q$ 的位置 $\boldsymbol{x}_n^i$、速度 $\boldsymbol{b}_n^i$ 和弹性变形梯度 $\boldsymbol{F}_n^{e,i}$.

输出： 时刻 t_{n+1} 各物质点的位置 $\boldsymbol{x}_{n+1}^i$、速度 $\boldsymbol{b}_{n+1}^i$ 和试探弹性变形梯度 $\boldsymbol{F}_{n+1}^{e,\mathrm{trial},i}$.

1. 通过 “粒子到网格” 传递，将 Lagrange 运动学量传至 Euler 网格；对 $b = 1, \ldots, G$ 计算：

$$m_{b,n} = \sum_{i=1}^{Q} N_b(\boldsymbol{x}_n^i) M_i, \qquad m_{b,n} \boldsymbol{b}_{b,n} = \sum_{i=1}^{Q} N_b(\boldsymbol{x}_n^i) M_i \boldsymbol{b}_n^i,$$

$$\boldsymbol{f}_{b,n}^{\sigma} = -\sum_{i=1}^{Q} \frac{J(\boldsymbol{F}_n^{e,i})}{\rho_0} \boldsymbol{\sigma}(\boldsymbol{F}_n^{e,i}) \nabla N_b(\boldsymbol{x}_n^i) M_i, \qquad \boldsymbol{f}_{b,n}^{e} = \sum_{i=1}^{Q} \frac{J(\boldsymbol{F}_n^{e,i})}{\rho_0} \boldsymbol{b}(\boldsymbol{x}_n^i) N_b(\boldsymbol{x}_n^i) M_i.$$

2. 对 $b = 1, \ldots, G$ 计算，以求解 Euler 控制方程：

$$\dot{\boldsymbol{b}}_{b,n+1} = \frac{1}{m_{b,n}} (\boldsymbol{f}_{b,n}^{\sigma} + \boldsymbol{f}_{b,n}^{e}), \qquad \Delta \boldsymbol{b}_{b,n+1} = \dot{\boldsymbol{b}}_{b,n+1} \Delta t, \qquad \boldsymbol{b}_{b,n+1} = \boldsymbol{b}_{b,n} + \Delta \boldsymbol{b}_{b,n+1}.$$

3. 通过 “网格到粒子” 传递更新 Lagrange 速度和变形梯度；对 $i = 1, \ldots, Q$ 计算：

$$\boldsymbol{b}_{n+1}^i = \sum_{i=1}^{G} N_i(\boldsymbol{x}_n^i) \boldsymbol{b}_{b,n+1},$$

$$\boldsymbol{F}_{n+1}^{e,\mathrm{trial},i} = \left(\boldsymbol{I} + \sum_{i=1}^{G} \boldsymbol{b}_{i,n+1} \otimes \nabla N_i(\boldsymbol{x}_n^p) \Delta t \right) \boldsymbol{F}_n^{e,i}.$$

4. 对 $i = 1, \ldots, Q$ 更新 Lagrange 位置：

$$\boldsymbol{x}_{n+1}^i = \boldsymbol{x}_n^i + \Delta t \boldsymbol{b}_{n+1}^i.$$

译注： 算法 4 第三步的求和指标与外层粒子指标重复，且 b、p 在相应式中未绑定；原文还以 $\boldsymbol{b}$ 表示速度及体力. 此处保留原记号，未推定作者本意.

至此完成 MPM 及算法 1 中 $\mathcal{I}$ 时间积分方案的背景介绍. 本工作引入的神经本构律通过两种方式参与时间积分：神经弹性本构律 $\mathcal{E}_{\theta_e}$ 计算第一 Piola–Kirchhoff 应力 $\boldsymbol{P}$（等价地，也可计算 Cauchy 应力 $\boldsymbol{\sigma}$ 或 Kirchhoff 应力 $\boldsymbol{\tau}$），该应力用于计算网格节点的力；神经塑性本构律 $\mathcal{P}_{\theta_p}$ 则通过将试探弹性变形梯度投影回允许的弹性区域，对其进行后处理.

B.2 用于训练的材料模型

下面列出生成真实训练数据所用的材料模型. 对每种材料，我们同时列出弹性本构律 $\mathcal{E}_\mu$ 和塑性本构律 $\mathcal{P}_\mu$. 我们强调，本方法能够使用相同网络架构和训练策略捕捉所有这些由专家设计的经典本构模型.

纯弹性. 纯弹性材料包括橡胶、生物组织和果冻. 我们使用 Stomakhin 等 [161] 的固定共旋超弹性模型：

$$\boldsymbol{P} = 2\mu(\boldsymbol{F} - \boldsymbol{R}) + \lambda J(J-1)\boldsymbol{F}^{-T}, \tag{B.7}$$

其中 $\boldsymbol{R}$ 来自极分解 $\boldsymbol{F} = \boldsymbol{R}\boldsymbol{S}$. 该材料没有塑性；等价地，塑性返回映射为恒等映射：

$$\mathcal{P}(\boldsymbol{F}) = \boldsymbol{F}. \tag{B.8}$$

具有 Drucker–Prager 屈服条件的弹性材料. 我们使用 Saint Venant–Kirchhoff 弹性模型. 对 $\boldsymbol{F} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^T$ 进行奇异值分解并计算 Hencky 应变 $\boldsymbol{\epsilon} = \log(\boldsymbol{\Sigma})$，得到 [11]

$$\boldsymbol{P} = \boldsymbol{U}(2\mu\boldsymbol{\epsilon} + \lambda \operatorname{Tr}(\boldsymbol{\epsilon}))\boldsymbol{U}^T. \tag{B.9}$$

译注： 式 (B.9) 及下文式 (B.12) 中矩阵与标量相加时，原文未显式写出单位矩阵；应力类型及变换关系亦需进一步核实. 这里保留原式.

此外，我们应用 Drucker–Prager 屈服条件 [34, 86, 207, 23]：

$$\operatorname{Tr}(\boldsymbol{\epsilon}) > 0 \quad \text{或} \quad \delta\gamma = \|\hat{\boldsymbol{\epsilon}}\| + \alpha\frac{(3\lambda + 2\mu)\operatorname{Tr}(\boldsymbol{\epsilon})}{2\mu} > 0, \tag{B.10}$$

其中 $\alpha = \sqrt{2/3}\,2\sin\theta/(3-\sin\theta)$，$\theta$ 为颗粒介质的摩擦角.

基于这一屈服准则，我们定义两种塑性投影. 材料受拉时，将应力投影至锥顶；材料受压且违反锥约束时，以等体积方式将应力投影至锥面. 数学上有

$$\mathcal{P}(\boldsymbol{F}) = \begin{cases} \boldsymbol{U}\boldsymbol{V}^T, & \operatorname{Tr}(\boldsymbol{\epsilon}) > 0, \\ \boldsymbol{F}, & \operatorname{Tr}(\boldsymbol{\epsilon}) \le 0 \ \&\ \delta\gamma \le 0, \\ \boldsymbol{U}\exp\left(\boldsymbol{\epsilon} - \delta\gamma\dfrac{\hat{\boldsymbol{\epsilon}}}{\|\hat{\boldsymbol{\epsilon}}\|}\right)\boldsymbol{V}^T, & \operatorname{Tr}(\boldsymbol{\epsilon}) \le 0 \ \&\ \delta\gamma > 0. \end{cases} \tag{B.11}$$

具有 von Mises 屈服条件的弹性材料. 我们采用前述相同的 Saint Venant–Kirchhoff 弹性模型：

$$\boldsymbol{P} = \boldsymbol{U}(2\mu\boldsymbol{\epsilon} + \lambda \operatorname{Tr}(\boldsymbol{\epsilon}))\boldsymbol{U}^T. \tag{B.12}$$

但此处使用 von Mises 屈服条件 [123, 65, 70]：

$$\delta\gamma = \|\hat{\boldsymbol{\epsilon}}\| - \frac{\tau_Y}{2\mu}, \tag{B.13}$$

其中 $\boldsymbol{\epsilon}$ 是由变形梯度 $\boldsymbol{F}$ 计算的归一化 Hencky 应变；τ_Y 是屈服应力，描述材料发生塑性流动的难易程度. 正的 $\delta\gamma$ 即 $\delta\gamma > 0$ 表明材料违反屈服约束. 对违反屈服准则的应力，我们通过等体积投影将其投回弹性区域：

$$\mathcal{P}(\boldsymbol{F}) = \begin{cases} \boldsymbol{F}, & \delta\gamma \le 0, \\ \boldsymbol{U}\exp\left(\boldsymbol{\epsilon} - \delta\gamma\dfrac{\hat{\boldsymbol{\epsilon}}}{\|\hat{\boldsymbol{\epsilon}}\|}\right)\boldsymbol{V}^T, & \delta\gamma > 0. \end{cases} \tag{B.14}$$

弱可压缩流体. 按照 Stomakhin 等 [163]、Tampubolon 等 [167]，我们用前述固定共旋弹性模型并取 $\mu = 0$ 对流体建模，相当于使流体没有剪切阻力，同时惩罚任何体积变化：

$$\boldsymbol{P} = \lambda J(J-1)\boldsymbol{F}^{-T}. \tag{B.15}$$

此外，我们采用 Stomakhin 等 [163]、Gao 等 [46] 的塑性模型：

$$\mathcal{P}(\boldsymbol{F}) = J^{1/3}\boldsymbol{I}. \tag{B.16}$$

实际上，这一流动法则以等体积方式将变形梯度投影到静水轴上. 本工作聚焦上述四类材料；探索其他弹塑性材料（例如非 Newton 流体 [206]）留作未来工作.

B.3 神经本构律算法

B.3.1 算法

算法 5 展示了完整流程. 注意，可以通过奇异值分解（SVD）$\boldsymbol{F} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^T$ 方便地实现极分解 $\boldsymbol{F} = \boldsymbol{R}\boldsymbol{S}$，得到 $\boldsymbol{R} = \boldsymbol{U}\boldsymbol{V}^T$ 和 $\boldsymbol{S} = \boldsymbol{V}\boldsymbol{\Sigma}\boldsymbol{V}^T$. 我们在 GPU 上使用高度优化的 3×3 SVD 实现 [46]，所有实验取 $\alpha = 0.001$.

算法 5：神经本构律

输入： $\boldsymbol{F}$（弹性时为 $\boldsymbol{F}^e$；塑性时为 $\boldsymbol{F}^{e,\text{trial}}$）.

输出： $\boldsymbol{P}$ 或 $\boldsymbol{F}^{e,\text{new}}$.

1. $\boldsymbol{F} \overset{\text{SVD}}{=} \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^T$（奇异值分解）.
2. $\boldsymbol{R} = \boldsymbol{U}\boldsymbol{V}^T$（旋转等变）.
3. $\boldsymbol{T}_1 = \text{NN}(\boldsymbol{\Sigma}, \boldsymbol{F}^T\boldsymbol{F}, \det(\boldsymbol{F}))$（旋转不变）.
4. $\boldsymbol{T}_2 = \frac{1}{2}(\boldsymbol{T}_1 + \boldsymbol{T}_1^T)$（保证对称性）.
5. $\boldsymbol{Y} = \boldsymbol{R}\boldsymbol{T}_2$（保证旋转等变性）.
6. 若为弹性，令 $\boldsymbol{P} = \boldsymbol{Y}$.
7. 若为塑性，令 $\boldsymbol{F}^{e,\text{new}} = \boldsymbol{F} + \alpha\boldsymbol{Y}$.

B.3.2 旋转不变性的证明

证明： 网络有三个输入；我们逐一证明它们相对于任意旋转 $\boldsymbol{R}^* \in \mathrm{SO}(3)$ 不变.

1. 奇异值 $\boldsymbol{\Sigma}$. 有

$$\begin{aligned} \boldsymbol{F}_{\mathrm{new}} &= \boldsymbol{R}^* \boldsymbol{F} && \text{(B.17)} \\ &= \boldsymbol{R}^* \boldsymbol{U} \boldsymbol{\Sigma} \boldsymbol{V}^T && \text{(B.18)} \\ &= (\boldsymbol{R}^* \boldsymbol{U}) \boldsymbol{\Sigma} \boldsymbol{V}^T, && \text{(B.19)} \end{aligned}$$

   其中 $\boldsymbol{R}^*\boldsymbol{U}$ 仍是酉矩阵，因为 $\boldsymbol{R}^*$ 和 $\boldsymbol{U}$ 均为酉矩阵. 因而施加 $\boldsymbol{R}^*$ 时，$\boldsymbol{\Sigma}$ 不变.

2. 右 Cauchy–Green 张量 $\boldsymbol{F}^T\boldsymbol{F}$. 有

$$\begin{aligned} \boldsymbol{F}_{\mathrm{new}}^T \boldsymbol{F}_{\mathrm{new}} &= (\boldsymbol{R}^* \boldsymbol{F})^T (\boldsymbol{R}^* \boldsymbol{F}) && \text{(B.20)} \\ &= \boldsymbol{F}^T (\boldsymbol{R}^*)^T \boldsymbol{R}^* \boldsymbol{F} && \text{(B.21)} \\ &= \boldsymbol{F}^T \boldsymbol{F}. && \text{(B.22)} \end{aligned}$$

3. 变形梯度的行列式 $\det(\boldsymbol{F})$. 有

$$\begin{aligned} \det(\boldsymbol{F}_{\mathrm{new}}) &= \det(\boldsymbol{R}^* \boldsymbol{F}) && \text{(B.23)} \\ &= \det(\boldsymbol{R}^*) \det(\boldsymbol{F}) && \text{(B.24)} \\ &= \det(\boldsymbol{F}), && \text{(B.25)} \end{aligned}$$

   因为酉矩阵的行列式为 1.[4]

因此神经网络的所有输入均旋转不变，蕴含网络输出旋转不变. 我们还有 $\mathrm{NN}(\boldsymbol{R}^*\boldsymbol{F}) = \mathrm{NN}(\boldsymbol{F})$；由于 $\boldsymbol{T}_1$ 和 $\boldsymbol{T}_2$ 仅依赖神经网络输出，二者也旋转不变. □

B.3.3 旋转等变性的证明

证明： 施加任意旋转 $\boldsymbol{R}^* \in \mathrm{SO}(3)$ 后，神经本构律的输入变为 $\boldsymbol{R}^*\boldsymbol{F}$. 根据极分解的定义：

$$\boldsymbol{F} \overset{\mathrm{PD}}{=} \boldsymbol{R}\boldsymbol{S}, \tag{B.26}$$

$$\boldsymbol{R}^* \boldsymbol{F} = \boldsymbol{F}_{\mathrm{new}} \overset{\mathrm{PD}}{=} \boldsymbol{R}_{\mathrm{new}} \boldsymbol{S} = (\boldsymbol{R}^* \boldsymbol{R}) \boldsymbol{S}. \tag{B.27}$$

因此，以 $\boldsymbol{F}_{\mathrm{new}}$ 为输入时，神经本构律的输出为

$$\boldsymbol{Y}_{\mathrm{new}} = \boldsymbol{R}_{\mathrm{new}} \boldsymbol{T}_2 = \boldsymbol{R}^* \boldsymbol{R} \boldsymbol{T}_2 = \boldsymbol{R}^* \boldsymbol{Y}, \tag{B.28}$$

这证明了 $\boldsymbol{Y}$ 的旋转等变性. 对弹性，旋转等变性自动继承. 对塑性，有

$$\begin{aligned} \boldsymbol{F}_{\mathrm{new}}^{e,\mathrm{new}} &= \boldsymbol{F}_{\mathrm{new}}^{e,\mathrm{trial}} + \alpha \boldsymbol{Y}_{\mathrm{new}} && \text{(B.29)} \\ &= \boldsymbol{R}^* \boldsymbol{F}^{e,\mathrm{trial}} + \alpha \boldsymbol{R}^* \boldsymbol{Y} && \text{(B.30)} \\ &= \boldsymbol{R}^* (\boldsymbol{F}^{e,\mathrm{trial}} + \alpha \boldsymbol{Y}) && \text{(B.31)} \\ &= \boldsymbol{R}^* \boldsymbol{F}^{e,\mathrm{new}}, && \text{(B.32)} \end{aligned}$$

这证明了神经塑性本构律的旋转等变性. □

[4]译注：一般酉矩阵的行列式仅必然具有单位模，不必等于 1；此处 $\boldsymbol{R}^* \in SO(3)$，故其行列式为 1.

B.4 实现细节

B.4.1 仿真

对每个训练环境，我们在原文记为 $0.5^3\,\mathrm{m}$ 的立方体内放入物质点. 为加速仿真，MPM 仿真器网格设为 $20 \times 20 \times 20$. 我们使用三个网格的阈值检测碰撞，并在原文记为 $1.0^3\,\mathrm{m}$ 的箱体内设置自由滑移边界条件.[5] 仿真始终施加 $-9.8\,\mathrm{m/s^2}$ 的重力加速度. 所有训练环境统一使用相同的 MPM 超参数. 可微 MPM 仿真器使用 Warp [118] 在 GPU 上实现.

B.4.2 网络与训练

神经本构律包含两个规模相同的神经网络，分别用于弹性和塑性，总计 11008 个参数. 神经网络使用 GELU [62] 作为非线性函数，不含归一化层. 我们使用 Adam 优化器 [84]，弹性与塑性的学习率分别为 1.0 和 0.1. 神经本构律训练 300 个 epoch，两者学习率均通过余弦退火调度衰减；梯度范数截断至最大 0.1. 我们还使用周期性从真实位置重新开始的 “教师强制” 方案，并通过余弦退火调度将该周期从 25 步增加到 200 步. 所有实验均在一块 NVIDIA RTX A6000 上训练，使用相同训练配置和超参数，不作临时调参.

我们使用 PyTorch [134] 实现神经网络；它可以与 Warp 共享 CUDA 内存，以降低与仿真器交互的开销.

B.4.3 真实世界实验

我们使用面团作为测试材料. 面团是由面粉和水混合而成的可塑且有弹性的糊状物，其复杂、难以预测的力学性质 [119, 3] 使之成为算法的良好测试对象. 我们将 125 g 面粉与 75 g 水混合并揉约 5 分钟制成面团，再将其塑造成多种二维形状用于训练和测试.

在物理实验中（图 B-1），面团从 50 cm 高处落下并与刚性表面碰撞. 此过程由高速相机（SONY RX100）以每秒 960 帧、曝光时间 1/4000 秒记录. 我们在面团正面标记约 50 个绿色点，随后跟踪这些点，以提供精确的变形历史信息.

B.5 额外讨论

B.5.1 向基于网格的仿真泛化

在物理仿真的所有组成部分中，我们的方法只用精心设计的神经网络替代人工调节的本构律；这是表达能力、一致性和多功能性更好的数据驱动表征. 因此神经本构律不依赖离散化和仿真框架. 当前工作选择物质点法（MPM）作为骨干，是因为它兼容多种材料，便于实现. 尽管如此，我们的框架具有通用性且与离散化无关. 我们预期该方法

[5] 译注：原文两处均写成数值的三次方后接单位 m；其量纲与预期表示的边长或体积之间的关系有歧义，译文保留原记法.

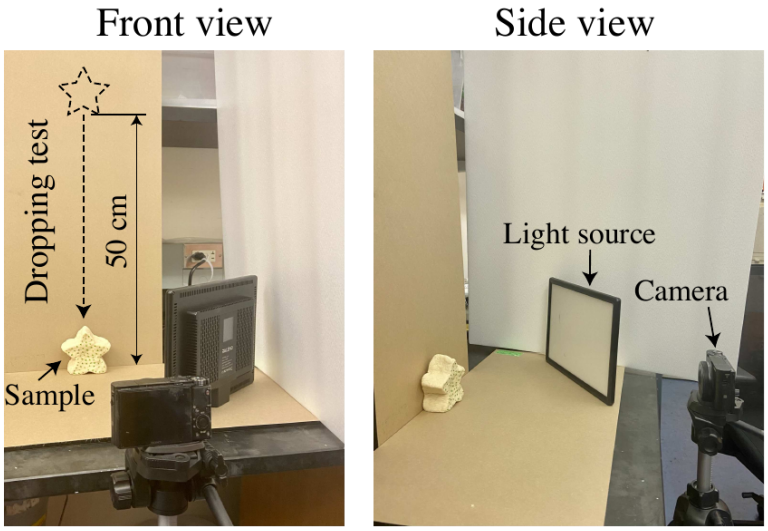


图 B-1: 真实实验. 实验装置的正视图和侧视图；虚线轮廓表示面团的下落位置，虚线箭头表示方向. Front view、Side view、Dropping test、Sample、Light source、Camera 分别为正视图、侧视图、下落测试、样品、光源、相机.

也适用于基于网格、网格 / 粒子混合仿真或任何离散化框架；但实现支持网格及网格 / 粒子耦合的可微仿真需要大量工程工作，我们认为最好以专门的论文处理.

B.6 额外实验

B.6.1 运行时间比较

我们评估本方法的运行时间性能，并在表 B.1 中与基线比较. 实验运行至仿真覆盖真实世界中的 0.5 秒. 由于时间步长不同，ours 和 analytical 需要 1000 步，gnn 需要 200 步. 我们将同一轨迹重复运行十次并取平均，以减少方差. 尽管 gnn 的步长大得多，我们发现本方法的速度与之相当. 我们推测原因是：方法的所有组件均在 GPU 上开发；半径图网络耗时较长. 由于我们未着重优化运行时间，方法仍有改进空间，包括消除弹性和塑性中的冗余 SVD、减少 Warp 与 PyTorch 间的通信，以及部署时用轻量级神经网络库替换 PyTorch.

方法	果冻	沙	橡皮泥	水
ours	2.56s	2.55s	2.62s	2.56s
gnn	2.35s	2.10s	2.19s	2.26s

表 B.1: 运行时间比较.

B.6.2 与较近期方法比较

我们选择一项相关的既有工作 [102]，在实验的一个子集上评估. 使用橡皮泥环境，四项任务的性能比较见表 B.2. Li 和 Farimani [102] 在四项任务中的三项上优于本文原

始 gnn 基线；然而本方法仍以数个数量级的优势强于这些基于 GNN 的方法. 可能有两个原因：一是本方法采用保证旋转等变性和未变形状态平衡的物理感知网络架构；二是将本构律学习与仿真框架解耦，提高数据效率.

任务	ours	gnn	Li 和 Farimani [102]
重建	6.5e−5	8.7e−3	9.3e−4
时间	1.4e−4	1.1e−2	5.2e−3
速度	4.6e−5	6.8e−3	1.0e−3
几何形状	2.3e−4	3.7e−2	4.7e−2

表 B.2: 与较近期方法比较.

B.6.3 参数辨识

我们使用水生成一条轨迹，随机选择 1000 个变形梯度作为神经网络输入. 测量 ours 和 labeled 相对于真实值的均方误差（MSE），结果见表 B.3. 注意，由于应力张量的物理意义，$\boldsymbol{P}$ 的误差自然大于变形梯度的误差（通常高出数个数量级）. 表中本方法在 $\boldsymbol{F}^{e,\text{new}}$ 上达到相近结果，但在 $\boldsymbol{P}$ 上落后于 labeled. 原因包括：labeled 实际使用了真实 $\boldsymbol{P}$ 和 $\boldsymbol{F}^{e,\text{new}}$ 的额外监督，而这些在真实世界中不易获得；本方法使用时间反向传播（BPTT）训练，而 labeled 使用没有时间步进的传统训练. 我们的方法在一定程度上牺牲训练效率，但更自然地适用于随时间变化的物理系统. 最后，在连续介质力学中，不同本构律可能以不同应力（例如 $\boldsymbol{P}$）描述相同运动学（例如 $\boldsymbol{F}$）. 本方法的目标是恢复能够泛化到新场景的运动学量（$\boldsymbol{x}, \boldsymbol{F}$）.

尽管本方法在 $\boldsymbol{P}$ 上不如 labeled 准确，它仍能预测精确且可泛化的运动学量. 如表 3.1 所示，本方法在所有环境中也比 labeled 更稳定.

项	ours	labeled
$\boldsymbol{P}$	6.1e−1	2.6e−3
$\boldsymbol{F}^{e,\text{new}}$	2.388e−6	2.381e−6

表 B.3: 参数辨识.

B.6.4 数据效率

对 gnn，我们尽可能保持 Sanchez-Gonzalez 等 [154] 的训练细节. 由于其论文中实现细节缺失，我们调整 spline 和 neural 的训练流程，使之与本方法兼容. 为通过改变训练轨迹数研究数据量对性能的影响，我们选择对数据依赖最强的基线 gnn，分别用 10 条和 100 条数据轨迹重新训练. 在水环境的速度泛化任务中，我们随机采样三个初始速度，计算评估结果与真实结果之间损失的平均值，比较见表 B.4. 随轨迹数增长而逐渐降低的损失表明，gnn 使用更多数据时泛化更好. 然而，即使用少 100 倍的数据，ours 仍以数个数量级优于 gnn，体现本方法的效率.

方法	轨迹数	损失
gnn	1	5.8e−2
gnn	10	2.5e−2
gnn	100	1.2e−2
ours	1	1.9e−5

表 B.4: 数据效率.

B.6.5 辅助损失

我们研究另一个重要状态变量——速度——的影响，额外加入惩罚仿真速度与真实速度差异的项. 表 B.5 比较水环境中有无速度差异惩罚时的位置损失. 结果显示，四项任务中有三项从速度差异惩罚中略微获益. 我们认为，引入互补损失能在一定程度上帮助训练；但快速入门时仅用位置监督可能已足够，而位置通常是现实中最容易获得的真实数据.

任务	无速度损失	有速度损失
重建	2.0e−5	1.3e−5
时间	3.5e−4	2.1e−4
速度	1.9e−5	1.8e−5
几何形状	2.4e−4	3.0e−4

表 B.5: 辅助损失.

B.6.6 更多可视化

我们在图 B-2 中展示训练数据集，在图 B-3 和图 B-4 中比较本方法、基线及真实结果对不同几何形状的泛化能力.

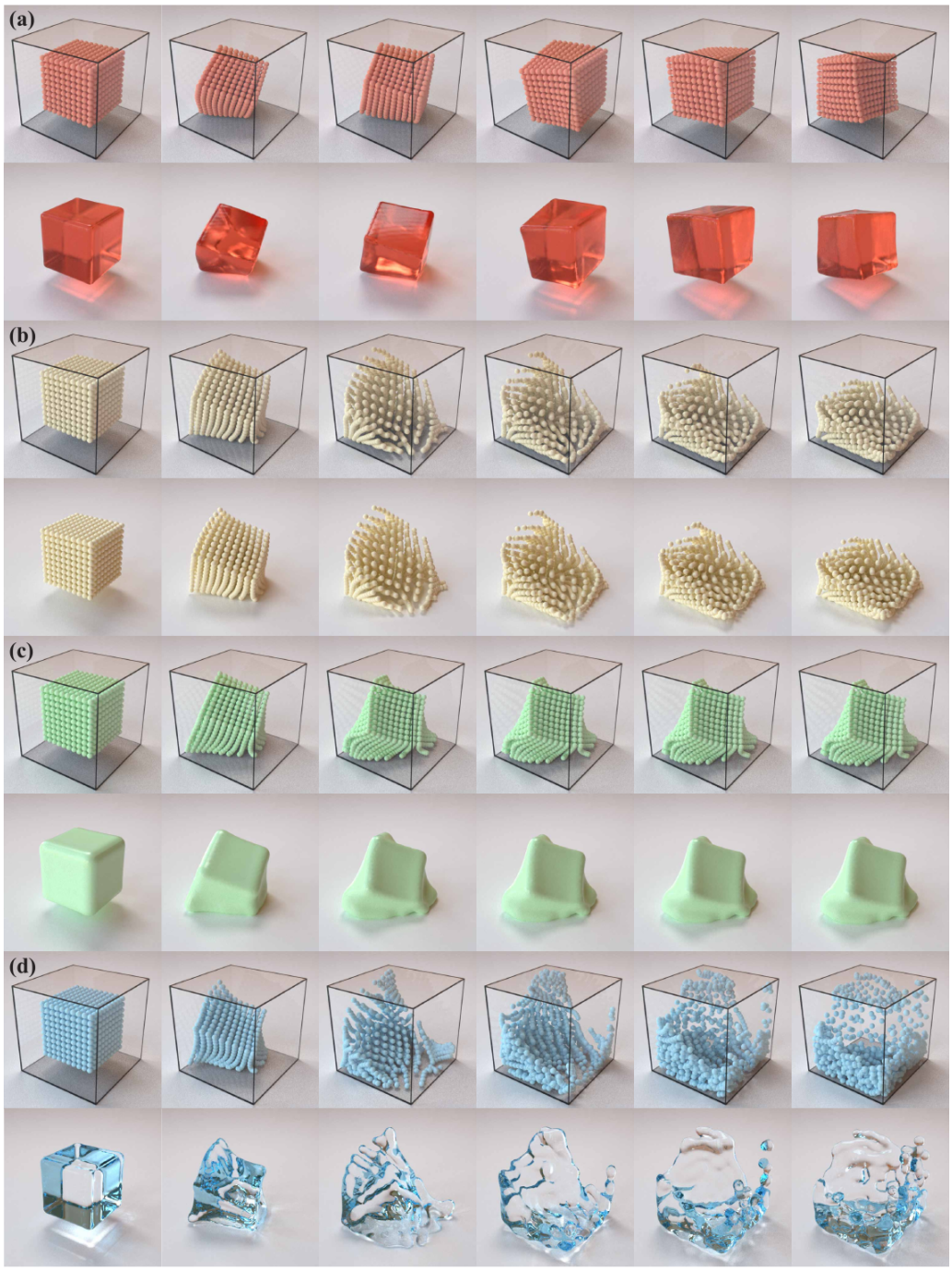


图 B-2: 训练数据. 展示 (a) 果冻、(b) 沙、(c) 橡皮泥、(d) 水的颗粒与真实感渲染.

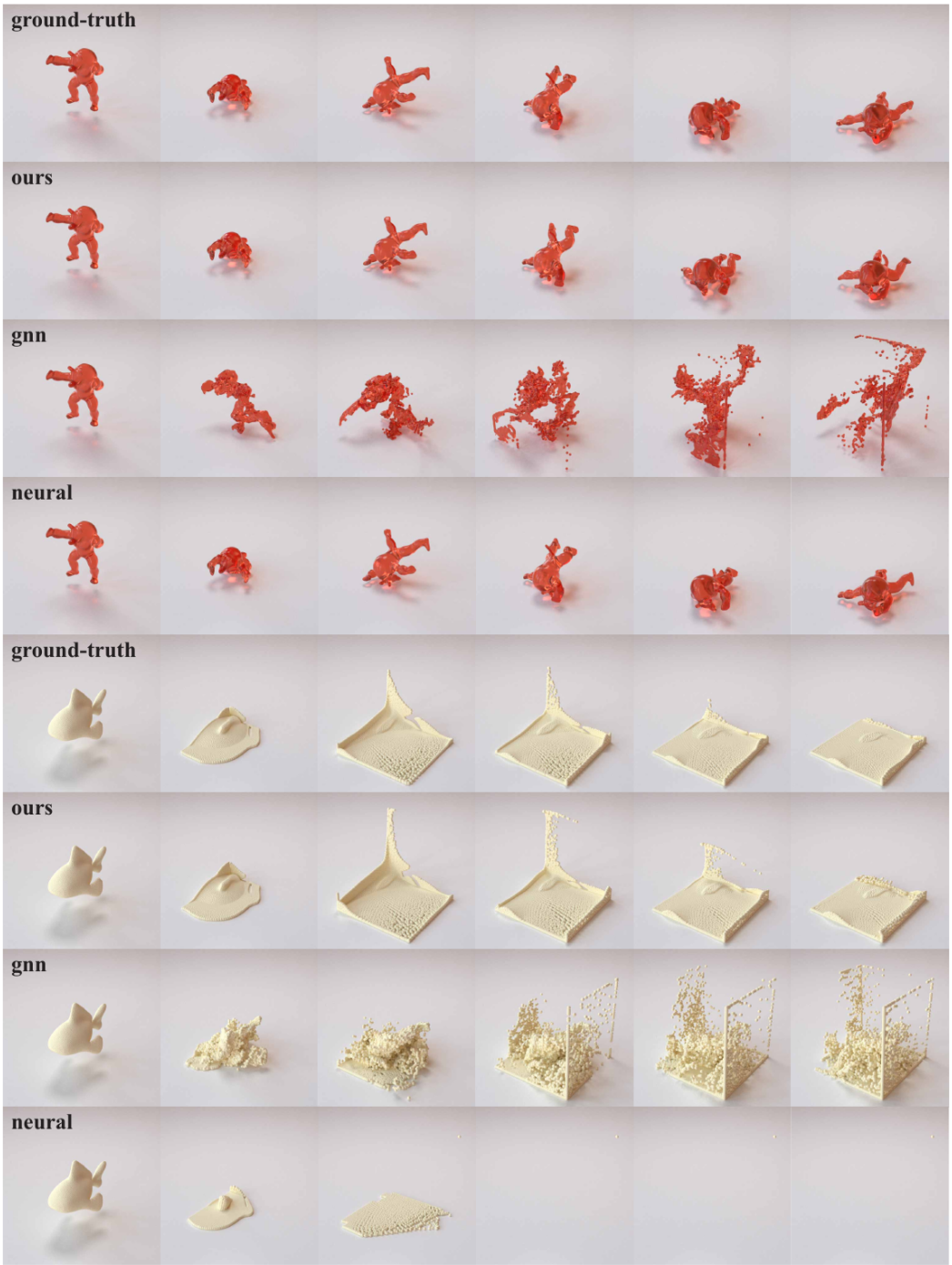


图 B-3: 果冻和沙的几何泛化. 果冻采用犰狳几何形状，沙采用金鱼几何形状；比较 ours、ground-truth 与基线 gnn 和 neural. 其中 ground-truth 为真实结果，ours 为本方法.

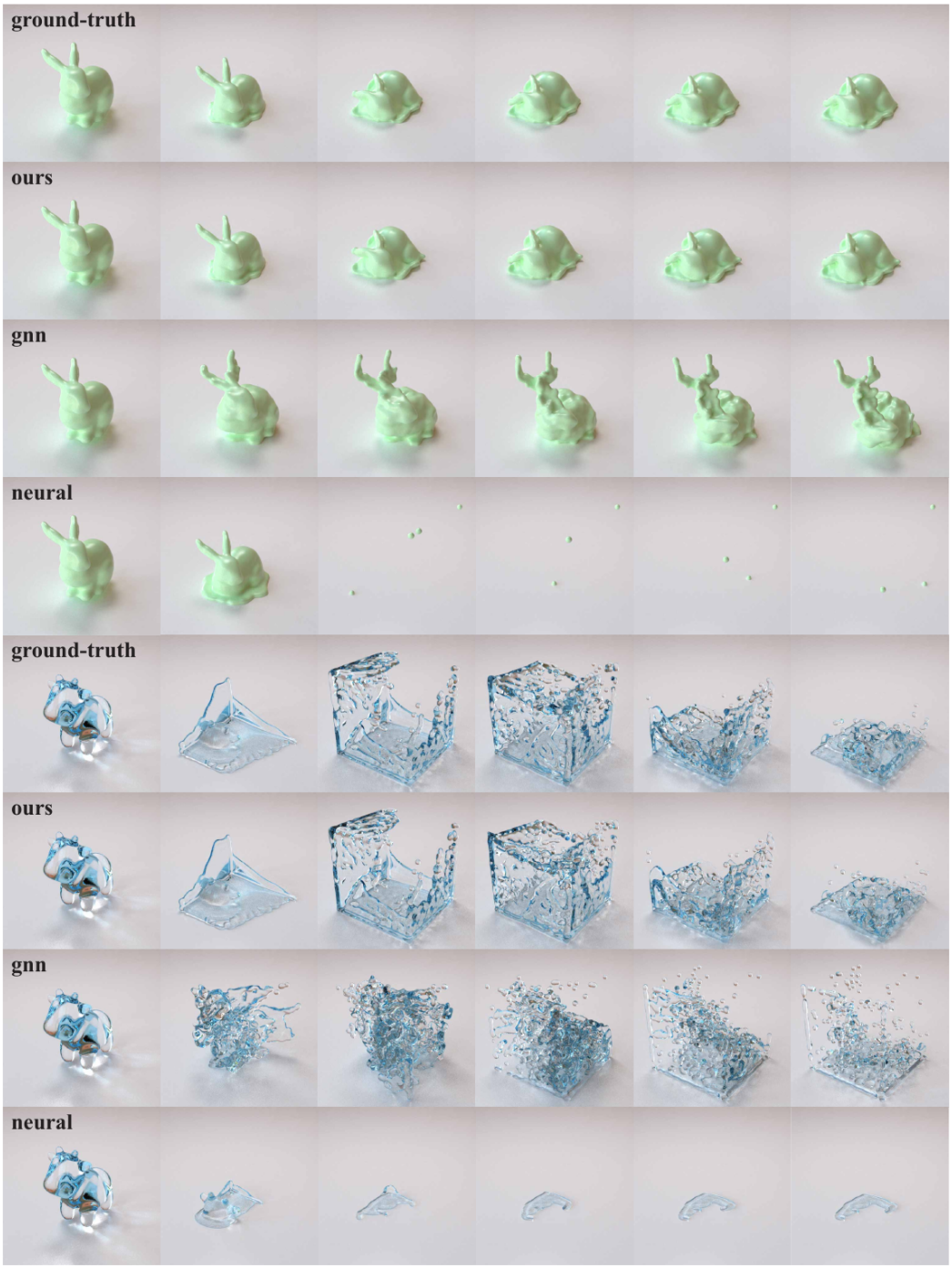


图 B-4: 橡皮泥和水的几何泛化. 橡皮泥采用兔子几何形状，水采用奶牛几何形状；比较 ours、ground-truth 与基线 gnn 和 neural. 其中 ground-truth 为真实结果，ours 为本方法.

Appendix C

LLM 与仿真作为双层优化器：物理科学发现新范式的附录

C.1 完整提示词

本构律发现的系统提示词：

你是一位擅长编程、物理仿真和科学发现的智能 AI 助手. 请仔细遵循用户要求，确保理解这些要求. 你的专业范围严格限于物理仿真、材料科学、数学和编程. 回答应简短、中肯，不提供未被请求的信息. 始终以代码注释记录并解释代码背后的原因. 使用 Markdown 排版解决方案. 你非常熟悉 Python 和 PyTorch. 不要使用示例中所用库之外的任何外部库.

分子设计的系统提示词：

你是一位擅长编程、分子设计和科学发现的智能 AI 助手. 请仔细遵循用户要求，确保理解这些要求. 你的专业范围严格限于物理仿真、材料科学、化学、分子设计、数学和编程. 回答应简短、中肯，不提供未被请求的信息. 始终以代码注释记录并解释代码背后的原因. 使用 Markdown 排版解决方案. 你非常熟悉 PyTorch，也非常熟悉 SMILES（简化分子线性输入规范）. 不要使用示例中所用库之外的任何外部库.

弹性本构律发现的代码格式提示词：
格式要求：PyTorch 提示.

1. 对两个矩阵逐元素相乘时，操作前确保维数匹配. 例如，将形状为 `(B,)` 的 `J` 与形状为 `(B, 3, 3)` 的 `I` 相乘前，应执行 `J.view(-1, 1, 1)`；同样，`(J - 1)` 也应重塑为 `(J - 1).view(-1, 1, 1)`. 若不确定，逐一写出表达式的每个分量，并在注释中标注维数以供验证.

2. 计算形状为 `(B, 3, 3)` 的张量 A 的迹时，用下列代码计算：

   `A.diagonal(dim1=1, dim2=2).sum(dim=1).view(-1, 1, 1)`.

   避免使用只支持二维矩阵的 `torch.trace` 或 `Tensor.trace`.

代码要求.

1. 编程语言始终为 Python.

2. 每次张量操作后以注释标注张量大小，例如 `# (B, 3, 3)`.

3. 唯一允许的库是 PyTorch. 遵循用户提供的示例并查阅 PyTorch 文档学习其用法.

4. 将代码分成可通过可微优化调节的连续物理参数，以及以 PyTorch 代码表示的符号本构律；分别在 `__init__` 和 `forward` 中定义.

5. `forward` 的第一个输出是更新后的变形梯度；第二个输出始终是 Kirchhoff 应力张量，定义为第一 Piola–Kirchhoff 应力张量与变形梯度张量转置的矩阵乘积，即 `tau = P @ F^T`. 除 Kirchhoff 应力张量外，不要直接返回任何其他类型的应力张量；使用此式计算 Kirchhoff 应力.

6. 所提代码必须严格遵循下面的结构和函数签名：

```
import torch
import torch.nn as nn

class Physics(nn.Module):
    def __init__(self, param: float = DEFAULT_VALUE):
        """
        Define trainable continuous physical parameters for differentiable optimization.
        Tentatively initialize the parameters with the default values in args.

        Args:
            param (float): the physical meaning of the parameter.
        """
        super().__init__()
        self.param = nn.Parameter(torch.tensor(param))

    def forward(self, F: torch.Tensor) -> torch.Tensor:
        """
        Compute Kirchhoff stress tensor from deformation gradient tensor.

        Args:
            F (torch.Tensor): deformation gradient tensor (B, 3, 3).

        Returns:
            kirchhoff_stress (torch.Tensor): Kirchhoff stress tensor (B, 3, 3).
        """
        return kirchhoff_stress
```

译注：上述弹性提示词要求 `forward` 返回两个输出，而原文所附模板只返回应力；此处分别保留.

代码文档字符串说明：初始化函数定义可微优化的可训练连续物理参数，并暂用参数默认值初始化；其中 `param (float)` 表示参数的物理含义. 前向函数以 $(B, 3, 3)$ 变形梯度张量为输入，返回相同形状的 Kirchhoff 应力张量（弹性示例）或修正后的变形梯度张量（塑性示例）. **解答要求.**

1. 根据反馈逐步分析此前迭代的潜在问题，思考先前本构律结果为何与真实值不匹配. 不要给优化建议，应聚焦本构律的表述. 以 `### Analysis` 开始本节；逐一分析历史中的每次迭代，每个小节以 `#### Iteration N` 开始，其中 N 为索引.

2. 逐步思考本次迭代需要做什么，以及如何将算法分为连续物理参数部分和符号本构律部分. 用伪代码极其详细地描述计划，并根据以往优化更新可训练物理参数的默认值. 以 `### Step-by-Step Plan` 开始本节.

3. 在单个 `python` 代码块中输出代码，块内附详细注释；代码块前后不要添加尾随评论. 以 `### Code` 开始本节.

塑性本构律发现的代码格式提示词：
格式要求：PyTorch 提示.

1. 对两个矩阵逐元素相乘时，操作前确保维数匹配. 例如，将形状为 `(B,)` 的 `J` 与形状为 `(B, 3, 3)` 的 `I` 相乘前，应执行 `J.view(-1, 1, 1)`；同样，`(J - 1)` 也应重塑为 `(J - 1).view(-1, 1, 1)`. 若不确定，逐一写出表达式的每个分量，并在注释中标注维数以供验证.

2. 计算形状为 `(B, 3, 3)` 的张量 A 的迹时，用下列代码计算：

   `A.diagonal(dim1=1, dim2=2).sum(dim=1).view(-1, 1, 1)`.

   避免使用只支持二维矩阵的 `torch.trace` 或 `Tensor.trace`.

代码要求.

1. 编程语言始终为 Python.

2. 每次张量操作后以注释标注张量大小，例如 `# (B, 3, 3)`.

3. 唯一允许的库是 PyTorch. 遵循用户示例并查阅 PyTorch 文档.

4. 将代码分成可通过可微优化调节的连续物理参数，以及以 PyTorch 代码表示的符号变形梯度修正模型；分别在 `__init__` 和 `forward` 中定义.

5. 所提代码必须严格遵循下面的结构和函数签名：

```
import torch
import torch.nn as nn

class Physics(nn.Module):
    def __init__(self, param: float = DEFAULT_VALUE):
        """
        Define trainable continuous physical parameters for differentiable optimization.
        Tentatively initialize the parameters with the default values in args.

        Args:
```

```
            param (float): the physical meaning of the parameter.
        """
        super().__init__()
        self.param = nn.Parameter(torch.tensor(param))

    def forward(self, F: torch.Tensor) -> torch.Tensor:
        """
        Compute corrected deformation gradient from deformation gradient tensor.

        Args:
            F (torch.Tensor): deformation gradient tensor (B, 3, 3).

        Returns:
            F_corrected (torch.Tensor): corrected deformation gradient tensor (B, 3, 3).
        """
        return F_corrected
```

代码文档字符串说明：初始化函数定义可微优化的可训练连续物理参数，并暂用参数默认值初始化；其中 `param (float)` 表示参数的物理含义. 前向函数以 $(B, 3, 3)$ 变形梯度张量为输入，返回相同形状的 Kirchhoff 应力张量（弹性示例）或修正后的变形梯度张量（塑性示例）. **解答要求.**

1. 根据反馈逐步分析此前各次迭代的潜在问题，思考先前本构律结果为何与真实值不匹配；不要给优化建议，聚焦本构律表述. 以 `### Analysis` 开始，逐一分析历史中的每次迭代，每个小节以 `#### Iteration N` 开始.
2. 逐步思考本次迭代需要做什么，以及是否需要塑性来改善性能；记住塑性不是必需的. 如果分析支持引入塑性，思考如何用塑性更新变形梯度. 思考如何将算法分为连续物理参数部分和符号变形梯度修正模型部分. 用伪代码极其详细地描述计划，并依据先前优化更新可训练物理参数的默认值. 以 `### Step-by-Step Plan` 开始.
3. 在单个 `python` 代码块中输出代码，块内附详细注释；代码块前后不要添加尾随评论. 以 `### Code` 开始.

分子设计的代码格式提示词：
代码要求.

1. 编程语言始终为 Python.
2. 每次张量操作后以注释标注张量大小，例如 `# (B, 3, 3)`.
3. 将代码分为 Python 字符串 `SMILES`（描述分子拓扑结构和原子类型）与保存所有原子三维坐标的 `coordinates` 矩阵；这些表示不应包含氢.
4. SMILES 字符串应有效. 运用简化分子线性输入规范知识帮助设计有效字符串.

5. SMILES 表示的原子数不少于 8，即 ≥ 8；尝试生成包含多种原子的分子.

6. 原子的三维坐标不得彼此重叠，即 `coordinates` 矩阵每行都应不同.

7. `coordinates` 的形状为 `(N, 3)`，其中 N 是分子原子数，必须与所提 SMILES 字符串表示的原子数相同. 如下例所示，在注释中写出所定义矩阵的形状及 SMILES 字符串表示的原子数.

8. 离散 SMILES 字符串定义结构且无法通过可微优化调节，因而至关重要. 为发现和评估更多结构，所提 SMILES 字符串应不同于上述所有示例或迭代；这非常重要.

9. 所提代码必须严格遵循以下结构和函数签名：

```
SMILES: str # N atoms
coordinates: list[list[float]] # (N, 3)
```

解答要求.

1. 根据反馈逐步分析此前各次迭代的潜在问题，思考先前分子结构结果为何与真实值不匹配；不要给优化建议，应聚焦 SMILES 字符串的构造. 以 `### Analysis` 开始，逐一分析历史中的每次迭代，每个小节以 `#### Iteration N` 开始.

2. 逐步思考本次迭代需要做什么，以及如何将算法分为连续三维坐标部分和离散 SMILES 字符串部分. 所提字符串始终应不同于先前迭代；提出后逐步计算和清点其原子数，连续参数应与原子数相符. 用伪代码极其详细地描述计划，以 `### Step-by-Step Plan` 开始.

3. 在单个 `python` 代码块中输出代码并附详细注释. 在 SMILES 字符串之后逐个计数以计算原子数；记住原子数不少于 8，并尝试生成含多种原子的分子. 代码块前后不要添加尾随评论，以 `### Code` 开始.

C.2 更多说明

C.2.1 数据流程

LLM 的完整输入有三个主要部分：系统提示词、迭代信息和格式提示词. 系统提示词根据 LLM 类型，在开始时插入或作为特殊指令输入. 对迭代信息，我们先拼接代码与反馈，再将前 K 个解简单叠放. 最后，在提示末尾追加格式提示词，以约束预期输出. 实验表明，保持提示顺序对确保性能和成功解析很重要. 更具体地，这一过程可由以下类 Python 代码表示：

```
prompts = []
prompts.append(system_prompt)
for solution in reversed(solutions.topk()):
```

```
    iteration_prompt = solution.code + '\n' + solution.feedback
    prompts.append(iteration_prompt)
prompts.append(format_prompt)
full_prompt = '\n'.join(prompts)
```

C.2.2 与符号回归任务的区别

- 我们的问题聚焦于由损失引导的一般科学发现，是常规回归问题的超集. 在本构律搜索任务中，我们不直接将输入 / 输出对送给方法，而考虑更具挑战性的任务：递归应用生成的本构律，以总体损失作为性能指标. 具体而言，经典符号回归方法给定 $\langle X, y\rangle$ 对求解 $\arg\min_f \|f(X) - y\|$；我们的方法给定 $\langle X, g(f(X))\rangle$ 对求解 $\arg\min_f \|g(f(X))\|$，其中 g 是物理仿真等复杂函数. 容易构造 g 使后一表述涵盖前者，从而证明问题设置的通用性. 我们如此表述问题，是为了反映科学发现中直接监督极为稀缺的更现实场景.

- 本方法支持任意数量的输入变量和输出特征，而大多数符号回归方法 [179] 对输入输出数量有限制. 输入限制严重制约了它们能够解决的任务复杂性；输出限制迫使它们忽略各输出维度之间的结构相关性. 相比之下，基于代码的表征支持多维数组和张量操作，使本方法能够支持任意问题设置.

- 本模型容易适应多学科应用，而传统符号回归方法通常通过硬编码约束和启发式方法纳入领域专家先验 [176]，这些先验有限、依赖领域且难以定制. 本方法建立在以互联网规模数据预训练的 LLM 上，数据涵盖多学科自然语言、数学表达式和代码；因此用户容易通过自然语言指导进行定制，以适应各种应用.

C.3 更多实验

C.3.1 符号回归

与符号回归方法比较的完整结果见表 C.1.

译注：原文表 4.2 与表 C.1 对部分同名方法给出的数值不同；例如 FFX 的 R^2/MSE 分别为 $0.9824/4.5 \times 10^5$ 和 $0.93124/1775263.7$. 两表均保留各自原值.

C.3.2 增加迭代次数

为进一步研究本方法的潜力并为实际使用者消融超参数，我们增加了关于迭代次数（问答循环次数）的研究. 我们将表 4.1 的实验延长至 20 次迭代，性能见表 C.2. 如表所示，迭代次数是对性能有显著影响的决定性超参数. 它对相对简单的任务影响很小，却显著改善了最具挑战性的任务，包括 (b) 和 (c). 对实际使用者而言，将本方法用于自己的任务时，应首先将迭代次数作为最重要的超参数考虑.

方法	R^2 ↑	MSE ↓	MAE ↓	符号解
AIFeynman [176]	0.05105	22814675.8	2520.0	✓
DSR [136]	0.57527	10966411.0	2045.0	✓
BSR [78]	0.66526	8642965.0	1938.6	✓
AdaBoost [155]	0.75058	6439962.9	1777.7	×
GP-GOMEA [182]	0.77734	5749076.4	1580.1	✓
SBP-GP [181]	0.81773	4706077.0	1367.5	✓
LightGBM [82]	0.83368	4294433.7	1129.9	×
XGBoost [24]	0.87775	3156500.5	1109.2	×
MRGP [6]	0.91074	2304682.5	950.5	✓
EPLEX [90]	0.91851	2104070.1	122.2	✓
FFX [120]	0.93124	1775263.7	801.7	✓
MLP	0.98240	454461.5	366.3	×
FEAT [21]	0.98761	319800.6	336.1	✓
DSO [125]	0.99642	92374.9	168.6	✓
Operon [88]	0.99684	81577.9	92.4	✓
SymbolicGPT [179]	0.52333	6862154.7	1680.7	✓
NeSymReS [13]	超过 3 个变量时不适用			✓
T-JSL [99]	超过 2 个变量时不适用			✓
Ours	**0.99901**	**17424.6**	**86.4**	✓

表 C.1: 符号回归. 末列表示结果是否为符号表达式.

#Iterations	**(a)** ↓	**(b)** ↓	**(c)** ↓	**(d)** ↓	**(e)** ↓	**(f)** ↓	**(g)** ↓	**(h)** ↓
5	5.2e-5	2.1e-1	6.0e-2	**1.4e-12**	**1.3e-4**	1.1e-1	5.4e-1	3.6e-5
20	**4.2e-6**	**4.0e-4**	**2.5e-3**	**1.4e-12**	**1.3e-4**	**6.5e-2**	**1.2e-1**	**5.6e-6**
Improvement	+1138.1%	+52400.0%	+2300.0%	0.0%	0.0%	+69.2%	+350.0%	+542.9%

表 C.2: 增加迭代次数. #Iterations 为迭代次数，Improvement 为改进率；(a)–(h) 对应八项任务.

C.4 更多结果

以下给出方法对各任务优化得到的最佳解. 代码采用可编辑转写；解释性文档字符串与注释在下文用中文说明，原有代码操作与数值保持对应. 变量形状中的 B 是批量大小.

C.4.1 本构律发现 (a)

训练曲线给出杨氏模量对数和泊松比 sigmoid 的最佳默认值. 初始化函数定义用于可微优化的可训练连续物理参数，并以前次反馈的最佳值初始化. 前向函数由变形梯度 F 计算 Kirchhoff 应力；输入输出形状均为 $(B, 3, 3)$. 首先将参数转换为实际值，计算 Lamé 参数（μ 为剪切模量，λ 为第一 Lamé 参数）；随后计算变形梯度行列式 J 并重塑批量维度、计算 F 的逆转置，再计算第一 Piola–Kirchhoff 应力的体积部分和结合 neo-Hookean 行为的偏量部分，后者用于描述材料的近不可压缩性质. 最后按原代码计算并返回 Kirchhoff 应力. **译注：**原代码中 `P_vol + P_dev @ F.transpose(1, 2)` 只将偏量项右乘 F^T，与注释所述 PF^T 不一致. 例如取 $F = 2I$、$\lambda = \mu = 1$，该表达式给出 $59I/8$；再令 Q 为绕 z 轴旋转 90° 的矩阵，则得到 $\tau(QF) = 7Q/2 + 31I/8$，不同于 $Q\tau(F)Q^T$. 此观察仅针对该代码片段，并不推断全部实验的结果.

```
import torch
import torch.nn as nn

class Physics(nn.Module):
    # 训练曲线中的最佳值
    DEFAULT_YOUNGS_MODULUS_LOG = 13.03
    DEFAULT_POISSONS_RATIO_SIGMOID = -1.99

    def __init__(self, youngs_modulus_log: float = DEFAULT_YOUNGS_MODULUS_LOG,
                 poissons_ratio_sigmoid: float = DEFAULT_POISSONS_RATIO_SIGMOID):
        """ 定义可微优化的可训练连续物理参数，并用前次反馈的最佳值初始化。"""
        super().__init__()
        # 将参数初始化为可训练参数
        self.youngs_modulus_log = nn.Parameter(
            torch.tensor(youngs_modulus_log))  # 杨氏模量的对数
        self.poissons_ratio_sigmoid = nn.Parameter(
            torch.tensor(poissons_ratio_sigmoid))  # 泊松比的 sigmoid

    def forward(self, F: torch.Tensor) -> torch.Tensor:
        """ 由变形梯度计算 Kirchhoff 应力。

        参数: F, 变形梯度张量 (B, 3, 3)。
        返回: kirchhoff_stress, Kirchhoff 应力张量 (B, 3, 3)。
        """
        # 将参数转为实际值
```

```
        youngs_modulus = self.youngs_modulus_log.exp()  # (1,)
        poissons_ratio = torch.sigmoid(self.poissons_ratio_sigmoid) * 0.49  # (1,)
        # Lamé 参数
        mu = youngs_modulus / (2 * (1 + poissons_ratio))  # 剪切模量 (1,)
        lam = youngs_modulus * poissons_ratio / (
            (1 + poissons_ratio) * (1 - 2 * poissons_ratio))  # 第一 Lamé 参数 (1,)
        # 行列式 J，重塑以用于运算 (B,)
        J = F.det().view(-1, 1, 1)
        # 应力计算所需的 F 逆转置 (B, 3, 3)
        F_invT = F.inverse().transpose(1, 2)
        # 计算第一 Piola-Kirchhoff 应力张量 P (B, 3, 3)：体积部分
        P_vol = lam * (J - 1) * F_invT
        # 偏量部分结合 neo-Hookean 行为，描述材料的近不可压缩性质
        P_dev = mu * (F - (1 / J) * F_invT)
        # 将第一 Piola-Kirchhoff 应力与 F 的转置相乘，计算 Kirchhoff 应力
        kirchhoff_stress = P_vol + P_dev @ F.transpose(1, 2)
        return kirchhoff_stress
```

C.4.2 本构律发现 (b)

γ 的默认值根据第五次迭代的最佳值选取；它作为可训练参数控制软变形修正的缩放. 前向函数对行列式偏离 1 的程度施加比例修正，以将变形梯度引向物理上合理的状态；输入和输出均为 $(B, 3, 3)$. 先计算 J 并以平滑阶跃函数构造偏差条件，结合批量维度准备修正量，扩展单位矩阵；当行列式偏离 1 时，将 F 拉向单位矩阵，返回修正后的变形梯度.

```
import torch
import torch.nn as nn

class Physics(nn.Module):
    def __init__(self, gamma: float = -0.07):  # 第五轮的最佳值
        """ 将 gamma 初始化为控制软变形修正缩放的可训练参数。

        参数：gamma (float)，变形修正的缩放因子。
        """
        super().__init__()
        self.gamma = nn.Parameter(torch.tensor(gamma))  # 初始化 gamma (1,)

    def forward(self, F: torch.Tensor) -> torch.Tensor:
        """ 按行列式偏离 1 的程度施加软修正，以将梯度引向物理合理状态。

        参数：F，变形梯度张量 (B, 3, 3)。
        返回：F_corrected，修正后的变形梯度张量 (B, 3, 3)。
        """
```

```
        # 计算 F 的行列式并据此构造条件 (B,)
        J = torch.det(F)  # (B,)
        # 用平滑阶跃函数作为偏差条件 (B,)
        J_deviation_condition = torch.tanh(J - 1)  # (B,)
        # 考虑批量维度，准备修正 (B,)
        gamma_correction = self.gamma * J_deviation_condition.view(-1, 1, 1)  # (B, 1, 1)
        # 将单位矩阵扩展至批量大小 (B, 3, 3)
        I = torch.eye(3, device=F.device).repeat(F.size(0), 1, 1)  # (B, 3, 3)
        # 行列式偏离 1 时将 F 拉向单位矩阵，以修正 F (B, 3, 3)
        F_corrected = F - gamma_correction * (F - I)  # (B, 3, 3)
        return F_corrected
```

C.4.3 本构律发现 (c)

弹性限值默认值取自上一轮最佳结果，并引入新参数捕捉硬化效应. 两个可训练物理参数分别决定初始屈服强度与硬化速率. 输入和输出变形梯度均为 $(B, 3, 3)$. 代码先通过奇异值分解获得极分解中的旋转与伸长；假设硬化随累积塑性应变线性影响弹性限值，计算假定塑性应变和硬化修正，截断并对角化伸长，构造修正梯度. 最后计算输入与修正梯度的行列式，通过体积比缩放，以保证体积守恒.

```
import torch
import torch.nn as nn

# elastic_limit 默认值取自上轮最佳结果；新增参数捕捉硬化效应
DEFAULT_ELASTIC_LIMIT = 0.92
DEFAULT_HARDENING_FACTOR = 0.1

class Physics(nn.Module):
    def __init__(self, elastic_limit: float = DEFAULT_ELASTIC_LIMIT,
                 hardening_factor: float = DEFAULT_HARDENING_FACTOR):
        """ 定义用于可微优化的可训练连续物理参数。

        参数: elastic_limit (float)，决定初始屈服强度；
              hardening_factor (float)，控制硬化速率。
        """
        super().__init__()
        self.elastic_limit = nn.Parameter(torch.tensor(elastic_limit))  # ()
        self.hardening_factor = nn.Parameter(torch.tensor(hardening_factor))  # ()

    def forward(self, F: torch.Tensor) -> torch.Tensor:
        """ 由变形梯度计算修正后的变形梯度。

        参数: F，变形梯度张量 (B, 3, 3)。
        返回: F_corrected，修正后的变形梯度张量 (B, 3, 3)。
```

```
        """
        # 获得极分解的旋转 R 和伸长 S
        U, S, V = torch.svd(F)  # U: (B, 3, 3), S: (B, 3), V: (B, 3, 3)
        R = U @ V.transpose(-2, -1)  # (B, 3, 3)
        # 利用硬化修正 S, 假设硬化随累积塑性应变线性影响弹性限值
        plastic_strain = torch.relu(S - self.elastic_limit)  # 假定的塑性应变
        hardening_adjustment = 1.0 + (self.hardening_factor * plastic_strain)
        S_clamped = torch.min(S, self.elastic_limit * hardening_adjustment)  # 截断 S
        S_corrected = torch.diag_embed(S_clamped)  # (B, 3, 3)
        F_corrected = R @ S_corrected  # 修正后的变形梯度 (B, 3, 3)
        # 保证体积守恒
        J = torch.det(F).view(-1, 1, 1)  # 输入 F 的行列式 (B, 1, 1)
        J_corrected = torch.det(F_corrected).view(-1, 1, 1)  # 修正后的行列式
        volume_ratio = (J / J_corrected) ** (1/3)
        F_corrected = F_corrected * volume_ratio  # 体积守恒的结果
        return F_corrected
```

C.4.4 本构律发现 (d)

默认值 0.0 是依据先前行为的最佳猜测；可训练参数调节趋向近似等体积行为的修正. 前向函数计算变形梯度行列式（体积变化），构造并扩展单位矩阵，计算体积部分 $J^{1/3}I$；将 F 除以 $J^{1/3}$ 得到偏量部分，再以参数调节修正并组合，返回修正后的变形梯度.

```
import torch
import torch.nn as nn

class Physics(nn.Module):
    DEFAULT_VALUE = 0.0  # 基于先前行为的最佳猜测
    def __init__(self, param: float = DEFAULT_VALUE):
        """ 定义可微优化的可训练物理参数；它调节趋向近似等体积行为的修正。"""
        super().__init__()
        self.param = nn.Parameter(torch.tensor([param]))  # 标量调节参数

    def forward(self, F: torch.Tensor) -> torch.Tensor:
        """ 符号变形梯度修正模型。"""
        # 计算变形梯度行列式 (体积变化)
        J = torch.det(F)  # (B,)
        # 计算体积部分 J^(1/3) * I; 将单位矩阵扩展至整个批量
        I = torch.eye(3).to(F.device)  # (3, 3)
        I = I.view(1, 3, 3).expand(F.size(0), -1, -1)  # (B, 3, 3)
        vol_deform = torch.pow(J, 1.0 / 3.0).view(-1, 1, 1) * I  # (B, 3, 3)
        # 计算 F 的偏量部分: F 除以 J^(1/3)
        dev_deform = F / torch.pow(J, 1.0 / 3.0).view(-1, 1, 1)  # (B, 3, 3)
```

```
        # 用 self.param 调节并构造修正项
        correction = self.param * (I - dev_deform)  # (B, 3, 3)
        # 合并体积部分与偏量修正
        F_corrected = vol_deform + correction  # (B, 3, 3)
        return F_corrected  # (B, 3, 3)
```

C.4.5 虚构本构律

两个默认物理参数取自先前迭代: κ 和 μ 分别作为体积模量修正因子及剪切模量修正因子, 均为标量可训练参数. 前向函数首先得到批量大小、扩展的单位矩阵和 Jacobian 行列式 J; 计算并截断体积修正系数, 形成体积修正项. 然后计算 F 的迹和偏量部分, 截断形状修正系数并形成形状修正项. 最后从 F 中减去体积和形状修正, 返回修正后的变形梯度.

```
import torch
import torch.nn as nn

# 根据先前迭代确定的物理参数默认值
DEFAULT_KAPPA = 0.08
DEFAULT_MU = 0.28

class Physics(nn.Module):
    def __init__(self, kappa: float = DEFAULT_KAPPA, mu: float = DEFAULT_MU):
        """ 初始化用于可微优化的连续物理参数 kappa 和 mu。"""
        super().__init__()
        self.kappa = nn.Parameter(torch.tensor(kappa))  # 体积模量修正因子 (标量)
        self.mu = nn.Parameter(torch.tensor(mu))  # 剪切模量修正因子 (标量)

    def forward(self, F: torch.Tensor) -> torch.Tensor:
        """ 由变形梯度张量计算修正后的变形梯度。"""
        B = F.size(0)  # 批量大小 (标量)
        I = torch.eye(3, device=F.device).unsqueeze(0).expand(B, -1, -1)  # (B, 3, 3)
        J = torch.det(F).view(-1, 1, 1)  # Jacobian 行列式 (B, 1, 1)
        # 体积修正因子 (B, 1, 1)
        vol_correction_factor = torch.clamp(self.kappa * (J - 1), min=0.0, max=1.0)
        vol_correction = vol_correction_factor * I  # 体积修正项 (B, 3, 3)
        # 计算 F 的迹, 供形状修正使用 (B, 1, 1)
        trace_F = F.diagonal(dim1=1, dim2=2).sum(dim=1).view(-1, 1, 1)
        dev_F = F - (trace_F / 3) * I  # F 的偏量部分 (B, 3, 3)
        # 形状修正因子 (标量)
        shape_correction_factor = torch.clamp(self.mu, min=0.0, max=1.0)
        shape_correction = shape_correction_factor * dev_F  # 形状修正项
        F_corrected = F - vol_correction - shape_correction  # 修正后的 F
        return F_corrected
```

参考文献

[1] S Mazdak Abulnaga, Esra Abaci Turk, Mikhail Bessmeltsev, P Ellen Grant, Justin Solomon, and Polina Golland. Placental flattening via volumetric parameterization. In International Conference on Medical Image Computing and Computer-Assisted Intervention, pages 39–47. Springer, 2019.

[2] Microsoft Research AI4Science and Microsoft Azure Quantum. The impact of large language models on scientific discovery: a preliminary study using gpt-4. arXiv preprint arXiv:2311.07361, 2023.

[3] Muhammad Rizwan Amjid, Aamir Shehzad, Shahzad Hussain, Muhammad Asim Shabbir, Moaazam Rafiq Khan, Muhammad Shoaib, et al. A comprehensive review on wheat flour dough rheology. Pakistan Journal of Food Sciences, 23(2):105–123, 2013.

[4] OpenAI: Marcin Andrychowicz, Bowen Baker, Maciek Chociej, Rafal Jozefowicz, Bob McGrew, Jakub Pachocki, Arthur Petron, Matthias Plappert, Glenn Powell, Alex Ray, et al. Learning dexterous in-hand manipulation. The International Journal of Robotics Research, 39(1):3–20, 2020.

[5] Anthropic. Introducing the next generation of claude, 2024. URL https://www.anthropic.com/news/claude-3-family.

[6] Ignacio Arnaldo, Krzysztof Krawiec, and Una-May O'Reilly. Multiple regression genetic programming. In Proceedings of the 2014 Annual Conference on Genetic and Evolutionary Computation, pages 879–886, 2014.

[7] Ellen M Arruda and Mary C Boyce. A three-dimensional constitutive model for the large stretch behavior of rubber elastic materials. Journal of the Mechanics and Physics of Solids, 41(2):389–412, 1993.

[8] Faisal As' ad, Philip Avery, and Charbel Farhat. A mechanics-informed artificial neural network approach in data-driven constitutive modeling. International Journal for Numerical Methods in Engineering, 123(12):2738–2759, 2022.

[9] Uri M Ascher and Linda R Petzold. Computer methods for ordinary differential equations and differential-algebraic equations, volume 61. Siam, 1998.

[10] Yohai Bar-Sinai, Stephan Hoyer, Jason Hickey, and Michael P Brenner. Learning data-driven discretizations for partial differential equations. Proceedings of the National Academy of Sciences, 116(31):15344–15349, 2019.

[11] Jernej Barbič and Doug L James. Real-time subspace integration for st. venant-kirchhoff deformable models. ACM transactions on graphics (TOG), 24(3):982–990, 2005.

[12] Gabriel Bender, Pieter-Jan Kindermans, Barret Zoph, Vijay Vasudevan, and Quoc Le. Understanding and simplifying one-shot architecture search. In International conference on machine learning, pages 550–559. PMLR, 2018.

[13] Luca Biggio, Tommaso Bendinelli, Alexander Neitz, Aurelien Lucchi, and Giambattista Parascandolo. Neural symbolic regression that scales. In International Conference on Machine Learning, pages 936–945. Pmlr, 2021.

[14] Alan Wilfred Bishop and David John Henkel. The measurement of soil properties in the triaxial test. 1962.

[15] Daniil A Boiko, Robert MacKnight, Ben Kline, and Gabe Gomes. Autonomous chemical research with large language models. Nature, 624(7992):570–578, 2023.

[16] Ronaldo I Borja. Plasticity, volume 2. Springer, 2013.

[17] Tim Brooks, Bill Peebles, Connor Holmes, Will DePue, Yufei Guo, Li Jing, David Schnurr, Joe Taylor, Troy Luhman, Eric Luhman, Clarence Ng, Ricky Wang, and Aditya Ramesh. Video generation models as world simulators. 2024. URL https://openai.com/research/video-generation-models-as-world-simulators.

[18] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. Advances in neural information processing systems, 33:1877–1901, 2020.

[19] Han Cai, Ligeng Zhu, and Song Han. ProxylessNAS: Direct neural architecture search on target task and hardware. In International Conference on Learning Representations, 2019.

[20] Tianle Cai, Xuezhi Wang, Tengyu Ma, Xinyun Chen, and Denny Zhou. Large language models as tool makers. In International Conference on Learning Representations, 2024.

[21] William La Cava, Tilak Raj Singh, James Taggart, Srinivas Suri, and Jason Moore. Learning concise representations for regression by evolving networks of trees. In International Conference on Learning Representations, 2019.

[22] Hsiao-yu Chen, Edith Tretschk, Tuur Stuyck, Petr Kadlecek, Ladislav Kavan, Etienne Vouga, and Christoph Lassner. Virtual elastic objects. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pages 15827–15837, 2022.

[23] Peter Yichen Chen, Maytee Chantharayukhonthorn, Yonghao Yue, Eitan Grinspun, and Ken Kamrin. Hybrid discrete-continuum modeling of shear localization

in granular media. Journal of the Mechanics and Physics of Solids, 153:104404, 2021.

[24] Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. In Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining, pages 785–794, 2016.

[25] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A simple framework for contrastive learning of visual representations. In International conference on machine learning, pages 1597–1607. PMLR, 2020.

[26] Raj P Chhabra. Bubbles, drops, and particles in non-Newtonian fluids. CRC press, 2006.

[27] Seyone Chithrananda, Gabriel Grand, and Bharath Ramsundar. Chemberta: large-scale self-supervised pretraining for molecular property prediction. arXiv preprint arXiv:2010.09885, 2020.

[28] Benoît Colson, Patrice Marcotte, and Gilles Savard. An overview of bilevel optimization. Annals of operations research, 153:235–256, 2007.

[29] Timothée Darcet, Maxime Oquab, Julien Mairal, and Piotr Bojanowski. Vision transformers need registers. In The Twelfth International Conference on Learning Representations.

[30] Filipe de Avila Belbute-Peres, Kevin Smith, Kelsey Allen, Josh Tenenbaum, and J Zico Kolter. End-to-end differentiable physics for learning and control. Advances in Neural Information Processing Systems, 31, 2018.

[31] Eduardo A de Souza Neto, Djordje Peric, and David RJ Owen. Computational methods for plasticity: theory and applications. John Wiley & Sons, 2011.

[32] Jonas Degrave, Michiel Hermans, Joni Dambre, et al. A differentiable physics engine for deep learning in robotics. Frontiers in neurorobotics, page 6, 2019.

[33] Congyue Deng, Or Litany, Yueqi Duan, Adrien Poulenard, Andrea Tagliasacchi, and Leonidas J Guibas. Vector neurons: A general framework for so (3)-equivariant networks. In Proceedings of the IEEE/CVF International Conference on Computer Vision, pages 12200–12209, 2021.

[34] Daniel Charles Drucker and William Prager. Soil mechanics and plastic analysis or limit design. Quarterly of applied mathematics, 10(2):157–165, 1952.

[35] Tao Du, Kui Wu, Andrew Spielberg, Wojciech Matusik, Bo Zhu, and Eftychios Sifakis. Functional optimization of fluidic devices with differentiable stokes flow. ACM Transactions on Graphics (TOG), 39(6):1–15, 2020.

[36] Tao Du, Josie Hughes, Sebastien Wah, Wojciech Matusik, and Daniela Rus. Underwater soft robot modeling and control with differentiable simulation. IEEE Robotics and Automation Letters, 6(3):4994–5001, 2021.

[37] Tao Du, Kui Wu, Pingchuan Ma, Sebastien Wah, Andrew Spielberg, Daniela Rus, and Wojciech Matusik. Diffpd: Differentiable projective dynamics. ACM Transactions on Graphics (TOG), 41(2):1–21, 2021.

[38] GW Ellis, C Yao, Rui Zhao, and Df Penumadu. Stress-strain modeling of sands using artificial neural networks. Journal of geotechnical engineering, 121(5):429–435, 1995.

[39] Robert Eymard, Thierry Gallouët, and Raphaèle Herbin. Finite volume methods. Handbook of numerical analysis, 7:713–1018, 2000.

[40] Xiaomin Fang, Lihang Liu, Jieqiong Lei, Donglong He, Shanzhuo Zhang, Jingbo Zhou, Fan Wang, Hua Wu, and Haifeng Wang. Geometry-enhanced molecular representation learning for property prediction. Nature Machine Intelligence, 4(2): 127–134, 2022.

[41] Santo Fortunato, Carl T Bergstrom, Katy Börner, James A Evans, Dirk Helbing, Staša Milojević, Alexander M Petersen, Filippo Radicchi, Roberta Sinatra, Brian Uzzi, et al. Science of science. Science, 359(6379):eaao0185, 2018.

[42] Alexander Fuchs, Yousef Heider, Kun Wang, WaiChing Sun, and Michael Kaliske. Dnn2: A hyper-parameter reinforcement learning game for self-design of neural network based elasto-plastic constitutive descriptions. Computers & Structures, 249: 106505, 2021.

[43] YC Fung. Elasticity of soft tissues in simple elongation. American Journal of Physiology-Legacy Content, 213(6):1532–1544, 1967.

[44] Tomonari Furukawa and Genki Yagawa. Implicit constitutive modelling for viscoplasticity using neural networks. International Journal for Numerical Methods in Engineering, 43(2):195–219, 1998.

[45] Chuang Gan, Jeremy Schwartz, Seth Alter, Damian Mrowca, Martin Schrimpf, James Traer, Julian De Freitas, Jonas Kubilius, Abhishek Bhandwaldar, Nick Haber, et al. ThreeDWorld: A platform for interactive multi-modal physical simulation. In Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 1), 2021.

[46] Ming Gao, Xinlei Wang, Kui Wu, Andre Pradhana, Eftychios Sifakis, Cem Yuksel, and Chenfanfu Jiang. Gpu optimization of material point methods. ACM Transactions on Graphics (TOG), 37(6):1–12, 2018.

[47] Moritz Geilinger, David Hahn, Jonas Zehnder, Moritz Bächer, Bernhard Thomaszewski, and Stelian Coros. ADD: Analytically differentiable dynamics for multi-body systems with frictional contact. ACM Trans. Graph., 39(6), nov 2020. ISSN 0730-0301. doi: 10.1145/3414685.3417766. URL https://doi.org/10.1145/3414685.3417766.

[48] Jamshid Ghaboussi, JH Garrett Jr, and Xiping Wu. Knowledge-based modeling of material behavior with neural networks. Journal of engineering mechanics, 117(1): 132–153, 1991.

[49] Oscar Gonzalez and Andrew M Stuart. A first course in continuum mechanics, volume 42. Cambridge University Press, 2008.

[50] Morton E Gurtin, Eliot Fried, and Lallit Anand. The mechanics and thermodynamics of continua. Cambridge University Press, 2010.

[51] David Ha and Jürgen Schmidhuber. Recurrent world models facilitate policy evolution. Advances in neural information processing systems, 31, 2018.

[52] Richard Haberman. Mathematical models: mechanical vibrations, population dynamics, and traffic flow. SIAM, 1998.

[53] Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. Learning latent dynamics for planning from pixels. In International conference on machine learning, pages 2555–2565. PMLR, 2019.

[54] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models. arXiv preprint arXiv:2301.04104, 2023.

[55] David Hahn, Pol Banzet, James M. Bern, and Stelian Coros. Real2Sim: Viscoelastic parameter estimation from dynamic motion. ACM Trans. Graph., 38(6), nov 2019. ISSN 0730-0301. doi: 10.1145/3355089.3356548. URL https://doi.org/10.1145/3355089.3356548.

[56] Thomas A Halgren. Merck molecular force field. i. basis, form, scope, parameterization, and performance of mmff94. Journal of computational chemistry, 17(5-6): 490–519, 1996.

[57] Nicklas Hansen, Hao Su, and Xiaolong Wang. Td-mpc2: Scalable, robust world models for continuous control. arXiv preprint arXiv:2310.16828, 2023.

[58] Nicklas Hansen, Jyothir SV, Vlad Sobal, Yann LeCun, Xiaolong Wang, and Hao Su. Hierarchical world models as visual whole-body humanoid controllers. arXiv preprint arXiv:2405.18418, 2024.

[59] Nikolaus Hansen. The cma evolution strategy: a comparing review. Towards a new evolutionary computation: Advances in the estimation of distribution algorithms, pages 75–102, 2006.

[60] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In Proceedings of the IEEE conference on computer vision and pattern recognition, pages 770–778, 2016.

[61] H Hencky. The elastic behavior of vulcanized rubber. Rubber Chemistry and Technology, 6(2):217–224, 1933.

[62] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (gelus). arXiv preprint arXiv:1606.08415, 2016.

[63] Jonathan Ho and Tim Salimans. Classifier-free diffusion guidance. arXiv preprint arXiv:2207.12598, 2022.

[64] Yining Hong, Li Yi, Joshua B Tenenbaum, Antonio Torralba, and Chuang Gan. PTR: A benchmark for part-based conceptual, relational, and physical reasoning. In Advances In Neural Information Processing Systems, 2021.

[65] Yuanming Hu, Yu Fang, Ziheng Ge, Ziyin Qu, Yixin Zhu, Andre Pradhana, and Chenfanfu Jiang. A moving least squares material point method with displacement discontinuity and two-way rigid body coupling. ACM Transactions on Graphics (TOG), 37(4):1–14, 2018.

[66] Yuanming Hu, Jiancheng Liu, Andrew Spielberg, Joshua B Tenenbaum, William T Freeman, Jiajun Wu, Daniela Rus, and Wojciech Matusik. Chainqueen: A real-time differentiable physical simulator for soft robotics. In 2019 International conference on robotics and automation (ICRA), pages 6265–6271. IEEE, 2019.

[67] Yuanming Hu, Luke Anderson, Tzu-Mao Li, Qi Sun, Nathan Carr, Jonathan Ragan-Kelley, and Fredo Durand. Difftaichi: Differentiable programming for physical simulation. In International Conference on Learning Representations, 2020.

[68] Daniel Z Huang, Kailai Xu, Charbel Farhat, and Eric Darve. Learning constitutive relations from indirect observations using deep neural networks. Journal of Computational Physics, 416:109491, 2020.

[69] Qian Huang, Jian Vora, Percy Liang, and Jure Leskovec. Benchmarking large language models as ai research agents. arXiv preprint arXiv:2310.03302, 2023.

[70] Zhiao Huang, Yuanming Hu, Tao Du, Siyuan Zhou, Hao Su, Joshua B Tenenbaum, and Chuang Gan. PlasticineLab: A soft-body manipulation benchmark with differentiable physics. In International Conference on Learning Representations, 2020.

[71] Thomas JR Hughes. The finite element method: linear static and dynamic finite element analysis. Courier Corporation, 2012.

[72] Stephen James, Paul Wohlhart, Mrinal Kalakrishnan, Dmitry Kalashnikov, Alex Irpan, Julian Ibarz, Sergey Levine, Raia Hadsell, and Konstantinos Bousmalis. Sim-to-real via sim-to-sim: Data-efficient robotic grasping via randomized-to-canonical adaptation networks. In 2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pages 12619–12629, 2019. doi: 10.1109/CVPR.2019.01291.

[73] Wojciech Jarosz, Volker Schönefeld, Leif Kobbelt, and Henrik Wann Jensen. Theory, analysis and applications of 2d global illumination. 31(5), sep 2012. ISSN 0730-0301. doi: 10.1145/2231816.2231823. URL https://doi.org/10.1145/2231816.2231823.

[74] Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. Mixtral of experts. arXiv preprint arXiv:2401.04088, 2024.

[75] Chenfanfu Jiang, Craig Schroeder, Andrew Selle, Joseph Teran, and Alexey Stomakhin. The affine particle-in-cell method. ACM Transactions on Graphics (TOG), 34(4):1–10, 2015.

[76] Chenfanfu Jiang, Craig Schroeder, Joseph Teran, Alexey Stomakhin, and Andrew Selle. The material point method for simulating continuum materials. In ACM SIGGRAPH 2016 Courses, pages 1–52. 2016.

[77] Wengong Jin, Regina Barzilay, and Tommi Jaakkola. Junction tree variational autoencoder for molecular graph generation. In International conference on machine learning, pages 2323–2332. PMLR, 2018.

[78] Ying Jin, Weilin Fu, Jian Kang, Jiadong Guo, and Jian Guo. Bayesian symbolic regression. arXiv preprint arXiv:1910.08892, 2019.

[79] Justin Johnson, Alexandre Alahi, and Li Fei-Fei. Perceptual losses for real-time style transfer and super-resolution. In European conference on computer vision, pages 694–711. Springer, 2016.

[80] Łukasz Kaiser, Mohammad Babaeizadeh, Piotr Miłos, Błażej Osiński, Roy H Campbell, Konrad Czechowski, Dumitru Erhan, Chelsea Finn, Piotr Kozakowski, Sergey Levine, et al. Model based reinforcement learning for atari. In International Conference on Learning Representations, 2019.

[81] George Em Karniadakis, Ioannis G Kevrekidis, Lu Lu, Paris Perdikaris, Sifan Wang, and Liu Yang. Physics-informed machine learning. Nature Reviews Physics, 3(6): 422–440, 2021.

[82] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. Lightgbm: A highly efficient gradient boosting decision tree. Advances in neural information processing systems, 30, 2017.

[83] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3d gaussian splatting for real-time radiance field rendering. ACM Trans. Graph., 42(4): 139–1, 2023.

[84] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. arXiv preprint arXiv:1412.6980, 2014.

[85] Alexander Kirillov, Eric Mintun, Nikhila Ravi, Hanzi Mao, Chloe Rolland, Laura Gustafson, Tete Xiao, Spencer Whitehead, Alexander C Berg, Wan-Yen Lo, et al. Segment anything. In Proceedings of the IEEE/CVF International Conference on Computer Vision, pages 4015–4026, 2023.

[86] Gergely Klár, Theodore Gast, Andre Pradhana, Chuyuan Fu, Craig Schroeder, Chenfanfu Jiang, and Joseph Teran. Drucker-prager elastoplasticity for sand animation. ACM Transactions on Graphics (TOG), 35(4):1–12, 2016.

[87] Dominik K Klein, Mauricio Fernández, Robert J Martin, Patrizio Neff, and Oliver Weeger. Polyconvex anisotropic hyperelasticity with neural networks. Journal of the Mechanics and Physics of Solids, 159:104703, 2022.

[88] Michael Kommenda, Bogdan Burlacu, Gabriel Kronberger, and Michael Affenzeller. Parameter identification for symbolic regression using nonlinear least squares. Genetic Programming and Evolvable Machines, 21(3):471–501, 2020.

[89] Stefan Kramer, Mattia Cerrato, Sašo Džeroski, and Ross King. Automated scientific discovery: From equation discovery to autonomous discovery systems. arXiv preprint arXiv:2305.02251, 2023.

[90] William La Cava, Thomas Helmuth, Lee Spector, and Jason H Moore. A probabilistic and multi-objective analysis of lexicase selection and ε-lexicase selection. Evolutionary Computation, 27(3):377–402, 2019.

[91] William La Cava, Patryk Orzechowski, Bogdan Burlacu, Fabricio de Franca, Marco Virgolin, Ying Jin, Michael Kommenda, and Jason Moore. Contemporary symbolic regression methods and their relative performance. In Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks, 2021.

[92] Lev Davidovich Landau, JS Bell, MJ Kearsley, LP Pitaevskii, EM Lifshitz, and JB Sykes. Electrodynamics of continuous media, volume 8. elsevier, 2013.

[93] Greg Landrum et al. Rdkit: A software suite for cheminformatics, computational chemistry, and predictive modeling. Greg Landrum, 8:31, 2013.

[94] BA Le, Julien Yvonnet, and Q-C He. Computational homogenization of nonlinear elastic materials using neural networks. International Journal for Numerical Methods in Engineering, 104(12):1061–1084, 2015.

[95] Yann LeCun. A path towards autonomous machine intelligence version 0.9. 2, 2022-06-27. Open Review, 62(1):1–62, 2022.

[96] Jiatong Li, Yunqing Liu, Wenqi Fan, Xiao-Yong Wei, Hui Liu, Jiliang Tang, and Qing Li. Empowering molecule discovery for molecule-caption translation with large language models: A chatgpt perspective. arXiv preprint arXiv:2306.06615, 2023.

[97] Tzu-Mao Li, Jaakko Lehtinen, Ravi Ramamoorthi, Wenzel Jakob, and Frédo Durand. Anisotropic gaussian mutations for metropolis light transport through hessian-hamiltonian dynamics. ACM Trans. Graph., 34(6), oct 2015. ISSN 0730-0301. doi: 10.1145/2816795.2818084. URL https://doi.org/10.1145/2816795.2818084.

[98] Tzu-Mao Li, Miika Aittala, Frédo Durand, and Jaakko Lehtinen. Differentiable monte carlo ray tracing through edge sampling. ACM Trans. Graph., 37(6), dec 2018. ISSN 0730-0301. doi: 10.1145/3272127.3275109. URL https://doi.org/10.1145/3272127.3275109.

[99] Wenqiang Li, Weijun Li, Linjun Sun, Min Wu, Lina Yu, Jingyi Liu, Yanjie Li, and Songsong Tian. Transformer-based model for symbolic regression via joint supervised learning. In The Eleventh International Conference on Learning Representations, 2022.

[100] Xuan Li, Yadi Cao, Minchen Li, Yin Yang, Craig Schroeder, and Chenfanfu Jiang. Plasticitynet: Learning to simulate metal, sand, and snow for optimization time integration. Advances in Neural Information Processing Systems, 35:27783–27796, 2022.

[101] Yunzhu Li, Jiajun Wu, Russ Tedrake, Joshua B Tenenbaum, and Antonio Torralba. Learning particle dynamics for manipulating rigid bodies, deformable objects, and fluids. In International Conference on Learning Representations, 2018.

[102] Zijie Li and Amir Barati Farimani. Graph neural network-accelerated lagrangian fluid simulation. Computers & Graphics, 103:201–211, 2022.

[103] Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Fourier neural operator for parametric partial differential equations. arXiv preprint arXiv:2010.08895, 2020.

[104] Junbang Liang, Ming Lin, and Vladlen Koltun. Differentiable cloth simulation for inverse problems. Advances in Neural Information Processing Systems, 32, 2019.

[105] Julia Ling, Andrew Kurzawski, and Jeremy Templeton. Reynolds averaged turbulence modelling using deep neural networks with embedded invariance. Journal of Fluid Mechanics, 807:155–166, 2016.

[106] Burigede Liu, Nikola Kovachki, Zongyi Li, Kamyar Azizzadenesheli, Anima Anandkumar, Andrew M Stuart, and Kaushik Bhattacharya. A learning-based multiscale method and its application to inelastic impact problems. Journal of the Mechanics and Physics of Solids, 158:104668, 2022.

[107] Hanxiao Liu, Karen Simonyan, and Yiming Yang. DARTS: Differentiable architecture search. In International Conference on Learning Representations, 2019.

[108] Minliang Liu, Liang Liang, and Wei Sun. A generic physics-informed neural network-based constitutive model for soft biological tissues. Computer methods in applied mechanics and engineering, 372:113402, 2020.

[109] Risheng Liu, Pan Mu, Xiaoming Yuan, Shangzhi Zeng, and Jin Zhang. A general descent aggregation framework for gradient-based bi-level optimization. IEEE Transactions on Pattern Analysis and Machine Intelligence, 45(1):38–57, 2022.

[110] Zhichao Liu, Ruth A Roberts, Madhu Lal-Nag, Xi Chen, Ruili Huang, and Weida Tong. Ai-based language models powering drug discovery and development. Drug Discovery Today, 26(11):2593–2607, 2021.

[111] Ilya Loshchilov and Frank Hutter. SGDR: Stochastic gradient descent with warm restarts. arXiv preprint arXiv:1608.03983, 2016.

[112] Lu Lu, Pengzhan Jin, and George Em Karniadakis. Deeponet: Learning nonlinear operators for identifying differential equations based on the universal approximation theorem of operators. arXiv preprint arXiv:1910.03193, 2019.

[113] Pingchuan Ma, Tao Du, Joshua B Tenenbaum, Wojciech Matusik, and Chuang Gan. RISP: Rendering-invariant state predictor with differentiable simulation and rendering for cross-domain parameter estimation. In International Conference on Learning Representations, 2021.

[114] Pingchuan Ma, Tao Du, John Z Zhang, Kui Wu, Andrew Spielberg, Robert K Katzschmann, and Wojciech Matusik. DiffAqua: A differentiable computational design pipeline for soft underwater swimmers with shape interpolation. ACM Transactions on Graphics (TOG), 40(4):1–14, 2021.

[115] Pingchuan Ma, Peter Yichen Chen, Bolei Deng, Joshua B Tenenbaum, Tao Du, Chuang Gan, and Wojciech Matusik. Learning neural constitutive laws from motion observations for generalizable pde dynamics. In International Conference on Machine Learning, pages 23279–23300. PMLR, 2023.

[116] Pingchuan Ma, Tsun-Hsuan Wang, Minghao Guo, Zhiqing Sun, Joshua B Tenenbaum, Daniela Rus, Chuang Gan, and Wojciech Matusik. Llm and simulation as bilevel optimizers: A new paradigm to advance physical scientific discovery. In International Conference on Machine Learning. PMLR, 2024.

[117] Yecheng Jason Ma, William Liang, Guanzhi Wang, De-An Huang, Osbert Bastani, Dinesh Jayaraman, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Eureka: Human-level reward design via coding large language models. In International Conference on Learning Representations, 2024.

[118] Miles Macklin. Warp: A high-performance python framework for gpu simulation and graphics. https://github.com/nvidia/warp, March 2022. NVIDIA GPU Technology Conference (GTC).

[119] Agata Marzec, Jolanta Kowalska, Ewa Domian, Sabina Galus, Agnieszka Ciurzyńska, and Hanna Kowalska. Characteristics of dough rheology and the structural, mechanical, and sensory properties of sponge cakes with sweeteners. Molecules, 26(21):6638, 2021.

[120] Trent McConaghy. Ffx: Fast, scalable, deterministic symbolic regression technology. Genetic Programming Theory and Practice IX, pages 235–260, 2011.

[121] Antoine McNamara, Adrien Treuille, Zoran Popović, and Jos Stam. Fluid control using the adjoint method. In ACM SIGGRAPH 2004 Papers, SIGGRAPH '04, page 449–456, New York, NY, USA, 2004. Association for Computing Machinery. ISBN 9781450378239. doi: 10.1145/1186562.1015744. URL https://doi.org/10.1145/1186562.1015744.

[122] Ben Mildenhall, Pratul P Srinivasan, Matthew Tancik, Jonathan T Barron, Ravi Ramamoorthi, and Ren Ng. NeRF: Representing scenes as neural radiance fields for view synthesis. In European conference on computer vision, pages 405–421. Springer, 2020.

[123] R v Mises. Mechanik der festen körper im plastisch-deformablen zustand. Nachrichten von der Gesellschaft der Wissenschaften zu Göttingen, Mathematisch-Physikalische Klasse, 1913:582–592, 1913.

[124] Joe J Monaghan. Smoothed particle hydrodynamics. Annual review of astronomy and astrophysics, 30:543–574, 1992.

[125] T. Nathan Mundhenk, Mikel Landajuela, Ruben Glatt, Claudio P. Santiago, Daniel M. Faissol, and Brenden K. Petersen. Symbolic regression via neural-guided genetic programming population seeding. In Advances in Neural Information Processing Systems, 2021.

[126] J Krishna Murthy, Miles Macklin, Florian Golemo, Vikram Voleti, Linda Petrini, Martin Weiss, Breandan Considine, Jérôme Parent-Lévesque, Kevin Xie, Kenny Erleben, et al. gradsim: Differentiable simulation for system identification and visuomotor control. In International Conference on Learning Representations, 2020.

[127] Merlin Nimier-David, Delio Vicini, Tizian Zeltner, and Wenzel Jakob. Mitsuba 2: A retargetable forward and inverse renderer. ACM Trans. Graph., 38(6), nov 2019. ISSN 0730-0301. doi: 10.1145/3355089.3356498. URL https://doi.org/10.1145/3355089.3356498.

[128] Raymond W Ogden. Non-linear elastic deformations. Courier Corporation, 1997.

[129] OpenAI. OpenAI: Introducing ChatGPT, 2022. URL https://openai.com/blog/chatgpt.

[130] OpenAI. OpenAI: GPT-4, 2023. URL https://openai.com/research/gpt-4.

[131] OptiTrack. OptiTrack motion capture systems. https://optitrack.com/. Accessed: 2021-10-05.

[132] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, et al. Dinov2: Learning robust visual features without supervision. arXiv preprint arXiv:2304.07193, 2023.

[133] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. Advances in neural information processing systems, 35:27730–27744, 2022.

[134] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. Advances in neural information processing systems, 32, 2019.

[135] Xue Bin Peng, Marcin Andrychowicz, Wojciech Zaremba, and Pieter Abbeel. Sim-to-real transfer of robotic control with dynamics randomization. In 2018 IEEE International Conference on Robotics and Automation (ICRA), pages 3803–3810, 2018. doi: 10.1109/ICRA.2018.8460528.

[136] Brenden K Petersen, Mikel Landajuela Larma, Terrell N Mundhenk, Claudio Prata Santiago, Soo Kyung Kim, and Joanne Taery Kim. Deep symbolic regression: Recovering mathematical expressions from data via risk-seeking policy gradients. In International Conference on Learning Representations, 2020.

[137] Tobias Pfaff, Meire Fortunato, Alvaro Sanchez-Gonzalez, and Peter Battaglia. Learning mesh-based simulation with graph networks. In International Conference on Learning Representations, 2020.

[138] Matt Pharr, Wenzel Jakob, and Greg Humphreys. Physically Based Rendering: From Theory to Implementation. Morgan Kaufmann, 2016.

[139] Karl Popper. The logic of scientific discovery. Routledge, 2005.

[140] Yi-Ling Qiao, Junbang Liang, Vladlen Koltun, and Ming Lin. Scalable differentiable physics for learning and control. In International Conference on Machine Learning, pages 7847–7856. PMLR, 2020.

[141] Yi-Ling Qiao, Junbang Liang, Vladlen Koltun, and Ming C Lin. Efficient differentiable simulation of articulated bodies. In International Conference on Machine Learning, pages 8661–8671. PMLR, 2021.

[142] Yiling Qiao, Junbang Liang, Vladlen Koltun, and Ming Lin. Differentiable simulation of soft multi-body systems. Advances in Neural Information Processing Systems, 34: 17123–17135, 2021.

[143] Qualisys. Qualisys motion capture systems. https://www.qualisys.com/. Accessed: 2021-10-05.

[144] Maziar Raissi, Paris Perdikaris, and George Em Karniadakis. Physics informed deep learning (part i): Data-driven solutions of nonlinear partial differential equations. arXiv preprint arXiv:1711.10561, 2017.

[145] Maziar Raissi, Paris Perdikaris, and George Em Karniadakis. Physics informed deep learning (part ii): Data-driven discovery of nonlinear partial differential equations. arXiv preprint arXiv:1711.10566, 2017.

[146] Maziar Raissi, Paris Perdikaris, and George E Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. Journal of Computational Physics, 378:686–707, 2019.

[147] Prajit Ramachandran, Barret Zoph, and Quoc V Le. Searching for activation functions. arXiv preprint arXiv:1710.05941, 2017.

[148] Raghunathan Ramakrishnan, Pavlo O Dral, Matthias Rupp, and O Anatole Von Lilienfeld. Quantum chemistry structures and properties of 134 kilo molecules. Scientific data, 1(1):1–7, 2014.

[149] Ravi Ramamoorthi, Dhruv Mahajan, and Peter Belhumeur. A first-order analysis of lighting, shading, and shadows. 26(1):2–es, jan 2007. ISSN 0730-0301. doi: 10.1145/1189762.1189764. URL https://doi.org/10.1145/1189762.1189764.

[150] Sereina Riniker and Gregory A Landrum. Better informed distance geometry: using what we know to improve conformation generation. Journal of chemical information and modeling, 55(12):2562–2574, 2015.

[151] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M Pawan Kumar, Emilien Dupont, Francisco JR Ruiz, Jordan S Ellenberg, Pengming Wang, Omar Fawzi, et al. Mathematical discoveries from program search with large language models. Nature, pages 1–3, 2023.

[152] Alex Rosenberg and Lee McIntyre. Philosophy of science: A contemporary introduction. Routledge, 2019.

[153] Fereshteh Sadeghi and Sergey Levine. CAD2RL: Real single-image flight without a single real image. In Robotics: Science and Systems, 2017.

[154] Alvaro Sanchez-Gonzalez, Jonathan Godwin, Tobias Pfaff, Rex Ying, Jure Leskovec, and Peter Battaglia. Learning to simulate complex physics with graph networks. In International conference on machine learning, pages 8459–8468. PMLR, 2020.

[155] Robert E Schapire. The boosting approach to machine learning: An overview. Nonlinear estimation and classification, pages 149–171, 2003.

[156] Gisbert Schneider. Automating drug discovery. Nature reviews drug discovery, 17 (2):97–113, 2018.

[157] Gaurav Sharma and Abhishek Thakur. Chatgpt in drug discovery. 2023.

[158] Yuelin Shen, K Chandrashekhara, WF Breig, and LR Oliver. Finite element analysis of v-ribbed belts using neural network based hyperelastic material model. International Journal of Non-Linear Mechanics, 40(6):875–890, 2005.

[159] Ankur Sinha, Pekka Malo, and Kalyanmoy Deb. Evolutionary algorithm for bilevel optimization using approximations of the lower level optimal solution mapping. European Journal of Operational Research, 257(2):395–411, 2017.

[160] Ankur Sinha, Pekka Malo, and Kalyanmoy Deb. A review on bilevel optimization: From classical to evolutionary approaches and applications. IEEE Transactions on Evolutionary Computation, 22(2):276–295, 2017.

[161] Alexey Stomakhin, Russell Howes, Craig A Schroeder, and Joseph M Teran. Energetically consistent invertible elasticity. In Symposium on Computer Animation, volume 1, 2012.

[162] Alexey Stomakhin, Craig Schroeder, Lawrence Chai, Joseph Teran, and Andrew Selle. A material point method for snow simulation. ACM Transactions on Graphics (TOG), 32(4):1–10, 2013.

[163] Alexey Stomakhin, Craig Schroeder, Chenfanfu Jiang, Lawrence Chai, Joseph Teran, and Andrew Selle. Augmented mpm for phase-change and varied materials. ACM Transactions on Graphics (TOG), 33(4):1–11, 2014.

[164] Deborah Sulsky, Shi-Jian Zhou, and Howard L Schreyer. Application of a particle-in-cell method to solid mechanics. Computer physics communications, 87(1-2): 236–252, 1995.

[165] Theodore Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas Griffiths. Cognitive architectures for language agents. Transactions on Machine Learning Research, 2024. ISSN 2835-8856. Survey Certification.

[166] Xiao Sun, Bahador Bahmani, Nikolaos N Vlassis, WaiChing Sun, and Yanxun Xu. Data-driven discovery of interpretable causal relations for deep learning material laws with uncertainty propagation. Granular Matter, 24(1):1–32, 2022.

[167] Andre Pradhana Tampubolon, Theodore Gast, Gergely Klár, Chuyuan Fu, Joseph Teran, Chenfanfu Jiang, and Ken Museth. Multi-species simulation of porous sand and water mixtures. ACM Transactions on Graphics (TOG), 36(4):1–11, 2017.

[168] Jie Tan, Tingnan Zhang, Erwin Coumans, Atil Iscen, Yunfei Bai, Danijar Hafner, Steven Bohez, and Vincent Vanhoucke. Sim-to-real: Learning agile locomotion for quadruped robots. In Robotics: Science and Systems, 2018.

[169] Alexandre M Tartakovsky, Carlos Ortiz Marrero, Paris Perdikaris, Guzel D Tartakovsky, and David Barajas-Solano. Learning parameters and constitutive relationships with physics informed deep neural networks. arXiv preprint arXiv:1808.03398, 2018.

[170] Joseph O Thompson. Hooke's law. Science, 64(1656):298–299, 1926.

[171] Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. In 2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pages 23–30, 2017. doi: 10.1109/IROS.2017.8202133.

[172] LRG Treloar. The elasticity of a network of long-chain molecules. i. Transactions of the Faraday Society, 39:36–41, 1943.

[173] Trieu H Trinh, Yuhuai Wu, Quoc V Le, He He, and Thang Luong. Solving olympiad geometry without human demonstrations. Nature, 625(7995):476–482, 2024.

[174] Cameron Tropea, Alexander L Yarin, John F Foss, et al. Springer handbook of experimental fluid mechanics, volume 1. Springer, 2007.

[175] Clifford Truesdell and Walter Noll. The non-linear field theories of mechanics. In The non-linear field theories of mechanics, pages 1–579. Springer, 2004.

[176] Silviu-Marian Udrescu, Andrew Tan, Jiahai Feng, Orisvaldo Neto, Tailin Wu, and Max Tegmark. Ai feynman 2.0: Pareto-optimal symbolic regression exploiting graph modularity. Advances in Neural Information Processing Systems, 33:4860–4871, 2020.

[177] Dmitry Ulyanov, Andrea Vedaldi, and Victor Lempitsky. Instance normalization: The missing ingredient for fast stylization. arXiv preprint arXiv:1607.08022, 2016.

[178] Kiwon Um, Robert Brand, Yun Raymond Fei, Philipp Holl, and Nils Thuerey. Solver-in-the-loop: Learning from differentiable physics to interact with iterative pde-solvers. Advances in Neural Information Processing Systems, 33:6111–6122, 2020.

[179] Mojtaba Valipour, Bowen You, Maysum Panju, and Ali Ghodsi. Symbolicgpt: A generative transformer model for symbolic regression. arXiv preprint arXiv:2106.14131, 2021.

[180] Vicon. VICON: Award-winning motion capture systems. https://www.vicon.com/. Accessed: 2021-10-05.

[181] Marco Virgolin, Tanja Alderliesten, and Peter AN Bosman. Linear scaling with and within semantic backpropagation-based genetic programming for symbolic regression. In Proceedings of the genetic and evolutionary computation conference, pages 1084–1092, 2019.

[182] Marco Virgolin, Tanja Alderliesten, Cees Witteveen, and Peter AN Bosman. Improving model-based genetic programming for symbolic regression of small expressions. Evolutionary computation, 29(2):211–237, 2021.

[183] Nikolaos N Vlassis and WaiChing Sun. Sobolev training of thermodynamic-informed neural networks for interpretable elasto-plasticity models with level set hardening. Computer Methods in Applied Mechanics and Engineering, 377:113695, 2021.

[184] Nikolaos N Vlassis and WaiChing Sun. Component-based machine learning paradigm for discovering rate-dependent and pressure-sensitive level-set plasticity models. Journal of Applied Mechanics, 89(2), 2022.

[185] Nikolaos N Vlassis and WaiChing Sun. Geometric deep learning for computational mechanics part ii: Graph embedding for interpretable multiscale plasticity. arXiv preprint arXiv:2208.00246, 2022.

[186] Nikolaos N Vlassis, Ran Ma, and WaiChing Sun. Geometric deep learning for computational mechanics part i: anisotropic hyperelasticity. Computer Methods in Applied Mechanics and Engineering, 371:113299, 2020.

[187] Nikolaos N Vlassis, Puhan Zhao, Ran Ma, Tommy Sewell, and WaiChing Sun. Molecular dynamics inferred transfer learning models for finite-strain hyperelasticity of monoclinic crystals: Sobolev training and validations against physical constraints. International Journal for Numerical Methods in Engineering, 2022.

[188] Bin Wang, Yuanmin Deng, Paul Kry, Uri Ascher, Hui Huang, and Baoquan Chen. Learning elastic constitutive material and damping models. In Computer Graphics Forum, volume 39, pages 81–91. Wiley Online Library, 2020.

[189] Hanchen Wang, Tianfan Fu, Yuanqi Du, Wenhao Gao, Kexin Huang, Ziming Liu, Payal Chandak, Shengchao Liu, Peter Van Katwyk, Andreea Deac, et al. Scientific discovery in the age of artificial intelligence. Nature, 620(7972):47–60, 2023.

[190] Kun Wang and WaiChing Sun. A multiscale multi-permeability poroplasticity model linked by recursive homogenizations and deep learning. Computer Methods in Applied Mechanics and Engineering, 334:337–380, 2018.

[191] Tsun-Hsuan Wang, Pingchuan Ma, Andrew Everett Spielberg, Zhou Xian, Hao Zhang, Joshua B Tenenbaum, Daniela Rus, and Chuang Gan. Softzoo: A soft robot co-design benchmark for locomotion in diverse environments. In The Eleventh International Conference on Learning Representations, 2022.

[192] Tsun-Hsuan Johnson Wang, Juntian Zheng, Pingchuan Ma, Yilun Du, Byungchul Kim, Andrew Spielberg, Josh Tenenbaum, Chuang Gan, and Daniela Rus. DiffuseBot: Breeding soft robots with physics-augmented generative diffusion models. Advances in Neural Information Processing Systems, 36, 2024.

[193] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. Advances in Neural Information Processing Systems, 35: 24824–24837, 2022.

[194] David Weininger. Smiles, a chemical language and information system. 1. introduction to methodology and encoding rules. Journal of chemical information and computer sciences, 28(1):31–36, 1988.

[195] Mignon Wuestman, Jarno Hoekman, and Koen Frenken. A typology of scientific breakthroughs. Quantitative Science Studies, 1(3):1203–1222, 2020.

[196] Zhou Xian, Bo Zhu, Zhenjia Xu, Hsiao-Yu Tung, Antonio Torralba, Katerina Fragkiadaki, and Chuang Gan. Fluidlab: A differentiable environment for benchmarking complex fluid manipulation. In The Eleventh International Conference on Learning Representations, 2022.

[197] Hongyi Xu and Jernej Barbič. Example-based damping design. ACM Transactions on Graphics (TOG), 36(4):1–14, 2017.

[198] Hongyi Xu, Funshing Sin, Yufeng Zhu, and Jernej Barbič. Nonlinear material design using principal stretches. ACM Transactions on Graphics (TOG), 34(4):1–11, 2015.

[199] Jie Xu, Tao Chen, Lara Zlokapa, Michael Foshey, Wojciech Matusik, Shinjiro Sueda, and Pulkit Agrawal. An end-to-end differentiable framework for contact-aware robot design. In Robotics: Science and Systems, 2021.

[200] Chao Xue, Xiaoxing Wang, Junchi Yan, Yonggang Hu, Xiaokang Yang, and Kewei Sun. Rethinking bi-level optimization in neural architecture search: A gibbs sampling perspective. In AAAI Conference on Artificial Intelligence, volume 35, pages 10551–10559, 2021.

[201] Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. In International Conference on Learning Representations, 2024.

[202] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik R Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In Conference on Neural Information Processing Systems, 2023.

[203] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In International Conference on Learning Representations, 2023.

[204] Yuan Yin, Vincent Le Guen, Jérémie Dona, Emmanuel de Bézenac, Ibrahim Ayed, Nicolas Thome, and Patrick Gallinari. Augmenting physical models with deep networks for complex dynamics forecasting. Journal of Statistical Mechanics: Theory and Experiment, 2021(12):124012, 2021.

[205] Naruki Yoshikawa, Kei Terayama, Masato Sumita, Teruki Homma, Kenta Oono, and Koji Tsuda. Population-based de novo molecule generation, using grammatical evolution. Chemistry Letters, 47(11):1431–1434, 2018.

[206] Yonghao Yue, Breannan Smith, Christopher Batty, Changxi Zheng, and Eitan Grinspun. Continuum foam: A material point method for shear-dependent flows. ACM Transactions on Graphics (TOG), 34(5):1–20, 2015.

[207] Yonghao Yue, Breannan Smith, Peter Yichen Chen, Maytee Chantharayukhonthorn, Ken Kamrin, and Eitan Grinspun. Hybrid grains: Adaptive coupling of discrete and continuum simulations of granular media. ACM Transactions on Graphics (TOG), 37(6):1–19, 2018.

[208] Lvmin Zhang, Anyi Rao, and Maneesh Agrawala. Adding conditional control to text-to-image diffusion models. In Proceedings of the IEEE/CVF International Conference on Computer Vision, pages 3836–3847, 2023.

[209] Marvin Zhang, Sharad Vikram, Laura Smith, Pieter Abbeel, Matthew Johnson, and Sergey Levine. Solar: Deep structured representations for model-based reinforcement learning. In International conference on machine learning, pages 7444–7453. PMLR, 2019.

[210] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Jeff Huang, Chuyue Sun, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. Efficiently programming large language models using sglang. arXiv preprint arXiv:2312.07104, 2023.

[211] Qinqing Zheng, Amy Zhang, and Aditya Grover. Online decision transformer. In international conference on machine learning, pages 27042–27059. PMLR, 2022.

[212] Gengmo Zhou, Zhifeng Gao, Qiankun Ding, Hang Zheng, Hongteng Xu, Zhewei Wei, Linfeng Zhang, and Guolin Ke. Uni-mol: A universal 3d molecular representation learning framework. In International Conference on Learning Representations, 2023.

[213] Zhenpeng Zhou, Steven Kearnes, Li Li, Richard N Zare, and Patrick Riley. Optimization of molecules via deep reinforcement learning. Scientific reports, 9(1):10752, 2019.

[214] Jun-Yan Zhu, Taesung Park, Phillip Isola, and Alexei A. Efros. Unpaired image-to-image translation using cycle-consistent adversarial networks. In 2017 IEEE International Conference on Computer Vision (ICCV), pages 2242–2251, 2017. doi: 10.1109/ICCV.2017.244.

术语索引

中文术语	英文术语	页码
世界模型	world model	1
神经物理	neural physics	1
可微物理	differentiable physics	1